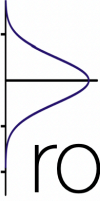


EE5311

Course Notes



Differentiable and Probabilistic Computing

v1.0.0

Mandar Chitre

National University of Singapore

Combining domain knowledge with machine learning leads to a class of algorithms that can solve difficult scientific and engineering problems with only small amounts of training data. At the foundation of these algorithms lie modern computing techniques such as automatic differentiation, Monte-Carlo sampling and variational inference. In this course, we study automatic differentiation and its applications, models such as physics-informed neural networks and neural differential equations to incorporate domain knowledge into machine learning, and probabilistic programming techniques. We also get hands-on experience in applying these techniques to real-world applications.

Table of contents

Preface	4
1 Introduction	6
1.1 Motivation	6
1.2 Approach	8
1.3 Netball-playing Robot	8
1.4 Summary	16
2 Differentiable Programming	18
2.1 Numerical Differentiation	18
2.2 Dual Numbers	25
2.3 Automatic Differentiation	29
2.4 Custom AD rules	41
2.5 Summary	51
3 Physics-guided Machine Learning	52
3.1 Differential Equations	52
3.2 Netball-playing Robot Revisited	56
3.3 Neural Networks	57
3.4 Neural Differential Equations	61
3.5 Physics-informed Neural Networks	71
3.6 Summary	81
4 Probabilistic Programming	83
4.1 Describing Uncertainty	83
4.2 Computing with Uncertainty	85
4.3 Bayesian Inference	94
4.4 Probabilistic Programming Languages	97
4.5 Markov Chain Monte Carlo	103
4.6 Variational Inference	115
4.7 Summary	120
5 Parting Words	121
References	122

Preface

My first exposure to neural networks was in the late 1980s. Neural networks then were mostly simple 3-layer networks with an input layer, a hidden layer and an output layer. Gradients were computed analytically, and the networks were trained using backpropagation. We have come a long way since then! With the proliferation of deep learning, neural networks today have much more sophisticated architectures. It is no longer possible to compute gradients by hand; instead, most modern machine learning techniques leverage automatic differentiation (AD) to automatically compute gradients that are needed for training these networks. While I have been a user of AD since about 2010, it was not until 2020 when the slowdown during COVID'19 gave me the time to study AD in depth. This course is largely an outcome of that exciting journey of discovery.

AD is not just a technique to train neural networks, but something much more powerful. It allows us to compute gradients of sophisticated computer programs, enabling us to effectively include them as part of optimization problems and neural network architectures. By doing so, we can leverage domain knowledge to build systems that learn from very little data. Sometimes we term such systems as *physics-informed*, although the term “physics” here is used much more generally to simply mean “domain knowledge” (in any field). In this course, I will show you many examples of systems that effectively combine data-driven techniques with domain knowledge through the use of AD (also known as *differentiable programming*), and hopefully impress upon you the idea that we don't have to choose between *data-driven* and *physics-driven* – we can combine both effectively using AD.

Real-life applications often need to work with uncertainty. Bayesian inference has been a fundamental theoretical tool to deal with uncertainty. However, its practical application is often limited by computational tractability. The advent of AD changed this. Variational inference and some sophisticated Monte-Carlo sampling techniques such as HMC and NUTS leverage AD to apply Bayesian inference to complicated real-life applications. In this course, I will also introduce you to *probabilistic programming* – a way of describing a problem in terms of probability distributions, and then applying Bayesian inference to solve it.

There are many tools that you can use for differentiable programming and probabilistic programming. Many of you may be familiar with Python, and there certainly are many mainstream tools in Python that you could use. I used Python heavily for my scientific computing and machine learning work from 2015-2020. In 2019, I discovered Julia – an upcoming scientific computing language that was designed from the ground up to be very fast, yet high-level and expressive. And in 2020, when I started studying AD in depth, I discovered that Julia has

an excellent ecosystem for AD, differential equation solving, computing with distributions and Bayesian inference. I now almost exclusively use Julia for all my scientific computing work, and will use it heavily in this course to demonstrate key ideas in differentiable and probabilistic scientific computing.

My goal in this course is to infect you with the same excitement that I felt when I first learnt about differentiable and probabilistic programming, and show you how to apply these techniques in a host of applications. I aim to equip you with a theoretical understanding of the techniques available, and practical experience with the tools necessary to start solving your problems of interest. I'd be grateful if you tell me by the end of the course how successful I was (or wasn't) in achieving these objectives, and what I might want to do differently the next time I teach this course.

1 Introduction

1.1 Motivation

Modeling is key to everything we do. Our brain builds models of the world around us, and uses them to guide all our actions – from low-level control of muscles to higher-level life decisions. For example, when babies learn to crawl or walk, their brain learns a *model* of how muscle actuation relates to motion. When we make decisions such as what to say or do, we typically invoke mental models of how others will react to it to guide us. The brain continuously learns these *models* by observing the world around us.

While the models we use for daily things like walking and talking are learnt implicitly at a very early age, there are other models that we learn explicitly through formal education. For example, most of us learn Newtonian physics in high school. It provides a model for how physical objects interact with their environment. We use such models to predict motion (or the lack of it) and design a variety of things from buildings to rockets. Some of us learn biochemical models of how our body works, and use them to design medical drugs to treat ailments. Others learn financial and economic models and use them to make investment and policy decisions. Models are important to pretty much everything we do, and we learn them either implicitly (through observation) or explicitly (formally defined equations or rules).

Explicit mathematical models are common in many scientific fields such as physics, chemistry, engineering, finance, etc. They are usually derived from a combination of empirical observations (e.g., controlled experiments), understanding of the mechanics of the processes involved, and logic, and then extensively tested in the real world. They are often simplifications of nature, but immensely useful in understanding and predicting how the world behaves. Irrespective of the domain they represent, we term them generically as *physics-based models*. Examples include Newtonian physics, quantum physics, fluid dynamics, stoichiometric models, biochemical models, lumped-circuit models, economic demand-supply models, etc. Physics-based models usually have a small number of *parameters* that have to be measured or inferred from data, and once these parameters are determined, the models provide predictive capability that can be harnessed for decision making.

When the processes involved are poorly understood or too complex for direct mathematical treatment, *statistical models* can be derived from empirical data. Such models draw on the statistical correlation between quantities, without necessarily knowing the mechanics of the processes involved. The most common statistical model is *regression*, where the model learns the relationship between a quantity of interest and various quantities that can be measured

or known otherwise. While traditional statistical models have been studied for decades and successfully applied in many domains, modern machine learning techniques have taken statistical modeling to a new level. Machine learning models often have thousands to millions of parameters, which once properly *trained*, allow the model to make accurate predictions. The training process involves determining the parameters from empirical data – when the number of parameters is large, the amount of data needed for the training also becomes large. Machine learning is therefore known to be *data-hungry*. While training data is easy to come by in some domains, others struggle to apply machine learning due to lack of good quality training data. In these course notes, we refer to traditional statistical models and modern machine learning models together as *data-driven models*.

Both, physics-based modeling and data-driven models, have their strengths and limitations. Physics-based models can learn from very little training data (some do not even require any), and extrapolate well, i.e., make good predictions in regimes beyond where training data is available. The parameters in physics-based model can be related to the mechanics of the processes involved, and so these models are *interpretable*. However, physics-based modeling requires good domain knowledge of the processes involved, and the models can sometimes be computationally expensive. On the other hand, data-driven models learn from empirical data and do not need detailed understanding of the underlying processes. They tend to be data-hungry, interpolate well but often extrapolate poorly, and are usually hard to interpret.

Wouldn't it be nice if we could combine the strengths of both, physics-based modeling techniques and data-driven modeling techniques, to create new modeling techniques that can learn from domain knowledge of the processes involved and empirical data? The field of study that focuses on the development and application of such techniques is known by many names – scientific machine learning (SciML) (Baker et al. 2019; Rackauckas et al. 2021), physics-guided or physics-informed machine learning (Karniadakis et al. 2021; Raissi, Perdikaris, and Karniadakis 2019), theory-guided data science (Karpadne et al. 2017), etc. The field, while relatively new, has already had success in many domains. Some examples:

- Engineering and environmental systems (Willard et al. 2022)
- Geosciences (Karpadne et al. 2019; Arrowsmith et al. 2022; Bergen and Beroza 2018)
- Computational fluid dynamics (Raissi, Perdikaris, and Karniadakis 2019)
- Medical imaging (Liu et al. 2021)
- Medicine and drug discovery (Vamathevan et al. 2019)
- Quantum physics (Raissi, Perdikaris, and Karniadakis 2019)
- Shallow-water wave dynamics and plasma acoustics (Raissi, Perdikaris, and Karniadakis 2019)
- Phase separation in alloys (Raissi, Perdikaris, and Karniadakis 2019)

Two key building blocks that power this field are *differentiable programming* (often referred to as automatic differentiation or AD) and *probabilistic programming*. In this course, we dive into both techniques to develop an understanding of how they work and how they can be applied to real-world problems.

1.2 Approach

While the theory underlying differentiable programming and probabilistic programming is inherently mathematical, we will take a lighthearted approach to it. We won't shy away from the mathematics, but we won't delve into rigorous theorems and proofs either. The objective of the course is to develop sufficient understanding to apply the theory to real-world problems. In doing so, we will require not only an understanding of mathematical theory, but also practical implementation as computer programs.

Since the course is designed to be application-focused, we will study numerous code examples to develop a practical understanding of how to apply the theory. The code examples will be in the [Julia programming language](#), as it is ideally suited for differentiable and probabilistic programming. Julia has a rich open source ecosystem of tools around [SciML](#), and we will leverage many of them in our study. For those of you who are Python programmers, you'll find many similar tools available in the Python ecosystem and should be able to apply most of what we learn in this course using those too. However, I would still encourage you to learn Julia and follow along with the examples in the notes, as the Julia ecosystem is much better suited to our needs and leads to cleaner, more elegant, and simpler-to-understand code.

In the next section, we start by exploring a problem that will form the basis of many examples throughout the course. We'll introduce it as a *toy problem* to keep things simple at first, but gradually increase the level of sophistication as we build expertise in the tools needed to tackle complexities in the problem.

Let's get cracking...

1.3 Netball-playing Robot

1.3.1 Problem statement

Let's say we're building a Netball-playing robot. When designated as the *goal shooter*, the robot has to score a goal by throwing the ball to drop through a *scoring hoop* about 3 m above the ground. The shot has to be taken from within 5 m of the hoop. Our task is to write a program to determine the initial angle and speed at which the robot should shoot the ball for it to drop through the hoop.

1.3.2 Estimating the trajectory

Let us start by assuming that the robot knows its distance from the scoring hoop accurately, and knows how to point exactly in the direction of the hoop for the throw. We can craft this problem as a two-dimensional shooting problem (illustrated in [Figure 1.1](#)), where given the distance d (in meters) to the scoring hoop, our program has to output initial angle θ (in

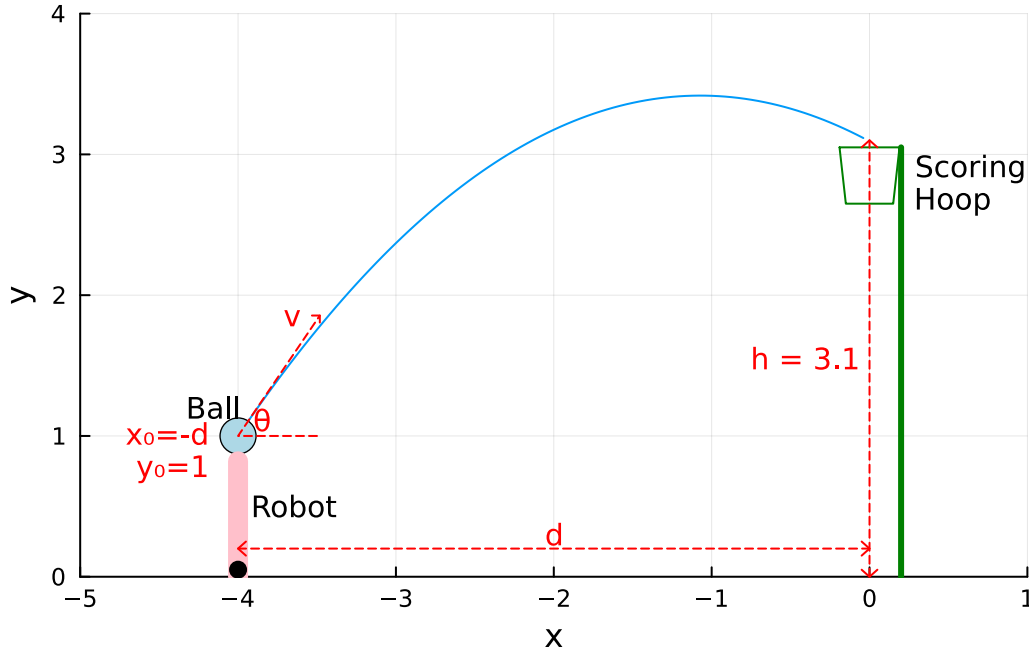


Figure 1.1: A schematic illustration of Netball-playing robot problem.

degrees) and speed v (in m/s) at which to shoot. The robot's shooting apparatus is at $y_0 = 1$ m height from the ground, and can accept $0 < \theta < 90$ and $0 < v \leq 15$. For the ball to fall through the hoop, we're told that the ball must have a downward velocity when it is at $h = 3.1$ m height from the ground right above the hoop¹.

To keep things simple, let's start with assuming no air resistance and otherwise ideal conditions, and apply Newton's laws to the problem. The trajectory of the ball is determined by Newton's laws²:

$$\begin{aligned}\ddot{x} &= 0, \\ \ddot{y} &= g,\end{aligned}\tag{1.1}$$

where $g = -9.81 \text{ m/s}^2$ is the gravitational acceleration. The initial position of the ball is $(x_0 = -d, y_0 = 1)$ and the imparted velocity is $(\dot{x}_0 = v \cos \theta, \dot{y}_0 = v \sin \theta)$. With this simple model, the differential equations above are easy to solve for the position of the ball by double integration:

$$\begin{aligned}x(t) &= x_0 + vt \cos \theta, \\ y(t) &= y_0 + vt \sin \theta + \frac{1}{2}gt^2.\end{aligned}\tag{1.2}$$

¹We use a coordinate system with x -axis horizontal and y -axis vertical, with $x = 0$ at the center of the hoop, and $y = 0$ at the ground.

²We use the notation $\dot{q}$ and $\ddot{q}$ to denote the first and second derivatives of quantity q with respect to time t . Furthermore $q(t)$ and q_t denote the value of quantity q at time t .

From this, we see that the ball will reach the position of the hoop ($x = 0$) at time $T = d/(v \cos \theta)$. At this time, the y position and velocity of the ball will be:

$$\begin{aligned} y_T &= y_0 + d \tan \theta + \frac{gd^2}{2v^2 \cos^2 \theta}, \\ \dot{y}_T &= v \sin \theta + \frac{gd}{v \cos \theta}. \end{aligned} \quad (1.3)$$

1.3.3 Iterative control parameter search

We require $y_T = h$ and $\dot{y}_T < 0$ for the ball to be right above the hoop and falling downwards. This can be thought of as a constrained optimization problem to minimize the squared residual $|y_T - h|^2$ subject to constraint $\dot{y}_T < 0$:

$$\begin{aligned} \arg \min_{v, \theta} & \left| y_0 + d \tan \theta + \frac{gd^2}{2v^2 \cos^2 \theta} - h \right|^2, \\ \text{s.t. } & 0 < \theta < 90, \\ & 0 < v \leq 15, \\ & v \sin \theta + \frac{gd}{v \cos \theta} < 0. \end{aligned} \quad (1.4)$$

The squared residual $|y_T - h|^2$ can be thought as a function of parameters (v, θ) . We can plot it (Figure 1.2) for some sample input d to get a feel for the minimization problem.

We see that there are infinitely many possible combinations of (v, θ) that solve the optimization problem in Equation 1.4. To minimize the energy used by the robot, we may want to pick a feasible solution that also minimizes the initial speed v . This can be achieved by minimizing an auxiliary loss function:

$$\mathcal{L}(v, \theta; d) = \left| y_0 + d \tan \theta + \frac{gd^2}{2v^2 \cos^2 \theta} - h \right|^2 + \alpha |v|^2, \quad (1.5)$$

for some small positive α , subject to the same constraints as Equation 1.4.

While a brute-force grid search over parameter combinations (v, θ) to find the minimum may be possible for this simple example, it is generally not computationally feasible when the search dimension is large or the parameter values span a large interval or are unbounded. Instead, minimization problems can be solved by starting with a conservative guess in the feasible region (e.g., $\theta = 0$ and any v) and iteratively updating the guess using gradient descent:

$$\begin{pmatrix} v \\ \theta \end{pmatrix} \leftarrow \begin{pmatrix} v \\ \theta \end{pmatrix} - \gamma \nabla \mathcal{L} \quad (1.6)$$

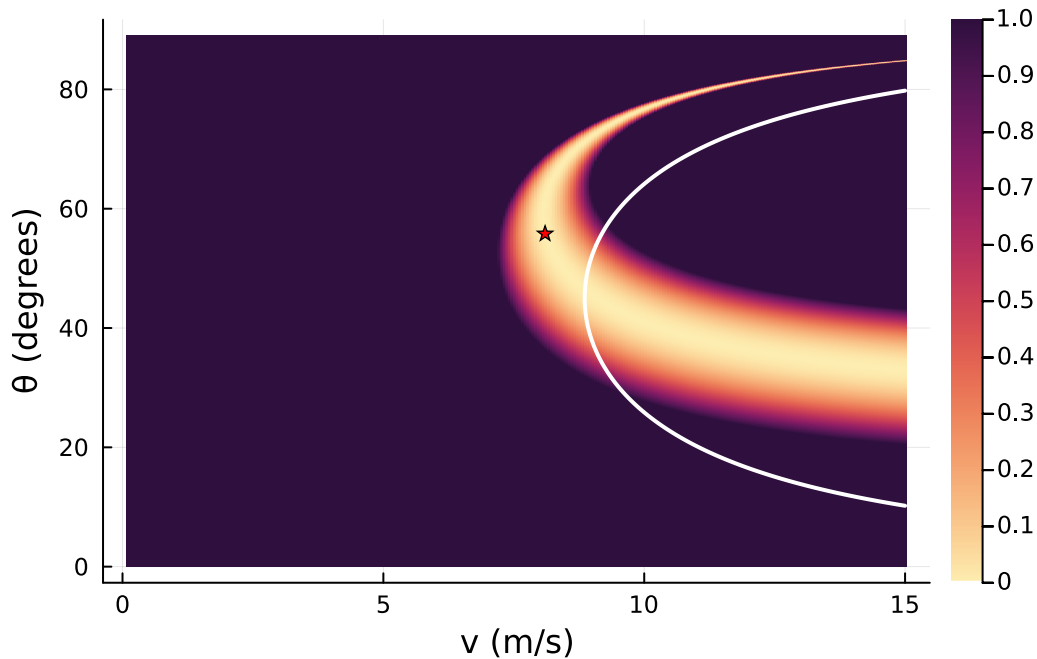


Figure 1.2: The squared residual $|y_T - h|^2$ for $d = 4.0$ m. The white line represents the last constraint in Equation 1.4. The region to the left of the line is the feasible region, while the region to the right is forbidden. The red star denotes an optimal solution where initial speed v is minimized.

for some small step size γ . The step size should be chosen adaptively for fast convergence – e.g., using a line search method (Swann 1969). However, to keep things simple initially, we start with a version with fixed step size and a large number of iterations for convergence.

The gradient $\nabla \mathcal{L} = (\frac{\partial \mathcal{L}}{\partial v}, \frac{\partial \mathcal{L}}{\partial \theta})$ may be computed from Equation 1.5. While we could analytically compute the gradient in this example, this may no longer be possible if our loss function becomes more complicated (as it will when we relax some of the simplifying assumptions we made). We therefore choose to use AD to compute the gradient automatically (we will learn how this works in Chapter 2).

The optimal solution can be obtained by gradient descent with AD (implemented in Julia) as shown below:

```
import ForwardDiff

soln = let α = 0.001, y₀ = 1.0, h = 3.1, d = 4.0, g = -9.81
    ℒ((v, θ)) = abs2(y₀ + d * tand(θ) + (g * d^2) / (2 * v^2 * cosd(θ)^2) - h) + α *
    ↪ abs2(v)
```

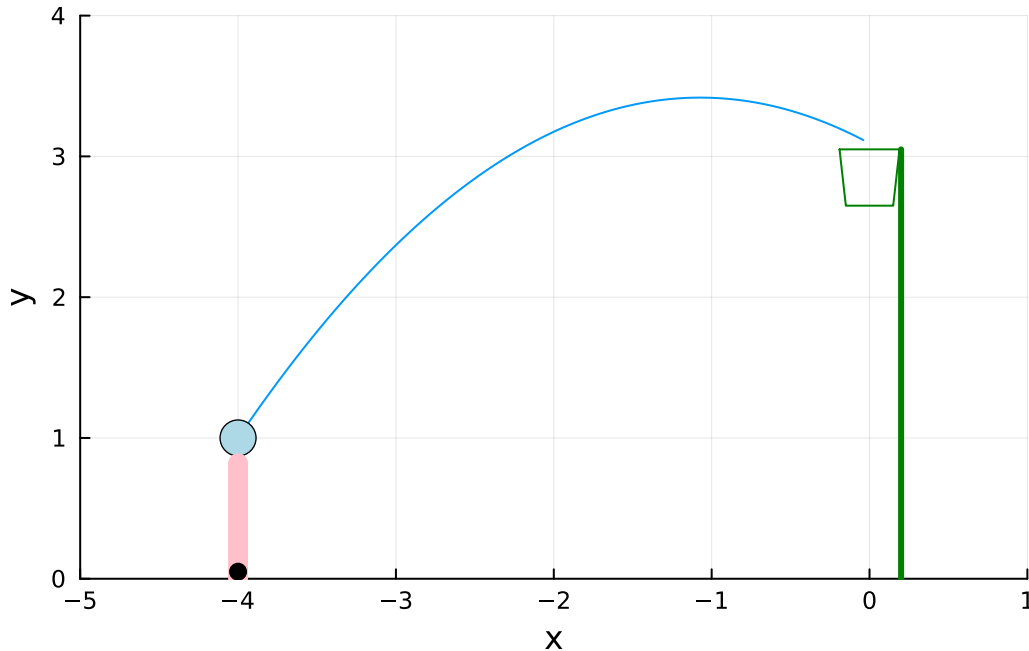


Figure 1.3: Ball trajectory for the optimal solution from Figure 1.2.

```

∇L(vθ) = ForwardDiff.gradient(L, vθ) # gradient using AD
γ = 0.1                               # step size
vθ = [10.0, 0.0]                       # initial guess
for i ∈ 1:1000000                      # lots of iterations
    vθ = vθ - γ * ∇L(vθ)               # gradient descent
end
(v=vθ[1], θ=vθ[2])
end

```

(v = 8.050896836448349, θ = 58.80939398017863)

This solution is shown as a red star in Figure 1.2. The throw represented by this solution is shown in Figure 1.3.

Although we obtained a good solution via gradient descent, it required an unreasonably large number of iterations due to the fixed step size. In practice, we would use a fancier version of gradient descent with line search (e.g., [BFGS](#)). We don't need to implement this from scratch, but instead can use the `Optim.jl` package for this, and ask it to use `ForwardDiff.jl` for AD:

```

using Optim

let  $\alpha = 0.001$ ,  $y_0 = 1.0$ ,  $h = 3.1$ ,  $d = 4.0$ ,  $g = -9.81$ 
 $\mathcal{L}(v, \theta) = \text{abs2}(y_0 + d * \tan(\theta) + (g * d^2) / (2 * v^2 * \cos(\theta)^2) - h) + \alpha * \text{abs2}(v)$ 
soln = optimize( $\mathcal{L}$ , [10.0, 0.0], BFGS(); autodiff=:forward)
println("Number of iterations = ", Optim.iterations(soln))
v,  $\theta$  = Optim.minimizer(soln)
(; v,  $\theta$ )
end

```

Number of iterations = 23

(v = 8.050896839008043, θ = 58.80940075258283)

We see that we get pretty much the same solution as our naïve method, but within just a few iterations.

1.3.4 Neural network to predict control parameters

While the iterative gradient-descent based search does yield good control parameters, its runtime is non-deterministic (and potentially long) as it may take different number of iterations to converge. Could we train a simple neural network (aka multilayer perceptron or MLP) that directly predicts control parameters, rather than perform an iterative search? To train such an MLP, we can use the an augmented loss function similar that in the previous section. However, the loss function will now be a function of the parameters Φ of the MLP that outputs parameters (v, θ) :

$$\mathcal{L}(\Phi; d) = \left| y_0 + d \tan \theta + \frac{gd^2}{2v^2 \cos^2 \theta} - h \right|^2 + \alpha |v|^2, \quad (1.7)$$

with $\begin{pmatrix} v \\ \theta \end{pmatrix} = \text{MLP}(d; \Phi)$.

The MLP training to minimize loss $\mathcal{L}$ requires gradient $\nabla \mathcal{L}$ with respect to MLP parameters Φ . That can be computed using AD.

The resulting machine learning architecture is shown in Figure 1.4.

The equivalent Julia code is shown below:

```

using Lux, Optimisers
import Zygote, Random

```

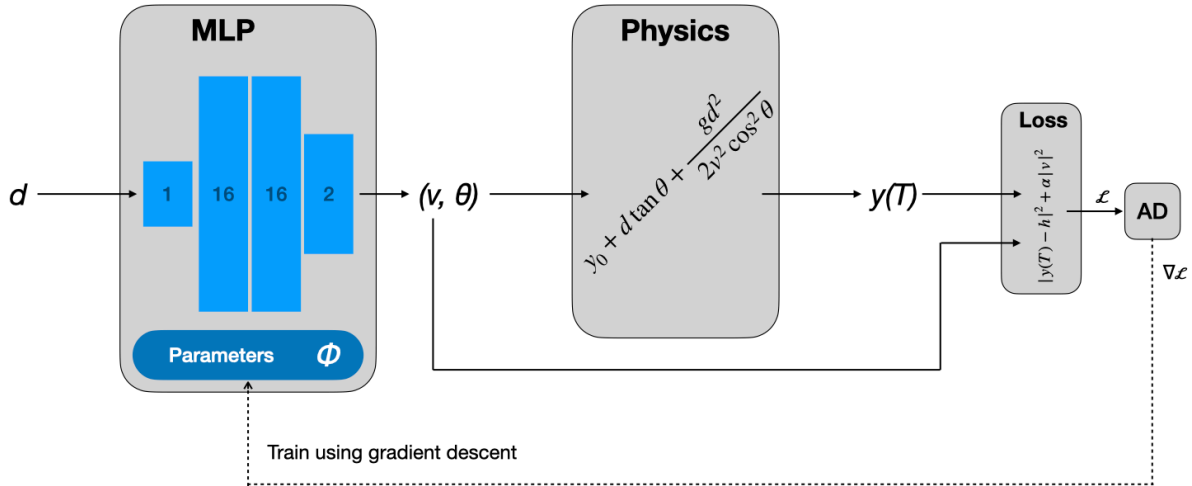


Figure 1.4: Machine learning architecture combining a 3-layer MLP and known physics. Given parameters (v, θ) , the physics predicts the height $y(T)$ of the ball when it reaches the scoring hoop at time T . The MLP learns the inverse by predicting parameters (v, θ) that ensure that the ball is at height h when it reaches the scoring hoop.

```
# neural network
MLP = Chain(
  x → Float32.(x) ./ 5,          # scale input d
  Dense(1 ⇒ 16, tanh),          # input layer with 1 neuron
  Dense(16 ⇒ 16, tanh),         # hidden layer with 16 neurons
  Dense(16 ⇒ 2, σ),             # output layer with 2 neurons
  vθ → hcat(vθ[1,:] .* 15, vθ[2,:] .* 90)' # scale output (v, θ)
)

# initialize MLP parameters ps (and state st)
ps, st = Lux.setup(Random.default_rng(), MLP)

# loss function
function ℒ(ps, ds; α=0.001, y₀=1.0, h=3.1, g=-9.81)
  vθs = Lux.apply(MLP, ds, ps, st)[1] # compute output of MLP
  mapreduce(+, eachcol(vcat(ds, vθs))) do (d, v, θ)
    y = y₀ + d * tand(θ) + (g * d^2) / (2 * v^2 * cosd(θ)^2)
    abs2(y - h) + α * abs2(v) # compute loss
  end
end
```

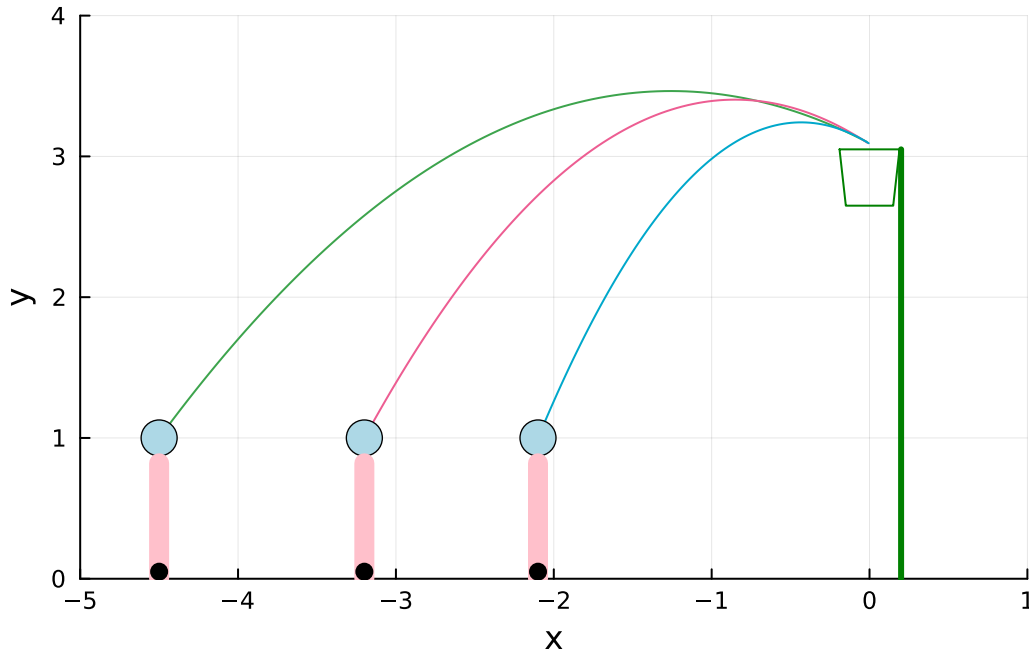


Figure 1.5: Ball trajectories for MLP solutions with $d = 4.5$, $d = 3.2$ and $d = 2.1$ m.

```
# training loop
opt = Optimisers.setup(Adam(0.001f0), ps)
for epoch ∈ 1:5000
    ds = 1 .+ 4 .* rand(Float32, 1, 128) # distances between 1 and 5 m
    gs = Zygote.gradient(L, ps, ds)[1]   # compute gradient wrt ps
    opt, ps = Optimisers.update(opt, ps, gs) # update ps using the gradient
end
```

Don't worry if you don't understand the code fully just yet. We will learn how to train such models in detail in Chapter 3. Once the MLP is trained, we can use it to predict parameters (v, θ) given distance d to the hoop. We visualize a few throws based on parameters predicted by the MLP in Figure 1.5.

On my laptop, the trained MLP computes a solution (v, θ) given distance d in about 650 ns. On the other hand, the iterative solution from Section 1.3.3 takes about 40 μ s to converge. The MLP solution for this simple example is about $60\times$ faster. In many applications, the difference can be much larger.

1.4 Summary

While our netball-playing robot in the previous section was a toy example from an application perspective, it was far from trivial in terms of the techniques we applied to it. Don't worry if you didn't follow all the details – we will explore all the ideas presented so far in much greater in subsequent chapters. The intent here was to give you a flavor of how AD and domain knowledge can be applied together to a problem at hand.

The main takeaways from this example are:

- We developed a physics-based model of the forward problem (trajectory of the ball given initial conditions).
- We posed the inverse problem (determining initial conditions necessary to drop the ball through the hoop) as an optimization problem.
- We noted that the optimization problem can be solved using a brute-force grid search, but this is only feasible when the search space is very small.
- We solved the optimization problem using gradient descent by computing the gradient of the loss function (which incorporates the known physics) using AD. A naïve implementation gives the correct answer, but is computationally expensive.
- We then solved the same optimization problem using BFGS combined with AD. This was computationally much more efficient and worked well for this problem.
- We noted that an iterative search using gradient descent runs in non-deterministic time, which could be undesirable in some situations. We therefore trained a MLP neural network to solve the inverse problem directly. This enabled us to solve the inverse problem in deterministic time without an iterative search. The trained MLP was much faster than the iterative search.
- The training of the MLP required the gradient of the loss function (containing the MLP as well as the physics) with respect to the MLP parameters (weights and biases). We used AD to obtain this gradient.

We saw an example of an inverse problem that can be solved iteratively using optimization techniques, or directly using machine learning. Such inverse problems are very common in practice, where the forward problem is easy to solve but the inverse one is much harder and usually involves some kind of an iterative search. We saw various different ways of solving such a problem, and saw how AD was a useful tool in many of the solutions. In Chapter 2, we study AD in much more detail to understand how it works and what its limitations are.

In the example we studied, the physics was completely known. This allowed us to derive an explicit closed-form solution for the forward problem. Such closed-form solutions are the exception, and not the norm. In fact, in many problems, even the physics is not fully understood. In Chapter 3, we explore ways to deal with situations where closed form solutions are unavailable or the physics is partially known.

The example we studied was also deterministic, i.e., there was no uncertainty about the forward problem. Real-world problems often include some uncertainty. If we had non-ideal conditions with wind and friction and uncertainties in measurement of angles or actuation of the shooting apparatus, we would have to deal with probability distributions in solving the forward and the reverse problem. Probabilistic programming deals with problems of this nature. We will get to solving problems with uncertainty in the last part of this course, in [Chapter 4](#).

2 Differentiable Programming

As we saw in the last chapter, gradients play an important role in modern computing. In many optimization or machine learning applications, we encounter situations where we are presented with an arbitrary computer program that computes some quantity of interest (e.g., a loss function). We are interested in taking the derivative of this program with respect to its parameters (e.g., model parameters). The program may be complicated and large, and may involve multiple subroutines, control flow, loops, recursion, etc. It is generally infeasible to compute the derivative manually or through computer algebra methods such as symbolic differentiation. In this chapter, we study a family of methods to compute derivatives of computer programs.

Roadmap

- We start with **numerical differentiation** based on finite differences – a method that is easy to understand, but only yields approximate derivatives and has scales poorly in high dimensions.
- We then study the theory of **dual numbers**, and see how it can be used to differentiate a computer program exactly.
- Next, we look at differentiation more generally in terms of the chain rule of Calculus. This view leads to forward-mode and reverse-mode ***automatic differentiation (AD)*** methods that form the basis of modern machine learning.
- Along the way, we come across the **Julia API for AD** and a variety of packages that implement different AD methods, each with its strong points and limitations.
- State-of-art AD systems today can automatically take derivatives of many functions, but fail or perform very poorly on some. In the last part of this chapter, we study ways to guide the AD systems past such hurdles by manually writing **custom AD rules**.

2.1 Numerical Differentiation

Numerical differentiation techniques based on finite differences naturally arise from the definition of derivatives in Calculus, and so it is a natural point to start our journey.

2.1.1 Scalar derivatives

Consider a function $f(x) : \mathbb{R} \rightarrow \mathbb{R}$. Assuming it is continuous and differentiable everywhere, we can define a derivative function:

$$f'(x) = \frac{df}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}. \quad (2.1)$$

This definition directly leads to a numerical procedure to approximate the derivative of an arbitrary function:

```
derivative(f, x, h) = (f(x + h) - f(x)) / h
```

The problem, however, is the choice of h . Since the approximation holds in the limit of h being small, a large value of h would be a poor choice. Clearly we cannot set $h = 0.0$ (to avoid a division by zero), so how small should we make it?

Let us explore the accuracy of the approximation as we change the value of h :

```
f(x) = sin(x) + cos(x)    # example function to study
f'(x) = cos(x) - sin(x)   # hand computed derivative

e(x, h) = abs(f'(x) - derivative(f, x, h))
h = 10.0 .^ (-16:0.01:-1)
plot(h, e.(0.2, h);
      xlabel="h", ylabel="|error|", legend=false,
      xscale=:log10, yscale=:log10)
```

In Figure 2.1, we see that there is an optimal value of h for which the approximation has the smallest error. When h reduces below this, the error increases. The increase in error at small values of h is due to amplification of quantization noise in floating point arithmetic when divided by a small number. We can see this by comparing the errors when using 32-bit and 64-bit floating point arithmetic:

```
e(x, h) = abs(f'(x) - derivative(f, x, h))
h = 10.0 .^ (-16:0.01:-1)    # Float64
plot(h, e.(0.2, h);
      xlabel="h", ylabel="|error|", label="Float64",
      xscale=:log10, yscale=:log10)
h = 10f0 .^ (-7:0.01f0:-1)   # Float32
plot!(h, e.(0.2f0, h); label="Float32")
```

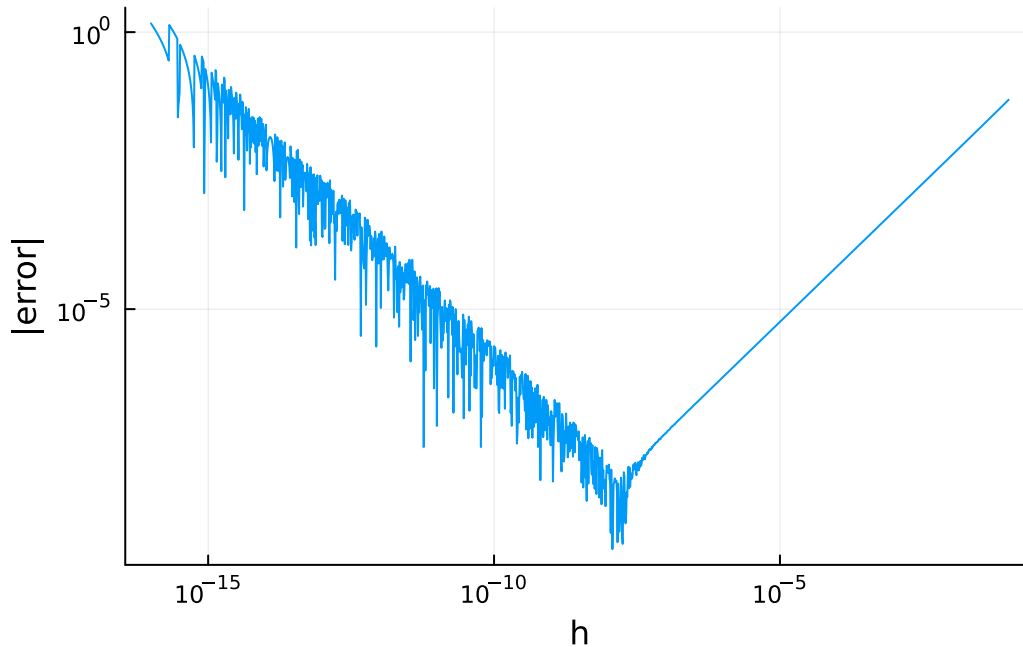


Figure 2.1: Accuracy of a forward finite difference approximation of derivative of $\sin(x) + \cos(x)$.

As we see in Figure 2.2, the smallest error is not only much larger for `Float32`, but the optimal h is also much larger.

The definition in Equation 2.1 is based on forward difference. We can, equivalently, define the derivative in terms of a backward difference:

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x) - f(x - h)}{h}, \quad (2.2)$$

or a central difference:

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x + h) - f(x - h)}{2h}. \quad (2.3)$$

The backward and central difference methods are easy to implement:

```
derivative_backward(f, x, h) = (f(x) - f(x - h)) / h
derivative_central(f, x, h) = (f(x + h) - f(x - h)) / 2h
```

The central difference based implementation is more accurate than the other two:

```
e1(x, h) = abs(f'(x) - derivative(f, x, h))
e2(x, h) = abs(f'(x) - derivative_backward(f, x, h))
```

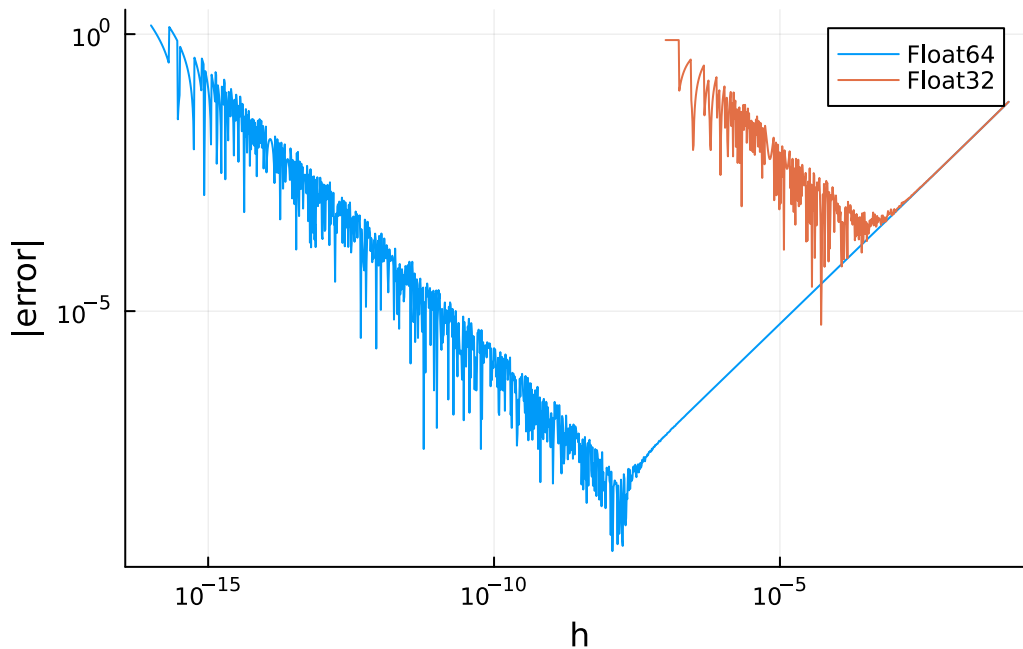


Figure 2.2: Accuracy of a forward finite difference approximation of derivative of $\sin(x) + \cos(x)$ when using 32-bit and 64-bit floating point arithmetic.

```
e3(x, h) = abs(f'(x) - derivative_central(f, x, h))
h = 10.0 .^ (-16:0.01:-1)
plot(h, [e1.(0.2, h) e2.(0.2, h) e3.(0.2, h)];
    xlabel="h", ylabel="|error|",
    label=["Forward" "Backward" "Central"],
    xscale=:log10, yscale=:log10)

# mark theoretical optimal h
vline!([eps(Float64)^(1/3)]; label=nothing, linestyle=:dash, color=:black)
```

We see in Figure 2.3 that the forward and backward finite difference methods have similar performance, but the central difference method yield a lower approximation error, but for a different optimal value of h .

It turns out that the lowest approximation error is obtained when h is the cube-root of machine epsilon (Sauer 2011, p248), which is a measure of the gap between two consecutive numbers that can be represented in finite precision floating point. We can check the value of machine epsilon for 64-bit floating point arithmetic and compute the optimal h :

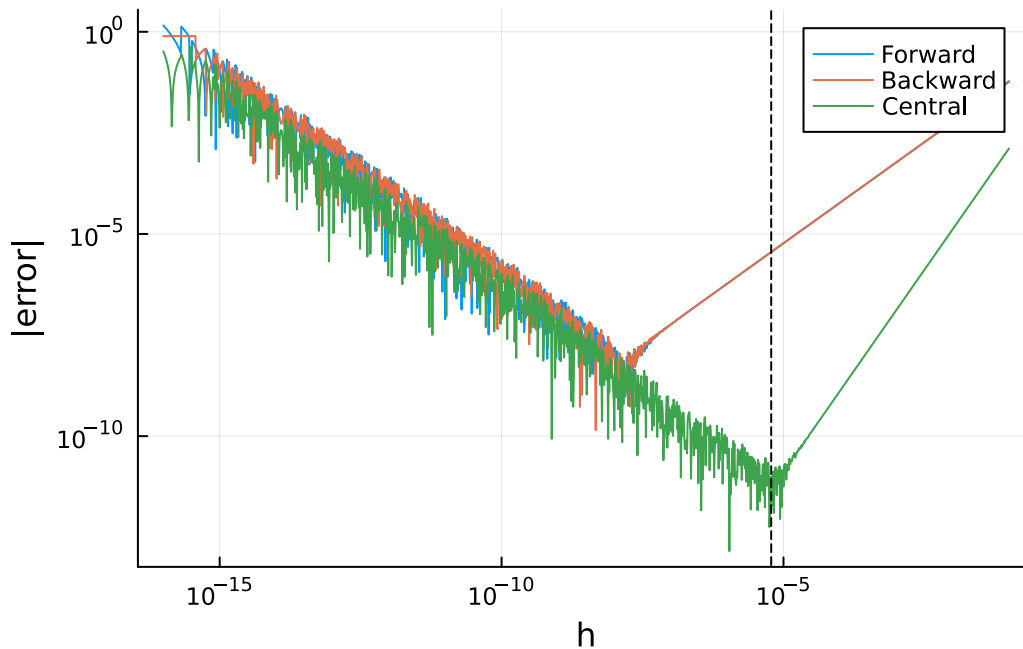


Figure 2.3: Accuracy of a forward finite difference approximation of derivative of $\sin(x) + \cos(x)$ when using forward, backward and central differences. The vertical dashed line marks the theoretical optimal value of h based on (Sauer 2011, p248).

```
eps(Float64), eps(Float64)^(1/3)
```

```
(2.220446049250313e-16, 6.055454452393343e-6)
```

The optimal h value is marked in Figure 2.3. We see that the minimum error indeed occurs very close to the recommended value of h .

More sophisticated algorithms for numerical differentiation exist, but we are not concerned with the details of those algorithms here. We can use well-tested implementations when we need them, with some understanding of their limitations. Packages such as `FiniteDifferences.jl` implement good numerical algorithms and deal with the complexities of selecting h on our behalf, yielding good approximations with minimal effort from us:

```
import FiniteDifferences

## use 5-point central finite differences to compute 1st order derivative
derivative_fd = FiniteDifferences.central_fdm(5, 1)
```

```
## check accuracy
abs(f'(0.2) - derivative_fd(f, 0.2))
```

1.4077627952246985e-13

2.1.2 Multivariate derivatives

We are often concerned with taking derivatives of functions of n -dimensional vectors that yield m -dimensional vectors: $\mathbf{f}(\mathbf{x}) : \mathbb{R}^n \rightarrow \mathbb{R}^m$.

💡 Functions of many variables

Functions of multiple variables of the form $\mathbf{f}(a, b, \dots)$ can be easily transformed into functions of vectors $\mathbf{f}(\mathbf{x})$ by defining a vector $\mathbf{x} = [a, b, \dots]^\top$. Hence, it suffices for us to study derivatives of functions of the form $\mathbf{f}(\mathbf{x})$.

The *Jacobian matrix* summarizes all the first-order partial derivatives of the function:

$$\mathbf{J}_f = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \dots & \frac{\partial f_1}{\partial x_n} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \dots & \frac{\partial f_2}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \frac{\partial f_m}{\partial x_2} & \dots & \frac{\partial f_m}{\partial x_n} \end{bmatrix},$$

where f_i refers to the i^{th} component of the vector output of function $\mathbf{f}$, and x_i is the i^{th} component of vector $\mathbf{x}$.

i Gradient

For the special case of $m = 1$, i.e., a scalar function of a vector: $f(\mathbf{x}) : \mathbb{R}^n \rightarrow \mathbb{R}$, the corresponding Jacobian matrix has only one row, and is therefore a row vector. The transpose of this row vector is known as the *gradient* of $f(\mathbf{x})$, as is usually written as ∇f . Therefore:

$$\nabla f = \mathbf{J}_f^\top.$$

Since a Jacobian matrix simply contains partial derivatives of one scalar output with respect to one scalar input at a time, each matrix entry can be computed using numerical methods outlined in Section 2.1.1. Computing a full Jacobian matrix therefore takes $\mathcal{O}(mn)$ evaluations of function $f(\mathbf{x})$. For applications where m and n are small, this may provide a feasible approach to approximating the Jacobian matrix. However, in many machine learning applications we

are interested in the gradient of the loss function with respect to model parameters. While the loss function yields a scalar loss (i.e. $m = 1$), the number of parameters (n) can be very large, and thus the cost of computing the gradient prohibitive.

Example

Consider the following $\mathbb{R}^2 \rightarrow \mathbb{R}^2$ function:

```
f(x) = [2sin(x[1])+cos(x[2]), cos(x[1])-sin(x[2])]
```

and its hand-derived Jacobian:

```
J(x) = [
    2cos(x[1])  -sin(x[2])
    -sin(x[1])  -cos(x[2])
]
```

We use finite differences to get a numerical Jacobian:

```
J_central(x) = FiniteDifferences.jacobian(derivative_fd, f, x)[1]
```

and compare the results against the hand-derived Jacobian:

```
J([0.2, 0.3]) - J_central([0.2, 0.3])
```

```
2×2 Matrix{Float64}:
 1.15685e-13  4.10227e-14
-9.24261e-15 -1.42109e-13
```

As expected, we see a similar level of accuracy for each entry as the scalar numerical derivatives in section Section 2.1.1. While an accuracy of about 10^{-13} might seem reasonable, the accuracy drops significantly about 10^{-7} (or a little worse) when working with 32-bit floating point arithmetic:

```
J([0.2f0, 0.3f0]) - J_central([0.2f0, 0.3f0])
```

```
2×2 Matrix{Float32}:
-2.38419f-7  8.9407f-8
 1.49012f-7  3.57628f-7
```


32-bit floating point numbers are very commonly used in modern machine learning implementations as they are natively supported by most GPUs today, and they are more memory efficient than 64-bit floating point numbers.

💡 Numerical Differentiation – Summary

Pros

- Follows from first principles and easy to understand.
- Simple to implement.
- Works with all *pure* functions¹.

Cons

- Poor accuracy, especially with 32-bit floating point operations.
- Scales poorly as it requires $\mathcal{O}(mn)$ evaluations of the function.

2.2 Dual Numbers

While numerical differentiation techniques enable approximation of derivatives of arbitrary computer programs easily, their accuracy is limited. Furthermore, they require $\mathcal{O}(mn)$ function evaluations to compute the Jacobian matrix for function $f(\mathbf{x}) : \mathbb{R}^n \rightarrow \mathbb{R}^m$.

AD methods aim to solve these problems. The first step in our quest to understand AD methods is to learn the theory of *dual numbers*, which offers us a way to obtain exact derivatives of a function. A dual number represents an infinitesimal perturbation to a number. When a function is evaluated with a dual number as an input, the mathematics of dual numbers faithfully propagates the perturbation through each step of the function and yields a perturbed output. By definition, the ratio of the output perturbation to input perturbation is the derivative of the function.

2.2.1 Theory

Dual numbers are a little like complex numbers, in the sense that they have two parts (a real part and an infinitesimal part). A dual number z represents a real number x with a small change $y\epsilon$:

$$z = x + y\epsilon,$$

¹A *pure* function is deterministic and has no side effects. It always yields the same result when called with the same parameters, and does not change the state of the program in any way.

where ϵ is the unit infinitesimal quantity (similar to i for complex numbers). The algebra for dual numbers is derived from the rules of real number algebra with the additional rule that $\epsilon^2 = 0$ (for complex numbers, recall that the equivalent rule was $i^2 = -1$):

$$\begin{aligned}(x_1 + y_1\epsilon) + (x_2 + y_2\epsilon) &= (x_1 + x_2) + (y_1 + y_2)\epsilon, \\ (x_1 + y_1\epsilon) \times (x_2 + y_2\epsilon) &= x_1x_2 + (x_1y_2 + x_2y_1)\epsilon.\end{aligned}$$

Recall from Calculus (Taylor series) that for $\epsilon \rightarrow 0$:

$$f(x + \epsilon) = f(x) + \epsilon \left. \frac{\partial f}{\partial x} \right|_x + \mathcal{O}(\epsilon^2).$$

The higher order terms $\mathcal{O}(\epsilon^2)$ can be neglected since $\epsilon \rightarrow 0$. If we view the argument of $x + \epsilon$ as a dual number, we see that the right hand side of the above expression is also a dual number with $f(x)$ as the real part and $\left. \frac{\partial f}{\partial x} \right|_x$ as the infinitesimal part!

💡 Takeaway

The evaluation of $f(z)$ for $z = x + 1\epsilon$ yields a dual number with $f(x)$ as the real part and the derivative $\left. \frac{\partial f}{\partial x} \right|_x$ as the infinitesimal part.

2.2.2 Implementation

With operator overloading in Julia, we can easily implement dual numbers:

```
# new data type representing a dual number (x + yϵ)
struct Dual{T} <: Number
    x::T
    y::T
end

# arithmetic rule to define how dual numbers are added
(Base.:+)(z1::Dual, z2::Dual) = Dual(z1.x + z2.x, z1.y + z2.y)

# arithmetic rule to define how dual numbers are multiplied
(Base.*)(z1::Dual, z2::Dual) = Dual(z1.x * z2.x, z1.x * z2.y + z1.y * z2.x)

# arithmetic rule to convert a real number x to a dual number (x + 0ϵ)
Base.convert{::Type{Dual{T}}}(x::Real) where T = Dual{T}(x, zero(T))

# promotion rule to say that a computation between a dual number and a real number
```

```
# should be done by first promoting (converting) the real number to a dual number
# and then applying dual number arithmetic
Base.promote_rule(::Type{Dual{T}}, ::Type{R}) where {T<:Real, R<:Real} =
    ⇨ Dual{promote_type(T,R)}
```

Let's test if the theory works:

```
f(x) = 3x*x + 2x + 1
f(Dual(0.2, 1.0))
```

```
Dual{Float64}(1.52, 3.2)
```

Tip

Note that we can evaluate $f(x)$ for a x being a `Dual{Float64}` only because $f(x)$ was defined generically for any x and not specialized to a particular data type (e.g., $f(x::Float64) = 3x*x + 2x + 1$). It is considered good practice in Julia to avoid excessive specialization for this reason.

If the theory is right (and it is!), the first entry in the tuple shown above should be $f(0.2)$ and the second entry should be its derivative $f'(0.2)$. Compare the output above against the function value evaluated directly, and its numerical derivative:

```
f(0.2), derivative_fd(f, 0.2)
```

```
(1.52, 3.1999999999999957)
```

The exact derivative of $f(x) = 3x^2 + 2x + 1$ is $f'(x) = 6x + 2$. When evaluated at $x = 0.2$, we have $f'(0.2) = 3.2$. We see that the dual number derivative is exact (to machine precision), whereas the numerical derivative has a larger error (as expected).

A single evaluation of the scalar function $f(\cdot)$ using a dual number input $z = x + 1\epsilon$ yields the value of the function $f(x)$ as well as its derivative with respect to x . With a little more programming effort (to override a few more primitives and add more promotion rules), dual numbers can be implemented to allow fairly complicated Julia functions² to be differentiated automatically.

²For a Julia function to be evaluated using dual numbers without any change, the function must not restrict its input types unnecessarily. Furthermore it should be written in a generic way by avoiding too many assumptions about input data types. This style of programming is considered [idiomatic in Julia](#).

AD using dual numbers is implemented in `ForwardDiff.jl`, and can be used without having to implement dual numbers ourselves:

```
import ForwardDiff

### evaluate f with dual number 0.2 + 1e
f(ForwardDiff.Dual(0.2, 1.0))
```

`Dual{Nothing}(1.52,3.2)`

If we only needed the derivative, `ForwardDiff.jl` provides a convenience API:

```
### evaluate the derivative of f at 0.2 using dual numbers
ForwardDiff.derivative(f, 0.2)
```

3.2

In case of functions with multiple inputs (or a vector input), derivatives have to be evaluated for each input element separately (by passing in a dual number for that input element). `ForwardDiff.jl` will do that for you automatically (but at a computational cost):

```
f(x) = [2sin(x[1])+cos(x[2]), cos(x[1])-sin(x[2])]

ForwardDiff.jacobian(f, [0.2, 0.3])
```

```
2×2 Matrix{Float64}:
 1.96013  -0.29552
-0.198669 -0.955336
```

Compare this against the exact derivative we obtained by hand differentiation in Section 2.1.2:

```
J([0.2, 0.3])
```

```
2×2 Matrix{Float64}:
 1.96013  -0.29552
-0.198669 -0.955336
```

If the function output was a scalar, we could use the gradient API:

```
f(x) = 2sin(x[1]) + cos(x[2])  
  
ForwardDiff.gradient(f, [0.2, 0.3])
```

```
2-element Vector{Float64}:  
 1.9601331556824833  
-0.29552020666133955
```

💡 AD using Dual Numbers – Summary

Pros

- Elegant mathematical basis.
- Works with any function that accepts generic number types.
- Scales well for functions with large number of outputs.
- Exact to machine precision.

Cons

- Unable to differentiate functions specialized for concrete input types.
- Scales poorly for functions with large number of inputs.

2.3 Automatic Differentiation

The theory of dual number provides us a way to obtain exact derivatives of certain computer programs. This is one of the many tools in the family of AD techniques. These techniques enable us to compute exact derivatives (to machine precision) at much lower computational cost than numerical differentiation. We study other AD techniques next – these techniques are more general, and help overcome some of the limitations of dual number based AD.

2.3.1 Basic idea

A computer program breaks down a complex computation into a series of primitive computations, slowly transforming the input into the desired output. For example, consider a function:

$$f(\mathbf{x}) = 2\sin(x_1) + \cos(x_2).$$

It transforms the 2-dimensional vector $\mathbf{x}$ into the scalar output in a series of primitive steps³:

$$\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \xrightarrow{\mathbf{f}_a} \begin{bmatrix} \sin(x_1) \\ \cos(x_2) \end{bmatrix} \xrightarrow{\mathbf{f}_b} \begin{bmatrix} 2 \sin(x_1) \\ \cos(x_2) \end{bmatrix} \xrightarrow{f_c} 2 \sin(x_1) + \cos(x_2).$$

If the derivatives of the primitive functions $\mathbf{f}_a$, $\mathbf{f}_b$ and f_c are known, the resultant derivative of f can be computed using the chain rule of Calculus:

$$\begin{aligned} f(\mathbf{x}) &= f_c(\mathbf{f}_b(\mathbf{f}_a(\mathbf{x}))) \\ \therefore \nabla f(\mathbf{x}) &= \mathbf{J}_f^\top = \left(\mathbf{J}_{f_c} \mathbf{J}_{\mathbf{f}_b} \mathbf{J}_{\mathbf{f}_a} \right)^\top. \end{aligned}$$

The derivatives of primitive functions are provided to the AD system as a set of rules. This allows it to compute $\mathbf{J}_{\mathbf{f}_a}$, $\mathbf{J}_{\mathbf{f}_b}$ and $\mathbf{J}_{f_c}$ easily. In our example, $\mathbf{f}_a$ transforms a 2-dimensional vector by taking the sine of the first term and cosine of the second term. The derivatives of sine and cosine primitives are known to the AD system. Hence it can compute the Jacobian of $\mathbf{f}_a$:

$$\mathbf{J}_{\mathbf{f}_a} = \begin{bmatrix} \cos(x_1) & 0 \\ 0 & -\sin(x_2) \end{bmatrix}.$$

$\mathbf{f}_b$ transforms its input 2-dimensional vector by multiplying the first element by 2 and leaving the second element untouched. Since the derivative of a multiplication by a constant is known to the AD system, it can compute the Jacobian of $\mathbf{f}_b$:

$$\mathbf{J}_{\mathbf{f}_b} = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}.$$

And finally f_c transforms its input 2-dimensional vector to a scalar by summing both components. The AD system knows the derivative of a sum, and hence computes the Jacobian of f_c :

$$\mathbf{J}_{f_c} = \begin{bmatrix} 1 & 1 \end{bmatrix}.$$

Multiplying the Jacobians of the primitive functions, we get the Jacobian of f :

$$\begin{aligned} \mathbf{J}_f &= \begin{bmatrix} 1 & 1 \end{bmatrix} \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} \cos(x_1) & 0 \\ 0 & -\sin(x_2) \end{bmatrix} \\ &= \begin{bmatrix} 2 \cos(x_1) & -\sin(x_2) \end{bmatrix}. \end{aligned}$$

Hence:

$$\nabla f(\mathbf{x}) = \mathbf{J}_f^\top = \begin{bmatrix} 2 \cos(x_1) \\ -\sin(x_2) \end{bmatrix}.$$

While the function in this illustrative example decomposed into a chain of only 3 primitive functions, in practice, the computational chain for a computer program of interest might have

³The function $\mathbf{f}_a$ could have been further decomposed into two steps: (1) the sine of the first element is taken and the second element is left unchanged, and (2) the first element is left unchanged and the cosine of the second element is taken. To keep the presentation in this section concise, we have combined the steps. However, you are encouraged to work through the math with the decomposed function for practice later.

thousands of primitive functions and thousands of intermediate variables. Computing and storing large Jacobian matrices at each step and then multiplying them together is often computationally infeasible. However, it is possible to break this computation involving the product of many primitive Jacobians into a sequence of simpler computations, as we see next.

2.3.2 Forward-mode AD

Consider a function $\mathbf{f}(\mathbf{x}) = \mathbf{f}_c(\mathbf{f}_b(\mathbf{f}_a(\mathbf{x})))$ with Jacobian:

$$\mathbf{J}_f = \mathbf{J}_{f_c} \mathbf{J}_{f_b} \mathbf{J}_{f_a}.$$

Let $\mathbf{e}_j$ represent a the j^{th} basis vector, i.e., a unit vector with all zeros except the j^{th} component, which is set to 1. If we multiply $\mathbf{J}_f$ by $\mathbf{e}_j$, we get:

$$\mathbf{J}_f^{(:,j)} = \mathbf{J}_f \mathbf{e}_j = \underline{\underline{\mathbf{J}_{f_c} \mathbf{J}_{f_b} \mathbf{J}_{f_a} \mathbf{e}_j}},$$

where the notation $\mathbf{J}_f^{(:,j)}$ denotes the j^{th} column of $\mathbf{J}_f$. To compute this, we start from right-to-left, first computing the Jacobian-vector product $\mathbf{J}_{f_a} \mathbf{e}_j$. This yields a vector which we shall call $\mathbf{v}_a^{(j)}$. We next compute the Jacobian-vector product $\mathbf{J}_{f_b} \mathbf{v}_a^{(j)}$ to yield vector $\mathbf{v}_b^{(j)}$. We continue computing Jacobian-vector products moving right-to-left in the expression, until all Jacobians are consumed. The resulting vector is $\mathbf{J}_f^{(:,j)}$ – the j^{th} column of $\mathbf{J}_f$:

$$\begin{aligned} \mathbf{v}_a^{(j)} &= \mathbf{J}_{f_a} \mathbf{e}_j \\ \mathbf{v}_b^{(j)} &= \mathbf{J}_{f_b} \mathbf{v}_a^{(j)} \\ \mathbf{J}_f^{(:,j)} &= \mathbf{J}_{f_c} \mathbf{v}_b^{(j)} \end{aligned}$$

One pass through the expression yields one column of the Jacobian. For a function $\mathbb{R}^n \rightarrow \mathbb{R}^m$, we need n passes (with $j \in \{1 \dots n\}$) to compute the full Jacobian $\mathbf{J}_f$.

i Why is it called forward-mode AD?

The right-to-left evaluation of the Jacobian-vector products is considered *forward-mode* because the evaluation happens in the same order as the function value evaluation.

In our example above, to compute the composite function $\mathbf{f}$, we first apply $\mathbf{f}_a$ to $\mathbf{x}$, then apply $\mathbf{f}_b$ to the output, and finally apply $\mathbf{f}_c$. To compute the Jacobian, we follow the same order: we compute the Jacobian-vector product $\mathbf{J}_{f_a} \mathbf{e}_j$ first, then multiply it by $\mathbf{J}_{f_b}$ on the left, and finally by $\mathbf{J}_{f_c}$.

This method only computes one Jacobian-vector product (often just called *jvp*) at a time, thus limiting computational complexity. The output of each computation step is always a vector

(called a *sensitivity* or *partials* vector). This limits memory storage needs. The Jacobian at each step is often very sparse, and the sparsity structure can be exploited to compute the Jacobian-vector product efficiently (and often without the need to explicitly create the Jacobian matrix in memory). The Jacobians of the primitive functions are utilized in the same order as the primitive functions are evaluated in computing the value of the function. Hence this method is known as *forward-mode AD*.

Since n passes are required to compute the Jacobian, forward-mode AD is efficient when n is small.

For each step in the computational chain, a sensitivity vector coming in from the previous step is multiplied by a Jacobian matrix to yield a new sensitivity vector. The function that takes in the sensitivity vector and yields the updated sensitivity vector is called the *pushforward*. The pushforward can be evaluated alongside the *primal* function⁴ at each step, and both the primal output and the output sensitivity is passed on to the next step of the chain.

Dual numbers and forward-mode AD

In Section 2.2, we saw that AD could be performed using dual numbers. The mathematics of dual numbers is equivalent to the forward-mode AD we studied in this section (hence the name `ForwardDiff.jl` for the Julia package that uses dual numbers for AD). The real part of the dual number contains the primal output and the infinitesimal part contains the sensitivity. The overloaded dual number operations for primitives essentially define the pushforwards for those primitives.

Example

Taking our previous example of $f(\mathbf{x}) = 2 \sin(x_1) + \cos(x_2)$, we can walk through the process:

We first start with seed $\mathbf{e}_1 = [1 \ 0]^\top$ to get the first column of the Jacobian:

$$\begin{aligned}\mathbf{v}_a^{(1)} &= \begin{bmatrix} \cos(x_1) & 0 \\ 0 & -\sin(x_2) \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} \cos(x_1) \\ 0 \end{bmatrix}, \\ \mathbf{v}_b^{(1)} &= \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} \cos(x_1) \\ 0 \end{bmatrix} = \begin{bmatrix} 2 \cos(x_1) \\ 0 \end{bmatrix}, \\ \mathbf{J}_f^{(\cdot 1)} &= [1 \quad 1] \begin{bmatrix} 2 \cos(x_1) \\ 0 \end{bmatrix} = 2 \cos(x_1).\end{aligned}$$

⁴The *primal* function is the function that is being differentiated. In case of an evaluation of one step in the computational chain, the primal function is the primitive function representing that step of computation.

Then with seed $\mathbf{e}_2 = [0 \ 1]^\top$ to get the second column of the Jacobian:

$$\begin{aligned}\mathbf{v}_a^{(2)} &= \begin{bmatrix} \cos(x_1) & 0 \\ 0 & -\sin(x_2) \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ -\sin(x_2) \end{bmatrix}, \\ \mathbf{v}_b^{(2)} &= \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ -\sin(x_2) \end{bmatrix} = \begin{bmatrix} 0 \\ -\sin(x_2) \end{bmatrix}, \\ \mathbf{J}_f^{(2)} &= [1 \ 1] \begin{bmatrix} 0 \\ -\sin(x_2) \end{bmatrix} = -\sin(x_2).\end{aligned}$$

Combining both, we get the expected answer:

$$\mathbf{J}_f = [\mathbf{J}_f^{(1)} \ \mathbf{J}_f^{(2)}] = [2 \cos(x_1) \ -\sin(x_2)].$$

2.3.3 Reverse-mode AD

When n is large but m is small, it is more efficient to compute the Jacobian matrix row-by-row. For this, we multiply $\mathbf{J}_f$ by $\mathbf{e}_i$ on the left:

$$\mathbf{J}_f^{(i)} = \mathbf{e}_i^\top \mathbf{J}_f = \underline{\underline{\mathbf{e}_i^\top \mathbf{J}_{f_c} \mathbf{J}_{f_b} \mathbf{J}_{f_a}}},$$

where the notation $\mathbf{J}_f^{(i)}$ denotes the i^{th} row of $\mathbf{J}_f$, $i \in \{1..m\}$.

To compute this, we start from left-to-right, first computing the vector-Jacobian product $\mathbf{e}_i^\top \mathbf{J}_{f_c}$. This yields a vector which we shall call $\mathbf{v}_c^{(i)\top}$. We next compute the vector-Jacobian product $\mathbf{v}_c^{(i)\top} \mathbf{J}_{f_b}$ to yield vector $\mathbf{v}_b^{(i)\top}$. We continue computing vector-Jacobian products moving left-to-right in the expression, until all Jacobians are consumed. The resulting vector is $\mathbf{J}_f^{(i)}$ – the i^{th} row of $\mathbf{J}_f$:

$$\begin{aligned}\mathbf{v}_c^{(i)\top} &= \mathbf{e}_i^\top \mathbf{J}_{f_c} \\ \mathbf{v}_b^{(i)\top} &= \mathbf{v}_c^{(i)\top} \mathbf{J}_{f_b} \\ \mathbf{J}_f^{(i)} &= \mathbf{v}_b^{(i)\top} \mathbf{J}_{f_a}\end{aligned}$$

One pass through the expression yields one row of the Jacobian. For a function $\mathbb{R}^n \rightarrow \mathbb{R}^m$, we need m passes to compute the full Jacobian $\mathbf{J}_f$.

i Why is it called reverse-mode AD?

The left-to-right evaluation of the vector-Jacobian products is considered *reverse-mode* because the evaluation happens in the reverse order as the function value evaluation. In our example above, to compute the composite function $\mathbf{f}$, we first apply $\mathbf{f}_a$ to $\mathbf{x}$, then apply $\mathbf{f}_b$ to the output, and finally apply $\mathbf{f}_c$. To compute the Jacobian, we follow the

reverse order: we compute the vector-Jacobian product $\mathbf{e}_i^\top \mathbf{J}_{\mathbf{f}_c}$ first, then multiply it by $\mathbf{J}_{\mathbf{f}_b}$ on the right, and finally by $\mathbf{J}_{\mathbf{f}_a}$.

This method only computes one vector-Jacobian product (often called *vjp*) at a time, thus limiting computational complexity. The output of each computation step is always a row vector (referred to as a *sensitivity* vector). The Jacobian at each step is often very sparse, and the sparsity structure can be exploited to compute the vector-Jacobian product efficiently (and often without the need to explicitly create the Jacobian matrix in memory). The Jacobians of the primitive functions are utilized in the opposite order as the primitive functions are evaluated in computing the value of the function. Hence this method is known as *reverse-mode AD*.

At each step of the computation, we multiply the sensitivity of the output of a primal function with the Jacobian of that function to yield the sensitivity at the input of the primal function. The function that does this vector-Jacobian product is called the *pullback* function.

In forward-mode AD, the Jacobian-vector products can be evaluated alongside the evaluation of the function, since both computations happen in the same direction. In reverse-mode AD, a computation graph (often called a *tape*) is built in memory as the function is evaluated. The tape stores the pullbacks as they are generated in the forward pass. The AD system then walks the graph in the reverse direction evaluating the pullbacks to compute the vector-Jacobian products yielding each row of the Jacobian. The requirement to store the graph in memory leads to larger memory requirements for reverse-mode AD as compared to forward-mode AD. However, when $n \gg m$ (as is the case in many machine learning applications where the gradient of the loss function $\mathbb{R}^n \rightarrow \mathbb{R}$ with n model parameters is needed), the computational advantage of reverse-mode outweighs the memory disadvantage.

Example

Again, let's work through our previous example of $f(\mathbf{x}) = 2 \sin(x_1) + \cos(x_2)$. We start with a seed $\mathbf{e}_1^\top = [1]$:

$$\begin{aligned} \mathbf{v}_c^{(1)\top} &= [1] \begin{bmatrix} 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 1 \end{bmatrix}, \\ \mathbf{v}_b^{(1)\top} &= \begin{bmatrix} 1 & 1 \end{bmatrix} \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 2 & 1 \end{bmatrix}, \\ \mathbf{J}_f^{(1\cdot)} &= \begin{bmatrix} 2 & 1 \end{bmatrix} \begin{bmatrix} \cos(x_1) & 0 \\ 0 & -\sin(x_2) \end{bmatrix} = \begin{bmatrix} 2 \cos(x_1) & -\sin(x_2) \end{bmatrix}. \end{aligned}$$

We have the Jacobian computed in one pass, since the output is scalar and hence the Jacobian has a single row:

$$\mathbf{J}_f = [\mathbf{J}_f^{(1\cdot)}] = \begin{bmatrix} 2 \cos(x_1) & -\sin(x_2) \end{bmatrix}.$$

2.3.4 Implementation

There are many ways to implement AD in practice. The Python ecosystem has `autograd`, `tensorflow`, `torch.autograd` (part of PyTorch) and `jax`, to name a few. The Julia ecosystem has `ForwardDiff.jl`, `ReverseDiff.jl`, `Tracker.jl`, `Zygote.jl`, `Yota.jl`, `Nabla.jl`, `ReversePropagation.jl`, `Difffractor.jl` and `Enzyme.jl`. Some implementations are fundamentally distinct, while others are similar with only small differences.

In this section, we explore a few common AD packages with distinct implementation styles to get an understanding of how each works and what it has to offer:

2.3.4.1 ForwardDiff.jl

`ForwardDiff.jl` is a forward-mode AD package that is implemented using dual numbers (Section 2.2). It works via operator overloading. Primitive operations are defined for the dual number data type. A function to be differentiated must accept generic inputs, and compute without making strong assumptions about the number types. This allows computation with dual numbers to propagate sensitivities through the function, yielding the function value and derivatives at the output of the function.

Consider a function:

```
function f1(x::Array{Float64})
    s = 0.0
    for (a, b) ∈ enumerate(x)
        s += a * b
    end
    return s
end
```

Differentiating this with `ForwardDiff.jl` would fail because this implementation only takes in arrays of `Float64` for `x` and `y`, and passing in arrays of dual numbers will fail:

```
try
    ForwardDiff.gradient(f1, [1.0, 1.1])
catch
    @error "AD failed"
end
```

```
[ Error: AD failed
@ Main In[24]:4
```

On the other hand, generically written versions of the same function can be easily differentiated:

```
function f2(x)
    s = zero(eltype(x))
    for (a, b) ∈ enumerate(x)
        s += a * b
    end
    return s
end

ForwardDiff.gradient(f2, [1.0, 1.1])
```

```
2-element Vector{Float64}:
 1.0
 2.0
```

or:

```
f3(x) = sum(x .* eachindex(x))

ForwardDiff.gradient(f3, [1.0, 1.1])
```

```
2-element Vector{Float64}:
 1.0
 2.0
```

💡 ForwardDiff.jl – Summary

Implementation

- Forward-mode AD using dual numbers.

Pros

- Efficient when the number of inputs (number of arguments or input vector sizes) is small (typically <100).

Cons

- Requires function to accept and compute with generic number types.

2.3.4.2 ReverseDiff.jl

`ReverseDiff.jl` is an implementation of reverse-mode AD that records pullbacks on a tape, and then evaluates them on the reverse pass. The tape is implemented as a data type with operator overloading for primitive operations. So the function to be differentiated must accept and be able to compute with generic input types, just like in the case of `ForwardDiff.jl`. However, `ReverseDiff.jl` is more efficient than `ForwardDiff.jl` when the number of inputs is large and the number of outputs is small.

The specialized function `f1` fails (as is the case with `ForwardDiff.jl`):

```
import ReverseDiff

try
  ReverseDiff.gradient(f1, [1.0, 1.1])
catch ex
  @error "AD failed"
end
```

```
[ Error: AD failed
@ Main In[27]:6
```

But the generic functions `f2` and `f3` can be differentiated:

```
ReverseDiff.gradient(f2, [1.0, 1.1])
```

```
2-element Vector{Float64}:
 1.0
 2.0
```

```
ReverseDiff.gradient(f3, [1.0, 1.1])
```

```
2-element Vector{Float64}:
 1.0
 2.0
```

ReverseDiff.jl

Implementation

- Reverse-mode AD using a tape.

Pros

- Efficient when the number of inputs (number of arguments or input vector sizes) is large (typically >100) and the number of outputs is small.

Cons

- Requires function to accept and compute with generic number types.

2.3.4.3 Zygote.jl

Zygote.jl is a source-to-source AD package that is capable of both forward-mode and reverse-mode AD. By default, it uses reverse-mode AD. It operates on the *intermediate representation* (IR) – a partially compiled form of the source function, to generate a function that computes the gradient. Since Zygote.jl operates on IR, it can operate on a function that is specialized. However, it has other limitations (e.g., it does not support array mutation and exception handling).

Zygote.jl can differentiate the specialized f1 as well as the generic f2 and f3 functions:

```
import Zygote

Zygote.gradient(f1, [1.0, 1.1])[1]
```

2-element Vector{Float64}:

```
1.0
2.0
```

```
Zygote.gradient(f2, [1.0, 1.1])[1]
```

2-element Vector{Float64}:

```
1.0
2.0
```

```
Zygote.gradient(f3, [1.0, 1.1])[1]
```

2-element Vector{Float64}:

```
1.0
2.0
```

 Tip

Zygote.jl supports gradients of functions with many arguments, and hence needs the `[1]` at the end to tell it that we want the gradient with respect to the first argument of the function. We see how this works:

```
g(x, y) = 2x + y

Zygote.gradient(g, 1.0, 1.1)
```

```
(2.0, 1.0)
```

The first element in the returned tuple is gradient with respect to `x` and the second element is the gradient with respect to `y`.

ForwardDiff.jl and ReverseDiff.jl only support differentiating functions with single arguments. This is not a major limitation as one can always define an anonymous function of one variable while taking the derivative with respect to one argument at a time:

```
ForwardDiff.derivative(x → g(x, 1.1), 1.0)
```

```
2.0
```

and

```
ForwardDiff.derivative(y → g(1.0, y), 1.1)
```

```
1.0
```

Alternatively, one could combine all the arguments into a single vector:

```
ReverseDiff.gradient(xy → g(xy[1], xy[2]), [1.0, 1.1])
```

```
2-element Vector{Float64}:
```

```
2.0
```

```
1.0
```

However, it cannot differentiate a function that mutates an array:

```
function f4(x)
    T = eltype(x)
    ab = T[]
    for (a, b) ∈ enumerate(x)
        push!(ab, a * b)
    end
    sum(ab)
end

try
    Zygote.gradient(f4, [1.0, 1.1])[1]
catch
    @error "AD failed"
end
```

```
[ Error: AD failed
  @ Main In[37]:13
```

Whereas, ForwardDiff.jl or ReverseDiff.jl would have no problem differentiating f4:

```
ForwardDiff.gradient(f4, [1.0, 1.1])
```

```
2-element Vector{Float64}:
 1.0
 2.0
```

```
ReverseDiff.gradient(f4, [1.0, 1.1])
```

```
2-element Vector{Float64}:
 1.0
 2.0
```

Zygote.jl – Summary

Implementation

- AD using source-to-source conversion.

Pros

- Able to differentiate functions specialized for specific data types.

- Defaults to reverse-mode AD, and is therefore efficient when the number of inputs (number of arguments or input vector sizes) is large (typically >100) and the number of outputs is small.

Cons

- Does not support mutation of arrays.
- Does not support exception handling (try-catch blocks).

2.4 Custom AD rules

AD enables us to differentiate complex computer programs automatically through repeated application of the chain rule and known derivatives of primitive functions (such as addition, multiplication, trigonometric functions, etc). While this works well for many functions, it occasionally fails when it encounters a function that AD does not know how to differentiate or decompose further. This may happen in several circumstances. For example:

- The derivatives of thousands of primitive functions are defined in packages such as `ChainRules.jl`. However, if an uncommon function is available as a CPU-supported hardware instruction or as a compiled library (potentially written in another language), but a rule for its derivative is not defined, AD may fail.
- AD may fail because of a limitation of the AD library being used (e.g., mutation for `Zygote.jl`, specialized function for `ForwardDiff.jl` or `ReverseDiff.jl`).
- Some AD libraries have support for higher-order AD, while others do not. If the function being differentiated internally uses AD, trying to differentiate it using AD again requires higher-order AD. This could fail.

In such cases, we need to aid the AD system by manually defining an AD rule that helps it differentiate the offending function. Even if AD works on some functions, sometimes we may know of a computationally more efficient way of computing the derivative. In such cases, defining an AD rule can help speed up computation.

In this section, we learn how to write AD rules through a series of illustrative examples.

Tip

Reverse-mode AD tools such as `Zygote.jl` and `ReverseDiff.jl` are able to understand reverse-mode differentiation rules (`rrule`) written using `ChainRulesCore.jl`. While `ChainRulesCore.jl` also supports writing of forward-mode differentiation rules (`frule`), these rules are not directly supported by forward-mode AD tools such as `ForwardDiff.jl` yet. One can either manually define dual number variants of the primitive function to the

differentiated or use `ForwardDiffChainRules.jl` to automatically generate such variants from the corresponding frule.

Example 1: Mutation

Consider the following function that internally mutates an array:

```
function f5(x::AbstractArray)
    y = similar(x)
    for i ∈ eachindex(x)
        y[i] = x[i] > 0 ? x[i]^2 : zero(eltype(x))
    end
    sum(y)
end
```

If we try to differentiate it using `Zygote.jl`, we get an exception:

```
try
    Zygote.gradient(f5, [1.7, 2.1, -1.5, -0.1, 3.1])[1]
catch
    @error "AD failed"
end
```

```
[ Error: AD failed
@ Main In[41]:4
```

In this simple example, one could rewrite the function to avoid the mutation and then differentiate it:

```
function f5a(x::AbstractArray)
    s = zero(eltype(x))
    for i ∈ eachindex(x)
        if x[i] > 0
            s += x[i]^2
        end
    end
    s
end
```

```
Zygote.gradient(f5a, [1.7, 2.1, -1.5, -0.1, 3.1])[1]
```

```
5-element Vector{Float64}:
```

```
3.4
4.2
0.0
0.0
6.2
```

However, this is not always possible. So let's learn to write a `rrule` to differentiate the original function `f5`:

💡 Multiple dispatch

One of the key innovations in Julia is called *multiple dispatch*. A function in Julia can have many methods, each with different argument types (and perhaps count). Julia automatically selects the correct method to invoke based on the data types that the function is being called with. Multiple dispatch is used heavily in the Julia ecosystem to enable different packages to interact and extend each other, without having to modify the source code of the peer package. In this section, we use multiple dispatch to extend the `ChainRulesCore.jl` package by adding methods to the `rrule` and `frule` functions, without having to modify the source code of that package.

An `rrule` is dispatched on the function type. It takes in the arguments of the function and returns a 2-tuple. The first entry in the tuple is the value of the function evaluated for the arguments. The second entry in the tuple is a closure⁵ called the *pullback*. The `rrule` is called on the forward pass through and the pullback is stored. This pullback is called during the reverse pass to compute the vector-Jacobian product (vjp). The return value of the pullback is tuple of vjps – one for each input of the primal function (including the primal function itself).

Let's write the rule for `f5`:

```
using ChainRulesCore

function ChainRulesCore.rrule(::typeof(f5), x)
    J = [x_i > 0 ? 2x_i : zero(x_i) for x_i ∈ x]
```

⁵A *closure* is a function (computer program) that is defined and assigned to a variable or returned. It carries the context in which it was defined with it: for example, if a locally scoped variable is accessed in the closure, the value of that variable is bundled with the closure so that it is available even after the local variable goes out of scope. The closure can be called later, at which point, the function executes in the context that was bundled with it. Closures are also called *anonymous functions* or *lambdas*.

```

    f5_pullback(v) = NoTangent(), v * J
    f5(x), f5_pullback
end

# reload Zygote rules
# (needed only because function f5 was used with Zygote before)
Zygote.refresh()

```

The `rrule` is dispatched on `typeof(f5)` and takes in argument `x` during the forward pass. The first line of the `rrule` computes the Jacobian `J` of `f5` at `x`. For each entry in `x` that is non-positive, there is a corresponding zero entry in `J` since the non-positive entries in `x` do not contribute to the output of `f5`. Positive entries in `x` are squared and added to the sum, and therefore each positive entry `xi` yields a `J` entry `2xi`. The second line defines the pullback that returns a 2-tuple. The first entry `NoTangent()` says that the derivative with respect to the function `f5` itself does not exist, since `f5` does not contain any state (it is not a *callable structure*⁶). The second entry defines the vjp `v * J`. The last line in the `rrule` returns the output of the primal function `f5(x)` and the pullback `f5_pullback`.

i Derivative with respect to the primal function

Since Julia allows callable structures that may contain attributes, the vjp with respect to the attributes may also be needed. Thus the first entry in the returned tuple corresponds to the primal function. If the function being called is not a callable structure, one must return a `NoTangent()` value to indicate that no derivative is necessary for the primal function. We do not use callable structures in this course, and so always return a `NoTangent()` as the first entry.

With the `rrule` defined, AD using Zygote will treat `f5` as a primitive function and not need to decompose it. Thus the AD should work:

```
Zygote.gradient(f5, [1.7, 2.1, -1.5, -0.1, 3.1])[1]
```

5-element Vector{Float64}:

```

3.4
4.2
0.0
0.0
6.2

```

⁶In Julia, structures work like functions. You can add methods to the structures, and then call those methods, just like you would call any function's methods. We do not use callable structures in this course.

Writing such rules can be error-prone. How do we check if we wrote the rule correctly? We could test it against the result from numerical differentiation using `FiniteDifferences.jl`:

```
FiniteDifferences.grad(derivative_fd, f5, [1.7, 2.1, -1.5, -0.1, 3.1])[1]
```

```
5-element Vector{Float64}:
 3.4000000000000059
 4.2000000000000125
 1.2860727053608563e-13
 1.2860727053608563e-13
 6.1999999999999265
```

It is a common practice to write unit tests for AD rules by testing against numerical differentiation. The Julia ecosystem therefore has a package which automates this:

```
using ChainRulesTestUtils
test_rrule(f5, [1.7, 2.1, -1.5, -0.1, 3.1]);
```

```
Test Summary: | Pass Total Time
test_rrule: f5 on Vector{Float64} | 7 7 0.7s
```

Consider a mutating function with two inputs:

```
function f6(x, y)
    z = similar(x)
    for i ∈ eachindex(x)
        z[i] = 2x[i] + y[i]
    end
    sum(z)
end
```

This would fail AD using `Zygote.jl` unless we define a `rrule` for it:

```
function ChainRulesCore.rrule(::typeof(f6), x, y)
    f6_pullback(v) = NoTangent(), fill(2v, length(x)), fill(v, length(y))
    f6(x, y), f6_pullback
end
```

Note that the return value of the pullback now is a 3-tuple with `NoTangent()` for the function `f6`, a vector of same length as `x` of `2v` (as the Jacobian with respect to `x` is 2), and a vector of same length as `y` of `v` (as the Jacobian with respect to `y` is 1).

We test this rule:

```
test_rrule(f6, [1.7, 2.1, -1.5], [5.1, -1.2, 0.0]);
```

Test Summary:		Pass	Total	Time
test_rrule: f6 on Vector{Float64}, Vector{Float64}		9	9	0.3s

Note

The [ChainRules.jl documentation](#) is well worth reading to understand how to write reverse and forward mode AD rules (rrule and frule) in greater detail.

Example 2: External library calls

Bessel functions $J_\alpha(\cdot)$ are generalizations of the sine function that have wide applications in physics and mathematics. While the C library `libm` provides implementations of some Bessel functions (`j0` and `j1`), the Julia base library does not provide them⁷. We can easily define Julia functions that call Bessel functions from `libm`:

```
j0(x::Float64) = @ccall j0(x::Float64)::Float64
j1(x::Float64) = @ccall j1(x::Float64)::Float64

j0(10.0), j1(10.0)
```

While we can call these functions successfully, we cannot use AD to differentiate them:

```
try
    Zygote.gradient(j0, 10.0)[1]
catch
    @error "AD failed"
end
```

```
[ Error: AD failed
@ Main In[51]:4
```

The derivative of $J_0(\cdot)$ is analytically known: $J_0(x) = -J_1(x)$. So we can define an rrule:

⁷The Julia library `SpecialFunctions.jl` provides Bessel functions, but we will not use it here since we want to illustrate how to implement a rrule for an external library call (`libm`).

```
function ChainRulesCore.rrule(::typeof(j0), x)
    J = -j1(x)
    j0_pullback(v) = NoTangent(), v * J
    j0(x), j0_pullback
end

Zygote.refresh()
```

and test it:

```
test_rrule(j0, 10.0);
```

```
Test Summary:          | Pass  Total  Time
test_rrule: j0 on Float64 |    7      7  0.2s
```

💡 Scalar AD rules

AD rules for scalar functions (scalar input and scalar output) are commonly needed. A convenience macro `@scalar_rule` is available to generate the appropriate `rrule` and `frule` automatically:

```
@scalar_rule j0(x) -j1(x)
```

Since `ForwardDiff.jl` does not support `ChainRules.jl` yet, forward mode AD using `ForwardDiff.jl` will still fail:

```
try
    ForwardDiff.derivative(j0, 10.0)
catch
    @error "AD failed"
end
```

```
[ Error: AD failed
  @ Main In[55]:4
```

This can be fixed by defining a `frule` and then using `ForwardDiffChainRules.jl` to convert it to a dual number specialization of the `j0` function. Alternatively, we can directly define the dual number version of `j0` ourselves:

```
function j0(x::ForwardDiff.Dual)
    v = ForwardDiff.partials(x) # extract vector v
    T = ForwardDiff.tagtype(x)  # extract dual type identifier
    x = ForwardDiff.value(x)    # extract real value x
    ForwardDiff.Dual{T}(j0(x), -j1(x) * v)
end

ForwardDiff.derivative(j0, 10.0)
```

```
-0.043472746168861445
```

Recall from Section 2.2 that a function evaluated using a dual number yields the Jacobian-vector product (jvp) in the infinitesimal part of the dual number. We define function `j0` specialized for dual numbers, and extract the vector `v` from the dual number in the first line. We then extract value `x` and compose a return dual number with value `j0(x)` and infinitesimal part containing the jvp `-j1(x) * v`. The tag `T` is propagated from the input dual type to the output dual type, as required by `ForwardDiff.jl`.

Example 3: Speed

Sometimes it is useful to define a `rrule` for a function that can already be differentiated with AD for performance reasons.

Consider the error function:

$$\operatorname{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-u^2} du.$$

This function has plenty of applications in statistics, detection theory and communication theory. The integral does not have a closed form value but can be easily numerically computed:

```
using QuadGK

erf(x) = quadgk(u → exp(-u^2), 0, x)[1]
erf(1.0)
```

```
0.746824132812427
```

We can differentiate it using `Zygote`, but it is slow as the derivative computation involves computing the integral numerically:


```
using BenchmarkTools
```

```
@btime Zygote.gradient(erf, 1.0)[1]
```

```
221.583 μs (1656 allocations: 59.22 KiB)
```

```
0.3678794411714424
```

Since the $\text{erf}(\cdot)$ is defined as an integral, differentiating it analytically is easy:

$$\frac{d}{dx}\text{erf}(x) = e^{-x^2}.$$

We can define the derivative `rrule` using the `@scalar_rule` macro and benchmark it:

```
@scalar_rule erf(x) exp(-x^2)
```

```
Zygote.refresh()
```

```
@btime Zygote.gradient(erf, 1.0)[1]
```

```
89.552 ns (0 allocations: 0 bytes)
```

```
0.36787944117144233
```

We see that the AD with the `rrule` added is about $2000\times$ faster the original version and requires no memory allocation.

Example 4: Implicit Functions

Consider a function $f_7(x)$ which is defined as the solution for y to an equation:

$$x^2 + xy + y^3 = 0. \tag{2.4}$$

Such a function is called an *implicit function* since we do not have a value for y in closed form. The solution may be iteratively estimated:

```
using Roots
```

```
f7(x) = find_zero(y → x^2 + x*y + y^3, 0.0)
```

```
f7(1.0)
```

```
-0.6823278038280193
```

Function `f7` cannot be differentiated using `Zygote.jl` as the `find_zero` function mutates an array internally. It cannot be differentiated using `ForwardDiff.jl` or `ReverseDiff.jl` as the `find_zero` function only supports floating point numbers.

We can, however, find an analytical derivative dy/dx by differentiating Equation 2.4:

$$\begin{aligned}\frac{d}{dx}(x^2 + xy + y^3) &= \frac{d}{dx}(0) \\ 2x + x\frac{dy}{dx} + y + 3y^2\frac{dy}{dx} &= 0 \\ 2x + y + (x + 3y^2)\frac{dy}{dx} &= 0 \\ \frac{dy}{dx} &= -\frac{2x + y}{x + 3y^2}\end{aligned}$$

We can define the `rrule` accordingly, and check that it works correctly:

```
function ChainRulesCore.rrule(::typeof(f7), x)
    y = f7(x)
    J = -(2x + y) / (x + 3y^2)
    f7_pullback(v) = NoTangent(), v * J
    y, f7_pullback
end

test_rrule(f7, 1.0);
```

Test Summary:		Pass	Total	Time
test_rrule: f7 on Float64		7	7	0.1s

and

```
Zygote.gradient(f7, 1.0)[1]
```

-0.5497828958771193

💡 Custom AD rules – Summary

While AD is able to automatically differentiate many functions, sometimes it is necessary to specify the derivative of a function explicitly. This may be to get around the limitations of an AD system or for performance reasons. Custom AD rules enable us to define chain rules for forward-mode or reverse-mode AD in a package agnostic way.

2.5 Summary

Gradients are extremely important in machine learning. While it is possible to estimate numerical gradients through finite differencing methods, they tend to have poor accuracy. As the number of dimensions increase, numerical gradients becomes computationally demanding. We studied AD methods that can compute exact gradients (to machine precision) efficiently. When the number of inputs is small, forward-mode AD methods are computationally efficient. When the number of outputs is small, reverse-mode AD methods are preferred. Sometimes AD fails or is too slow – we learned how to manually write AD rules for such times.

In Chapter 3, we extensively use gradients in applications that combine domain knowledge with data-driven methods. In Chapter 4, we learn how to work with uncertainty and discover that gradients are needed there too. AD is thus a foundation on which we build the rest of this course.

3 Physics-guided Machine Learning

Now that we understand how AD can be used to compute gradients (and potentially higher order derivatives), we move our focus to understanding how AD can be used to solve real-world problems. Neural networks, for example, rely on gradients for training of parameters (weights and biases). Since neural networks typically have a large number of parameters, they tend to be data-hungry. In this chapter, we explore machine learning techniques that incorporate domain knowledge (we generically call it *physics*) to enable learning from a small amount of training data. These techniques also require gradients during training, and therefore critically depend on AD.

Roadmap

- We begin this chapter with a peek into the Julia ecosystem for solving **differential equations** and training **neural networks**. These tools are foundational to the techniques we study in the rest of this chapter.
- In the rest of the chapter, we study two approaches to combining observed data with domain knowledge in the form of system dynamics described by differential equations:
 - If the differential equations governing the dynamics are not fully known, we model the derivative of the state of the system by a neural network. The resulting **neural differential equation** can be trained with available data.
 - If the dynamics of the system are fully understood (except for some parameters that may be estimated from data), we use **physics-informed neural networks** to directly model the solution of the differential equation.

3.1 Differential Equations

In Section 1.3, we considered a simple problem of developing a Netball-playing robot. The physics of throwing the ball was described in terms of differential equations. The dynamics of many real-world problems are described in terms of differential equations – not just in physics, but also in chemistry, biology, pharmacology, sociology, finance, etc. Differential equations therefore play an important role in modeling real-world phenomena of interest to us.

We solved the differential Equation 1.1 analytically to yield Equation 1.2. For this simple example, it was possible to find a closed form solution via integration. However, often this isn't the case and we have to resort to numerical methods to solve differential equations. For example, if we consider air resistance (drag), the differential equations describing the motion of the ball change to:

$$\begin{aligned}\ddot{x} &= -\frac{D\dot{x}}{\nu}, \\ \ddot{y} &= -\frac{D\dot{y}}{\nu} + g,\end{aligned}\tag{3.1}$$

with:

$$\begin{aligned}\nu &= \sqrt{\dot{x}^2 + \dot{y}^2}, \\ D &= \frac{1}{2}C_d\rho A\nu^2,\end{aligned}$$

where ν denotes the speed of the ball and D denotes the magnitude of the drag force. The constants C_d , ρ and A are the drag coefficient, density of air and the cross-sectional area of the ball respectively. This system of differential equations is not easily solved analytically.

Ordinary differential equation solver API

The `DifferentialEquations.jl` package provides a comprehensive suite of tools to work with differential equations. To solve ordinary differential equations (ODE), we write a Julia function that computes the derivative of the dependent variable `u` with respect to the independent variable `t` (typically time):

```
dudt(u, p, t) = ...
```

The input `p` contains additional user-defined parameters, if required. We then define an initial-value ODE problem:

```
using DifferentialEquations

prob = ODEProblem(dudt, u0, tspan)
```

where `u0` is the initial value of `u`, and `tspan` is a 2-tuple defining the span of `t` over which the solution is desired. If additional user-defined parameters are required, they are passed in as a fourth argument to `ODEProblem()`. Finally we solve the problem using a given solver (e.g. `Tsit5()`):

```
sol = solve(prob, Tsit5())
```

A list of available solvers (along with an explanation of their pros and cons) is available

[here](#).

Once a solution `sol` is obtained, it can be queried for the value of u at t as `sol(t)`. It can also be visualized using `plot(sol)`.

For more details, see [package documentation](#).

Equation 3.1 is a system of second-order differential equations. To solve it numerically, we define vector $\mathbf{u} = [x, y, \dot{x}, \dot{y}]^\top$, and rewrite Equation 3.1 as a first order vector differential equation:

$$\frac{d\mathbf{u}}{dt} = \begin{pmatrix} \dot{x} \\ \dot{y} \\ -D\dot{x}/\nu \\ -D\dot{y}/\nu + g \end{pmatrix}. \quad (3.2)$$

This can be readily solved numerically in Julia:

```
using DifferentialEquations

# define the differential equation (parameters p and time t not used)
function dudt(u, p, t; g=-9.81, Cd=0.47, ρ=1.225, A=π*0.11^2)
    x, y, ẋ, ẏ = u
    v = sqrt(ẋ^2 + ẏ^2)
    D = 0.5 * Cd * ρ * A * v^2
    [ẋ, ẏ, -D * ẋ/v, -D * ẏ/v + g]
end

# solve the differential equation
u0 = [-4.0, 1.0, 4.1, 6.9]          # initial state (x, y, ẋ, ẏ)
tspan = (0.0, 0.95)               # time span to solve for
prob = ODEProblem(dudt, u0, tspan)
sol = solve(prob, Tsit5())
```

The solution of the differential equation gives us the trajectory of the ball taking into consideration the air resistance. We plot this solution and compare it against the drag-free analytical solution in Equation 1.2 in Figure 3.1.

```
using Plots

# numerical solution with drag
plot(sol; idxs=(1,2), xlims=(-5,1), ylims=(0,4), label="with drag", xlabel="x",
      ↪ ylabel="y")
```

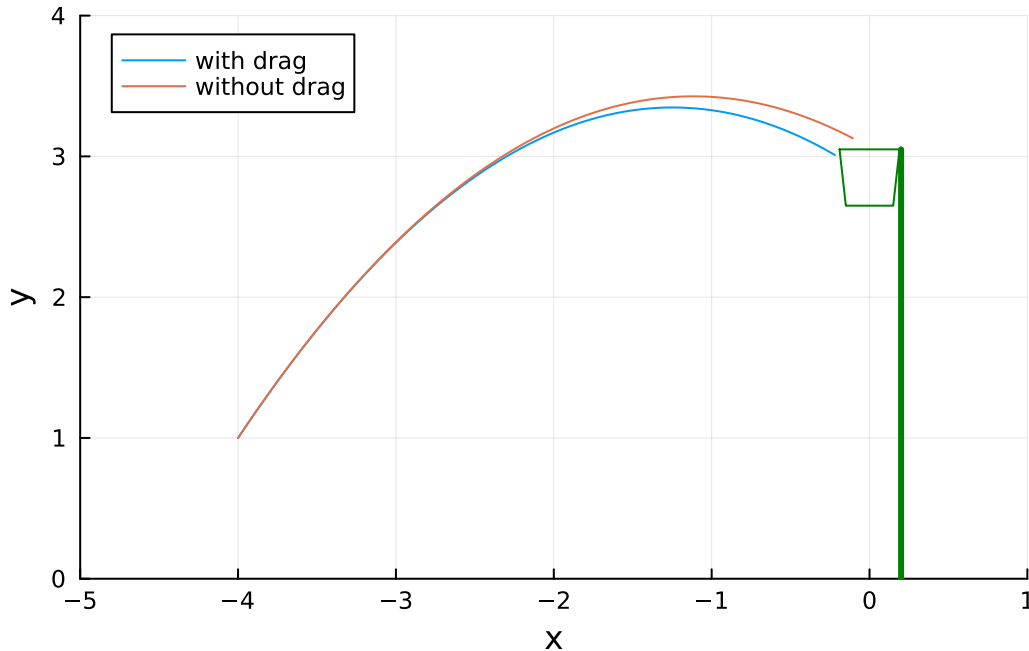


Figure 3.1: Ball trajectory with and without drag.

```
# analytical solution without drag
x0, y0, vx, vy = u0
v = sqrt(vx^2 + vy^2)
θ = atan(vy, vx)
t = range(tspan...; length=100)
plot!(x0 .+ v*t*cos(θ), y0 .+ v*t*sin(θ) .- 9.81/2 * t.^2; label="without drag")

# draw basket
plot!([(0.2,3.05), (0.2,0.0)]; color=:green, linewidth=3, label=nothing)
plot!([(-0.19,3.05), (0.19,3.05), (0.15,2.65), (-0.15,2.65), (-0.19,3.05)];
color=:green, label=nothing)
```

We observe that the drag causes the ball to fall short of the basket for the optimal solution found in Section 1.3, since we ignored drag when deriving that solution. In the rest of this chapter, we explore various ways to teach our robot to throw the ball into the basket without knowing the details of the drag forces acting on the ball explicitly.

3.2 Netball-playing Robot Revisited

The Netball-playing robot problem defined in Section 1.3 obeyed *ideal* physics laws. The ball trajectory was determined completely by Newton's laws of motion with no air resistance, no slope, no wind, and no other disturbances. Moreover, we assumed that the parameters of the problem (such as g , d , y_0 , etc) were perfectly known. In practice, these assumptions are not met, and so the robot designed with such assumptions is unlikely to win Netball games. Every Netball game that the robot plays may use a different ball with slightly different mass and drag parameters. The Netball court may have a gentle slope, and so the direction of gravity may not be exactly along the negative y -axis. Depending on the location on Earth, the value of g may differ slightly. Weather may affect air resistance and wind. Therefore it is important for a good Netball-playing robot to learn how to throw the ball by watching what the ball does during the game.

A common solution to this type of problem is to use reinforcement learning or a neural network to learn the inverse function $d \rightarrow (v, \theta)$ through trial-and-error. This is similar to the approach in Figure 1.4, except that the physics is unknown and therefore we do not know how to map $(v, \theta) \rightarrow y(T)$ in simulation. Instead, we could throw the ball with velocity (v, θ) in the real world and observe the result $y(T)$. The problem with this approach is that we would need a large number of throws to train the neural network.

Would it be possible to learn to throw accurately with just one or two trial throws? It certainly would, if we could train a *surrogate* forward model that would allow us to map $(v, \theta) \rightarrow y(T)$ in simulation. How can we train such a model with just one or two trial throws? Let's find out...

When the robot throws a ball, we assume it is able to measure the trajectory (with some noise) of the ball for the first second or so. This may be accomplished through a calibrated monocular camera to measure distance of ball by the apparent change in size of the ball in the camera image, or via a stereo camera setup, or even using a laser rangefinder to track the ball. We wish to use the measured trajectory to update our model so that the robot is able to improve its throw in the next attempt.

Here is some data collected (noisy measurements) during one throw of the ball by the robot:

i	t	v	θ	x	y
1	0.0	8.94	63.4	0.0	0.0
2	0.1	8.94	63.4	0.404731	0.740347
3	0.2	8.94	63.4	0.800349	1.38249
4	0.3	8.94	63.4	1.17928	1.93416
5	0.4	8.94	63.4	1.56923	2.37687
6	0.5	8.94	63.4	1.9616	2.70341
7	0.6	8.94	63.4	2.34156	2.94354

8	0.7	8.94	63.4	2.72002	3.06893
9	0.8	8.94	63.4	3.09716	3.10138
10	0.9	8.94	63.4	3.47015	3.09218
11	1.0	8.94	63.4	3.86619	2.91732

Here v and θ are the initial speed and angle for the throw respectively. i is the measurement index, t the measurement time, and (x, y) the measured position of the ball with respect to the shooting apparatus.

Our task is to use this data to learn a model that will allow us to make accurate throws. A forward model takes in (x_0, y_0, v, θ, t) as inputs and yield (x, y) as outputs. Given a forward model such as in Equation 1.2, we saw how to train a neural network to solve the inverse problem of computing optimal (v, θ) in Section 1.3.4. However, for the problem in Section 3.2, we do not have a forward model available.

3.3 Neural Networks

Since we have some measured data, perhaps we can train a neural network to approximate the forward model?

i Training a neural network in Julia

Two of the most popular libraries for neural networks in Julia are `Flux.jl` and `Lux.jl`. While `Flux.jl` has been around for longer and is more mature, `Lux.jl` is better suited to physics-based machine learning and so we use it throughout this course. To define a neural network, we create it with the appropriate layers:

```
using Lux
import Random

model = Chain(
    Dense{3 ⇒ 8, tanh}, # input layer with 3 neurons
    Dense{8 ⇒ 8, tanh}, # hidden layer with 8 neurons
    Dense{8 ⇒ 8, tanh}, # hidden layer with 8 neurons
    Dense{8 ⇒ 2}        # output layer with 2 neurons
)

ps, st = Lux.setup(Random.default_rng(), model)
```

On initialization, `model` contains the architecture of the neural network, `ps` contains the parameters (weights and biases), and `st` contains the state. Neural networks with dense

layers do not have any internal state, but layers such as recurrent layers, dropout layers, etc, may have internal state that is stored in `st`. The call to `Lux.setup()` specifies a random number generator to be used for initializing the weights of the network. We may choose to provide a seeded random number generator for repeatable results.

To make predictions, we can define a `predict(x)` function that takes in a input vector `x` and produces an output vector:

```
predict(x, ps) = Lux.apply(model, x, ps, st)[1]
```

If the input is provided as a matrix, each column of the matrix is treated as an input vector, and the output vectors are concatenated column-wise. `Lux.apply` returns the output of the chain as well as the updated state. Since our example neural network does not have any state, we use `[1]` to extract only the output of the network and ignore the state.

To train the network, we define a loss function $\mathcal{L}$ of parameters `ps`. We define an optimizer to use for training. For each training batch, we use our favorite AD tool to take a gradient of $\mathcal{L}$ with respect to `ps` and use it to update the parameters `ps`. Since the number of parameters is typically large, we use a reverse-mode AD tool such as `Zygote`. The resulting training loop typically looks like:

```
import Optimisers
import Zygote

# sum square error loss function
 $\mathcal{L}(\text{ps}, x, y) = \text{sum}(\text{abs2}, \text{predict}(x, \text{ps}) .- y)$ 

# pick an optimizer
opt = Optimisers.setup(Optimisers.Adam(0.001f0), ps)

# loop over epochs and batches of training data of the form [(x, y), ...]
for epoch ∈ 1:max_epochs
    for (x, y) ∈ batched_training_data
        gs = Zygote.gradient( $\mathcal{L}$ , ps, x, y)[1]      # compute gradient wrt ps
        opt, ps = Optimisers.update(opt, ps, gs)  # update ps using the gradient
    end
end
```

For more details, see [package documentation](#).

Example: Netball-playing robot

Without loss of generality, we can use a coordinate system where $x_0 = 0$ and $y_0 = 0$ to simplify the forward problem to take in (v, θ, t) as inputs and yield (x, y) as outputs. We can use the measured data as training data for a neural network. Let us attempt training a neural network with two hidden layers to learn from the training data:

```
using Lux, Random
import Optimisers, Zygote

# neural network
MLP = Chain(
    Dense(3 ⇒ 16, tanh), # input layer with 3 neurons for (v, θ, t)
    Dense(16 ⇒ 16, tanh), # hidden layer with 16 neurons
    Dense(16 ⇒ 16, tanh), # hidden layer with 16 neurons
    Dense(16 ⇒ 2)         # output layer with 2 neurons yielding (x, y)
)

# initialize MLP parameters ps (and state st)
ps, st = Lux.setup(MersenneTwister(0), MLP)

# prepare training data
vθt = vcat(data.v', data.θ', data.t')
xy = vcat(data.x', data.y')

# loss function
ℒ(ps) = sum(abs2, xy .- Lux.apply(MLP, vθt, ps, st)[1])

# training loop
@info "Loss before training: $(ℒ(ps))"
opt = Optimisers.setup(Optimisers.Adam(0.001f0), ps)
for epoch ∈ 1:5000
    gs = Zygote.gradient(ℒ, ps)[1] # compute gradient wrt ps
    opt, ps = Optimisers.update(opt, ps, gs) # update ps using the gradient
end
@info "Loss after training: $(ℒ(ps))"
```

```
[ Info: Loss before training: 167.27424802957688
[ Info: Loss after training: 0.0037650868449636225
```

We see that the loss after training is small. Let's plot the trajectory predicted by the neural network against the ground truth to see how well it does for two different values of (v, θ) for

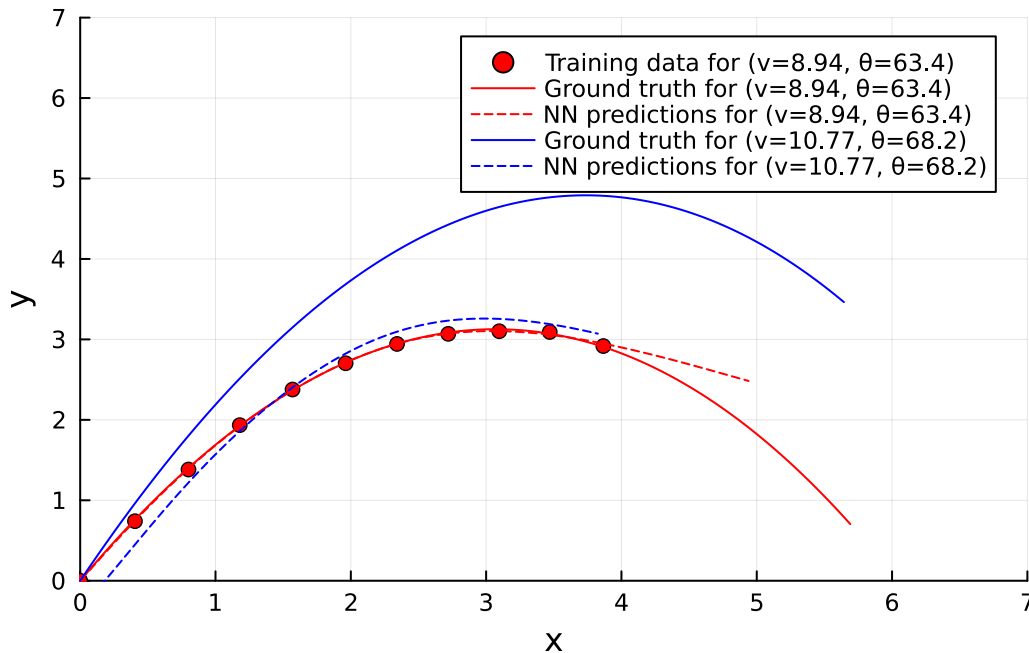


Figure 3.2: Comparison of predicted trajectory from neural network (NN) against ground truth

1.5 seconds in Figure 3.2.

For $(v=8.94, \theta=63.4)$, the predictions are accurate during for the first second, after which the predicted trajectory deviates significantly from the ground truth. For a different initial velocity $(v=10.77, \theta=68.2)$, the predictions are completely inconsistent with the ground truth. This is not surprising. Neural networks are good at interpolating, and hence they make good predictions in the region where we have training data. Beyond this region the neural network performs poorly. Since the training data is only for the first second, it has little information about what the ball might do after that. The training data has no variation in (v, θ) , and so the neural network has no way to learn the dependence on (v, θ) .

If we used this neural network as the surrogate model in Figure 1.4, we would do badly, as this model is poor at mimicking the physics of the real world at any other initial angle other than the trial throw. For a pure data-driven machine learning technique such as a neural network, it isn't possible to do much better. You're welcome to try to show otherwise!

But with neural differential equations, we can...

3.4 Neural Differential Equations

The neural network failed to learn a good forward model after successfully fitting the data because the problem was under-determined. In machine learning, this problem is termed as *over-fitting*. The neural network model has the *capacity* to learn arbitrary functions, and the training data is insufficient to let it distinguish between the function of interest and other functions that match the data. To reduce over-fitting, we have to constrain the function that is learnt by either using a model with more structure (less *capacity*) or through some form of *regularization*. Let us first explore adding structure to the model using our knowledge of physics.

3.4.1 Example: Netball-playing robot

We know that ball must obey Newton's equations of motion with some forces acting on the ball. Forces such as gravity may be constant, whereas others such as drag may depend (perhaps non-linearly) on velocity of the ball, but none depend on the position of the ball¹. We can therefore write the physics of the ball throw as a system of differential equations:

$$\begin{pmatrix} \ddot{x} \\ \ddot{y} \end{pmatrix} = \mathbf{a}(\dot{x}, \dot{y}), \quad (3.3)$$

where $\mathbf{a}(\cdot)$ represents the resultant acceleration from all the forces acting on the ball. Since we do not know the details of the forces, we do not have a symbolic expression for $\mathbf{a}(\cdot)$. However, we can approximate it with a neural network and train the network using available measurements. A differential equation with the derivative defined by a neural network is known as a *neural differential equation* (Chen et al. 2019).

Neural differential equation solver API

The `DiffEqFlux.jl` package implements architectures and utility functions for deep implicit learning, mixing differential equations with machine learning. It supports training of neural ODEs. To define a neural ODE, we first define a neural network using `Lux.jl` or `Flux.jl`:

¹Gravitational variation over space and altitude is small enough to be negligible. We assume that wind or other disturbances do not have a strong position dependence.

```

using Lux
using Random
using ComponentArrays

dudt = Lux.Chain(
    # define the neural network layers here
)

ps, st = Lux.setup(Random.default_rng(), dudt)
ps = ComponentArray(ps)

```

We need to wrap parameters `ps` in a `ComponentArray` for compatibility with many optimization frameworks, as `ps` is a nested `NamedTuple` but optimizers expect a flattened parameter array. We next define a `NeuralODE` with the neural network giving the derivative:

```

using DiffEqFlux

prob = NeuralODE(dudt, tspan, Tsit5(); saveat=ts)

```

where `tspan` is the span over which the ODE is to be solved, and `ts` is the list of points at which the solution is to be saved. The ODE can be solved by calling `prob(u0, ps, st)` where `u0` contains the initial conditions.

A loss function $\mathcal{L}$ measures the difference between the solution of the ODE for neural network parameters `ps` and the training data `data` (measured at each point in `ts`):

```

function L(ps)
    sol, _ = prob(u0, ps, st)      # second return value is updated st
    sum(abs2, Array(sol) .- data)  # Array(sol) returns [u at ts]
end

```

The neural network can be trained by optimizing² over parameters `ps`:

```

using Optimization
using OptimizationOptimisers

# optimize for loss function with derivative computed using Zygote
ofun = OptimizationFunction((ps, _) → ℒ(ps), AutoZygote())

# optimize starting with initial neural network parameters ps
oproblem = OptimizationProblem(ofun, ps)

# optimize using ADAM learning rate 0.01 for 100 iterations
osol = solve(oproblem, Adam(0.01); maxiters=100)

# extract trained parameters from the solution
ps = osol.u

```

For more details, see [NeuralODE documentation](#) and [Optimization.jl documentation](#).

Defining $\mathbf{u} = [x, y, \dot{x}, \dot{y}]^\top$, Equation 3.3 can be written as:

$$\frac{d\mathbf{u}}{dt} = \begin{pmatrix} \dot{x} \\ \dot{y} \\ \mathbf{a}(\dot{x}, \dot{y}) \end{pmatrix}.$$

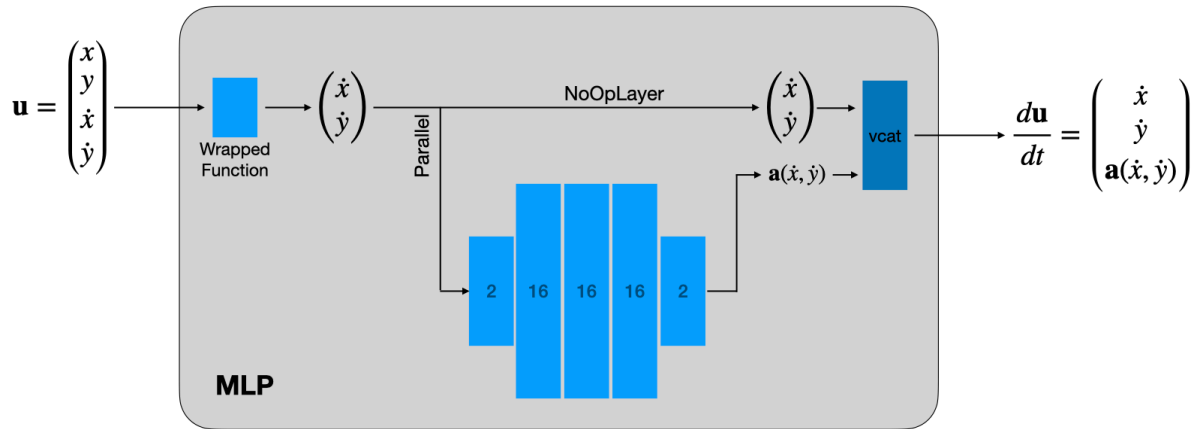
In Julia code, this is defined using a Lux neural network:

```

dudt_model = Chain(
  WrappedFunction(u → u[3:4]), # extract (ẋ, ẏ) from input (x, y, ẋ, ẏ)
  Parallel(vcat,               # output is concatenation of:
    NoOpLayer(),              # inputs (ẋ, ẏ)
    Chain(                     # neural network a(ẋ, ẏ)
      Dense(2 ⇒ 16, tanh),
      Dense(16 ⇒ 16, tanh),
      Dense(16 ⇒ 16, tanh),
      Dense(16 ⇒ 2)
    )
  )

```

²In Section 1.3, we used Optim.jl to perform optimization, whereas we used Optimization.jl here. Why not use Optim.jl here too? The short answer is that we could have. However, Optimization.jl provides wrapper functions that combine capabilities from many optimization packages (including Optim.jl) in the Julia ecosystem using a single API, and integrates well with many AD packages. Here, we require an Adam() optimizer – this isn't available in Optim.jl. We also want to use Zygote.jl for AD – with Optim.jl, we would need to explicitly call gradient() to define a gradient function as an argument to optimize(); with Optimization.jl, we simply have to pass in AutoZygote().

Figure 3.3: Neural network taking in $\mathbf{u}$ and computing $d\mathbf{u}/dt$.

```

    )
  )

  ps, st = Lux.setup(MersenneTwister(0), dudt_model)
  ps = ComponentArray(ps)

```

The network is visualized in Figure 3.3.

The ODE and loss function can then be defined:

```

using DiffEqFlux

u0 = [data.x[1], data.y[1], data.v[1]*cosd(data.θ[1]), data.v[1]*sind(data.θ[1])]
prob = NeuralODE(dudt_model, (data.t[1], data.t[end]), Tsit5(); saveat=data.t)
xy_training = collect(Array{data[:, :x, :y]}')

function ℒ(ps)
  xy_predicted = Array(prob(u0, ps, st)[1])[1:2, :]
  sum(abs2, xy_predicted .- xy_training)
end

```

Finally we train the network:

```

using Optimization, OptimizationOptimisers

```

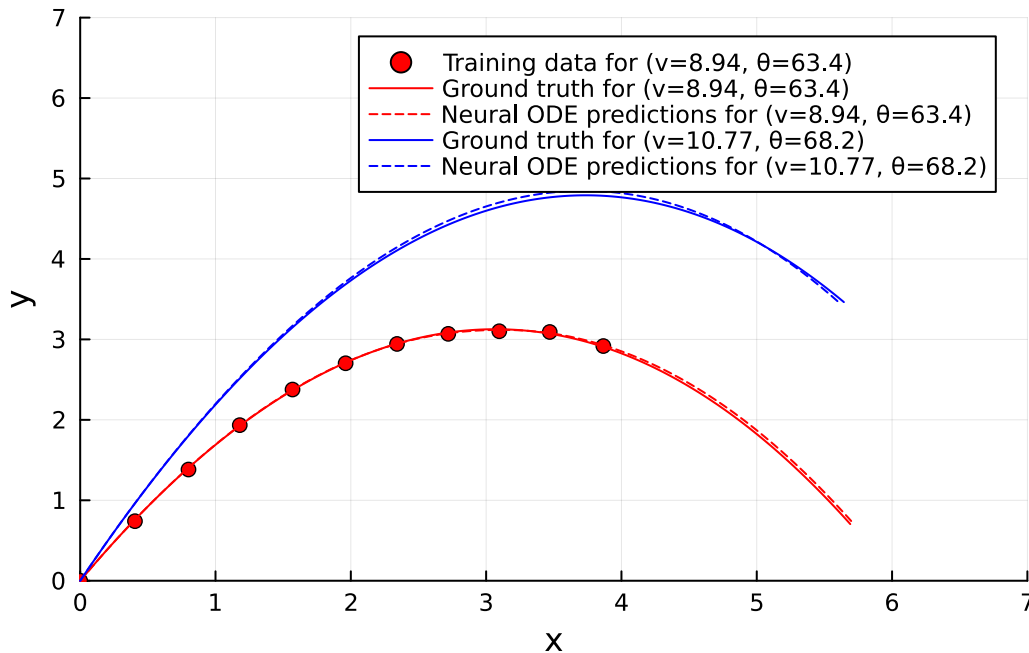



Figure 3.4: Comparison of predicted trajectory from neural ODE against ground truth

```
@info "Loss before training:  $\mathcal{L}(\text{ps})$ "
ofun = OptimizationFunction((ps, _) →  $\mathcal{L}(\text{ps})$ , AutoZygote())
oproblem = OptimizationProblem(ofun, ps)
osol = solve(oprob, Adam(0.01); maxiters=5000)
ps = osol.u
@info "Loss after training:  $\mathcal{L}(\text{ps})$ "
```

```
[ Info: Loss before training: 68.6914718980525
[ Info: Loss after training: 0.001204662882924478
```

We see that the loss after training is small. Let's plot the trajectory predicted by the neural network against the ground truth to see how well it does for two different values of (v, θ) for 1.5 seconds in Figure 3.4.

Amazingly, it has not only learnt to extrapolate in time well, but also figured out how to predict trajectories for other launch angles and velocities accurately!

We now have a trained forward model that can be used to solve the inverse problem like we did in Section 1.3. We can use this as a surrogate model in Figure 1.4 to train an inverse model that the robot can use to make perfect throws after just one trial throw!

i Why do we need a NeuralODE?

The key idea behind a neural differential equation is to solve an ODE with a derivative given by a neural network `dudt_model`, and then to optimize the neural network parameters `ps` to fit the solution of the ODE to data. We could do this by defining `dudt_model` and then solving the ODE:

```
prob = ODEProblem(dudt_model, u0, tspan)
sol = solve(prob, Tsit5())
```

Once we have `sol`, we can define the loss function $\mathcal{L}$ and take the gradient of $\mathcal{L}$ with respect to parameters `ps`. So why do we need a new API `NeuralODE` that we used above? While the solution outlined above will work, it involves reverse-mode AD through the ODE solver. A typical ODE solver is iterative, and may take a very large number of steps. Reverse-mode AD through the solver then requires all of the steps to be stored on a tape for the backward pass. This is memory intensive and often the bottleneck in solving real world problems. It turns out that ODE solutions are reversible in time, i.e., if we start with the final state of a system and reverse time (replace t by $-t$), the solution of the reversed ODE is the original solution flipped around in time. Therefore, rather than store the solution on a tape, an AD system can compute the gradient through an ODE solver by solving the ODE backward (Chen et al. 2019). By defining a `NeuralODE`, we tell the AD system to use that method to compute the gradient of the ODE solver.

3.4.2 Example: Forecasting the spread of an infectious disease

With our recent experience with COVID'19, it is easy to understand the importance of modeling the spread of an infectious disease to inform policy and planning. In this section, we see how we can use data from the initial days of an epidemic to forecast the evolution of the epidemic over months to come.

Problem statement: We live in a small town of a 100,000 people. The world is facing a pandemic. In order to protect the people in the town, the town council declared a lockdown 4 months ago and has severely restricted travel in and out of the town. In spite of that, the town is seeing an increasing number of infections and is concerned about the future. The healthcare personnel in the town have kept careful records of the number of infections and recoveries each day over the past 4 months. While there have been no deaths due to the disease, the load on the healthcare system is substantial. We are tasked with analyzing that data to forecast the likely evolution of the pandemic in the town to help inform town council decisions regarding preparations of ad hoc healthcare facilities to manage the evolving crisis.

Let's first take a look at the available data in Figure 3.5. The data shows the number of new infections and number of recoveries (previously infected people who no longer test positive for

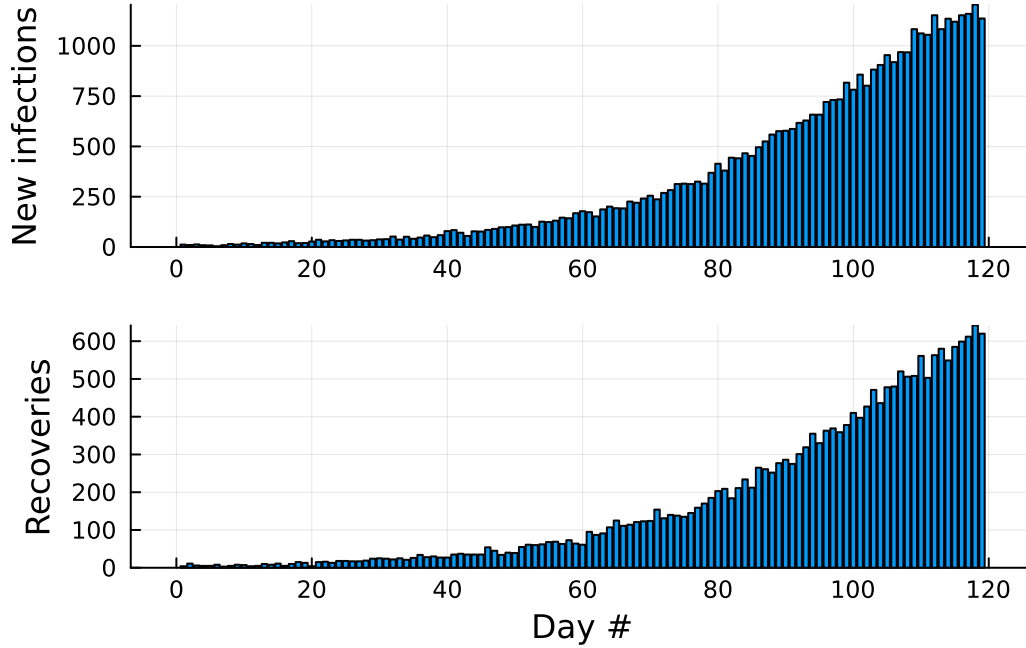


Figure 3.5: Infection and recovery data for first 4 months of the lockdown.

the disease) each day.

Studying some literature on how infectious diseases are modeled, we gain some domain knowledge. Let $S(t)$ be the fraction of population that is susceptible to the disease (not infected or recovered) on day t , $I(t)$ be the fraction of population that is infected, and $R(t)$ be the fraction of population that has recovered. The number of new infections per day would depend on the number of infected individuals and the number of susceptible individuals. Similarly, the number of recoveries each day depend on the number of infected individuals. So, we can write state $\mathbf{u}(t) = [S(t) \ I(t) \ R(t)]^\top$ and its derivative $\frac{d\mathbf{u}(t)}{dt} = [\frac{dS}{dt} \ \frac{dI}{dt} \ \frac{dR}{dt}]^\top$ with:

$$\begin{aligned} \frac{dS}{dt} &= -f(S(t), I(t)), \\ \frac{dI}{dt} &= f(S(t), I(t)) - g(I(t)), \\ \frac{dR}{dt} &= g(I(t)), \end{aligned} \tag{3.4}$$

where $f(\cdot, \cdot)$ models the fraction of population being infected and $g(\cdot)$ models the fraction of population recovering. While we do not know the exact form of $f(\cdot, \cdot)$ and $g(\cdot)$, we do know that both must be non-negative functions with $f(0, \cdot) = 0$, $f(\cdot, 0) = 0$ and $g(0) = 0$. We can model Equation 3.4 as a neural network with a structure designed to enforce these constraints:

```

model = Chain(                                # inputs: [S, I, R]
  Parallel(vcat, Chain(
    Dense(3 ⇒ 8, tanh), # inputs: [S, I, R]
    Dense(8 ⇒ 8, tanh),
    Dense(8 ⇒ 8, tanh),
    Dense(8 ⇒ 2, σ)      # outputs: [ $\bar{f}(S, I)$ ,  $\bar{g}(I)$ ]
  ), NoOpLayer()),
  # concatenated into [ $\bar{f}(S, I)$ ,  $\bar{g}(I)$ , S, I, R]
  WrappedFunction(x → let  $\bar{f}$  = x[1],  $\bar{g}$  = x[2], S = x[3], I = x[4]
    [ - $\bar{f}$  * S * I,          # dS/dt
       $\bar{f}$  * S * I -  $\bar{g}$  * I,  # dI/dt
       $\bar{g}$  * I ]              # dR/dt
    end)
)

```

The neural network is visualized in Figure 3.6. The constraints are imposed by modeling:

$$\bar{f}(S, I) = \frac{f(S, I)}{SI},$$

$$\bar{g}(I) = \frac{g(I)}{I},$$

as non-negative functions using a neural network. Since:

$$f(S, I) = SI \bar{f}(S, I),$$

$$g(I) = I \bar{g}(I),$$

we have $f(S, I) = 0$ if either $S = 0$ or $I = 0$ and $g(I) = 0$ if $I = 0$. The σ activation function ensures that $\bar{f}(S, I)$ and $\bar{g}(I)$ are both non-negative.

Now, we are ready to write this as a neural differential equation $\frac{d\mathbf{u}}{dt} = \text{model}(\mathbf{u})$ and train it:

```

import OptimizationOptimJL: BFGS

ps, st = Lux.setup(MersenneTwister(0), model)
ps = ComponentArray(ps)

# neural ODE solved over 120 days of available training data
prob = NeuralODE(model, (0f0, 119f0), Tsit5(); saveat=1)

# prepare training data (convert counts S, I, R to fractional population)
training = vcat(data.S', data.I', data.R') / 100000

```

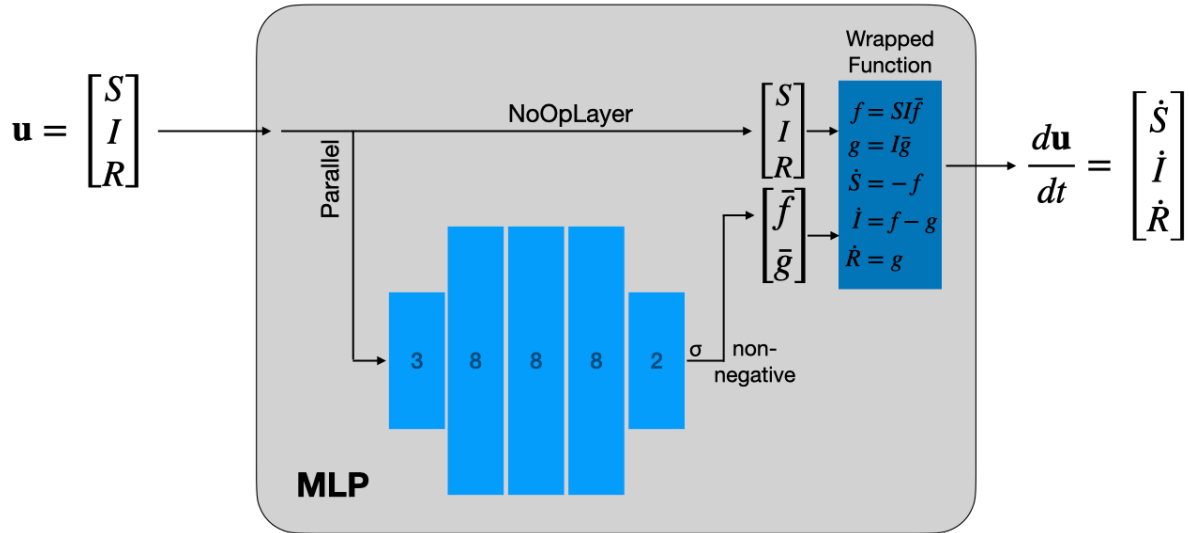


Figure 3.6: Neural network modeling Equation 3.4.

```
# loss function
function  $\mathcal{L}$ (ps)
    predicted = Array{prob(training[:,1], ps, st)[1]}
    sum(abs2, predicted .- training)
end

# prepare to train with gradient taken using ForwardDiff
# ForwardDiff is faster than Zygote for small problems
@info "Loss before training:  $\mathcal{L}(\text{ps})$ "
ofun = OptimizationFunction((ps, _) →  $\mathcal{L}$ (ps), AutoForwardDiff())
oprob = OptimizationProblem(ofun, ps)

# initially use Adam, which is good at finding a rough solution
osol = solve(oprob, Adam(0.001); maxiters=2000)
@info "Loss after ADAM training:  $\mathcal{L}(\text{osol.u})$ "

# then finish off with BFGS, which is better at converging to a local minimum
# we use remake() to start the optimization from where Adam left off
osol = solve(remake(oprob; u0=osol.u), BFGS())
@info "Loss after BFGS training:  $\mathcal{L}(\text{osol.u})$ "
```

```
[ Info: Loss before training: 11.183172235793508
[ Info: Loss after ADAM training: 0.0006184511352387585
```

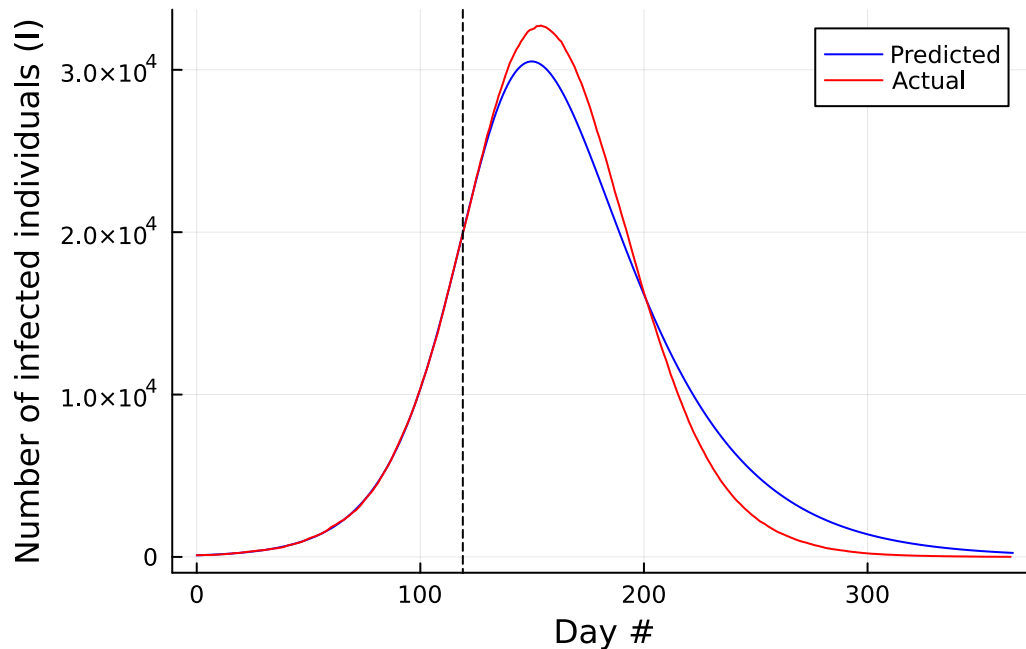


Figure 3.7: Comparison of forecasted infections against ground truth (what actually happened in the hypothetical town in the later months). The vertical dashed line shows the time at which the forecast was made, i.e., data before it was available for training.

[Info: Loss after BFGS training: 2.3155306362836305e-5

Once the training is complete, we can use the trained neural ODE to forecast infections:

```
forecast_prob = NeuralODE(model, (0f0, 365f0), Tsit5(); saveat=1.0)
forecast = forecast_prob(training[:,1], osol.u, st)[1]
plot(forecast; idxs=2) # index 2 refers to I(t)
```

The results are shown in Figure 3.7. We also show the ground truth of what happened in the pandemic over the year for comparison. We see that while the predictions are not perfect (since the model is heavily extrapolating), it predicts the peak of the infections fairly well, and also models the recovery phase reasonably.

💡 Universal differential equations

Neural differential equations assume that we know nothing about the derivative of the state of the dynamic system, and we model it as a neural network. Sometimes we know the basic structure of the derivative, and only have a small part of the derivative that

is unknown. In such cases, we can define the derivative using a mix of a deterministic function and a neural network. Such differential equations are called *universal differential equations* (Rackauckas et al. 2020), and can also be solved using `DiffEqFlux.jl`. For more details, see [package documentation](#).

3.5 Physics-informed Neural Networks

In the previous section, we saw how to add structure to a machine learning model by using differential equations to encode our domain knowledge. In this section, we explore an alternative approach to regularization – one that adds terms that encode our domain knowledge to the loss function. A neural network trained with a loss function that requires the network to fit the data and also domain knowledge encoded in a differential equation is known as a *Physics-informed Neural Network* (PINN).

! Important

Unlike in the case of neural differential equations, where the neural network estimated the derivative $d\mathbf{u}/dt$ in an unknown differential equation, the physics-informed neural network approximates the solution $\mathbf{u}(t)$ to a known differential equation.

Before we attempt solving our Netball-playing robot problem using a PINN, let us understand how a PINN works through a simpler illustrative example. We will solve the well-known logistic equation – a famous first order ODE that is used to model population growth – using a PINN. While this problem can be solved using more traditional methods, using a PINN to solve it will help us understand how a PINN works.

3.5.1 Example: Ecological modeling

The logistic equation is:

$$\frac{du}{dt} = ru(1 - u), \quad (3.5)$$

where $u(t)$ is the fractional population at time t and r is the rate of maximum population growth (also known as the Malthusian parameter). This equation is typically used to model population dynamics in ecology, where $u(t)$ represents the population expressed as a fraction of the maximum population that resources in an ecosystem can support. The equation says that the rate of population growth is proportional to the population u (due to reproduction) and the available resources $(1 - u)$ (as population increases, available resources decrease). The Malthusian parameter r is simply the constant of proportionality, and is experimentally measured for an ecosystem being studied.

The analytical solution to the logistic equation is:

$$u(t) = \frac{1}{1 + \left(\frac{1}{u_0} - 1\right)e^{-rt}}, \quad (3.6)$$

where $u_0 = u(0)$. If u_0 and r are known, the value of $u(t)$ can be determined for any time t . On the other hand, if u_0 and r are not known, they may be determined from noisy observations of $u(t)$ through regression.

For illustrative purposes, however, we will assume we do not know the solution in Equation 3.6 and instead approximate $u(t)$ with a neural network:

```
u_model = Chain(
    Dense(1 ⇒ 8, tanh),
    Dense(8 ⇒ 8, tanh),
    Dense(8 ⇒ 1)
)

ps, st = Lux.setup(MersenneTwister(0), u_model)
ps = ComponentArray(ps)
```

We do not know $u(0)$ or r , but we are given the following noisy measurements (`u_measured`) of fractional population at three different points in time:

i	t	u
1	0.2	0.133818
2	0.5	0.194571
3	1.5	0.511426

If we define a loss function as the error between predictions and data:

```
u_pred(t, ps) = Lux.apply(u_model, t', ps, st)[1]'
ℒ(ps) = sum(abs2, u_pred(u_measured.t, ps) .- u_measured.u)
```

and train the neural network to minimize it:

```
import ForwardDiff

@info "Loss before training: $(ℒ(ps))"
opt = Optimisers.setup(Optimisers.Adam(0.01f0), ps)
for epoch ∈ 1:500
    # ForwardDiff is faster than Zygote for small problems
```

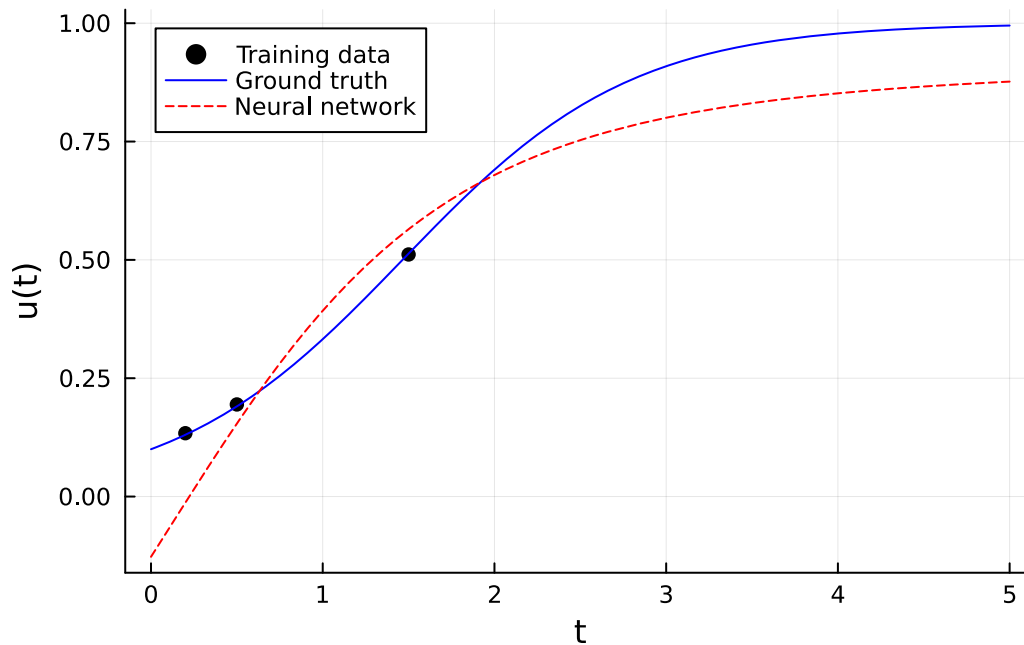



Figure 3.8: Comparison of neural network predicted population growth against training data and ground truth

```
gs = ForwardDiff.gradient(L, ps)[1]
opt, ps = Optimisers.update(opt, ps, gs)
end
@info "Loss after training: $(L(ps))"
```

Comparing the predictions from the trained neural network with training data and the real solution (ground truth) in Figure 3.8, we see that the neural network matches the training data perfectly but does not approximate the real solution.

Recall, however, that we know more than just the training data – we also know the derivative of the desired function from Equation 3.5. We can compute the derivative of the neural network with respect to its input and minimize its mismatch to Equation 3.5 to guide the training. We therefore add a second term to the loss function to measure the mismatch against the model at a discrete set of grid points:

```
# use AD to compute du/dt
dudt_pred(t, ps) = ForwardDiff.derivative(t → u_pred([t], ps)[1], t)[1]
```

```

# t_grid defines grid points, and  $\alpha$  balances between the two loss terms
function  $\mathcal{L}$ (ps; t_grid=0:0.1:5,  $\alpha$ =0.05)
    loss_data = sum(abs2, u_pred(u_measured.t, ps.model) .- u_measured.u)
    u_grid = u_pred(t_grid, ps.model)
    dudt_grid = dudt_pred.(t_grid, Ref(ps.model))
    loss_model = sum(abs2, dudt_grid .- ps.r .* u_grid .* (1 .- u_grid))
    loss_data +  $\alpha$  * loss_model
end

```

The parameters `ps` above are to be passed in as a `ComponentArray` with `ps.model` being the neural network parameters and `ps.r` being the `r` value to be estimated. We retrain the model using this new loss function:

```

ps, st = Lux.setup(MersenneTwister(0), u_model)
ps = ComponentArray(model=ps, r=1.0)

@info "Loss before training:  $\mathcal{L}(ps)$ "
opt = Optimisers.setup(Optimisers.Adam(0.01f0), ps)
for epoch  $\in$  1:500
    gs = ForwardDiff.gradient( $\mathcal{L}$ , ps)
    opt, ps = Optimisers.update(opt, ps, gs)
end
@info "Loss after training:  $\mathcal{L}(ps)$ "

```

```

[ Info: Loss before training: 0.20893244042372225
[ Info: Loss after training: 0.00018540114257830203

```

Comparing the predictions from the trained PINN with training data and the real solution (ground truth) in Figure 3.9, we see that the neural network is a much better match for the training data and the real solution.

3.5.2 Using `NeuralPDE.jl` to build physics-informed neural networks

PINNs are versatile and can even be used to solve partial differential equations (PDEs), but are notoriously difficult to train (Wang et al. 2023). Different training strategies and discretization methods have been developed to help improve the training. Rather than implement them from scratch, it is useful to work with a library that aids with PINN development. `NeuralPDE.jl` allows us to write differential equations in a domain-specific language (DSL) and train PINNs based on such descriptions. To understand how to do this, let us re-implement the PINN for solving the logistic equation using the `NeuralPDE.jl` API.

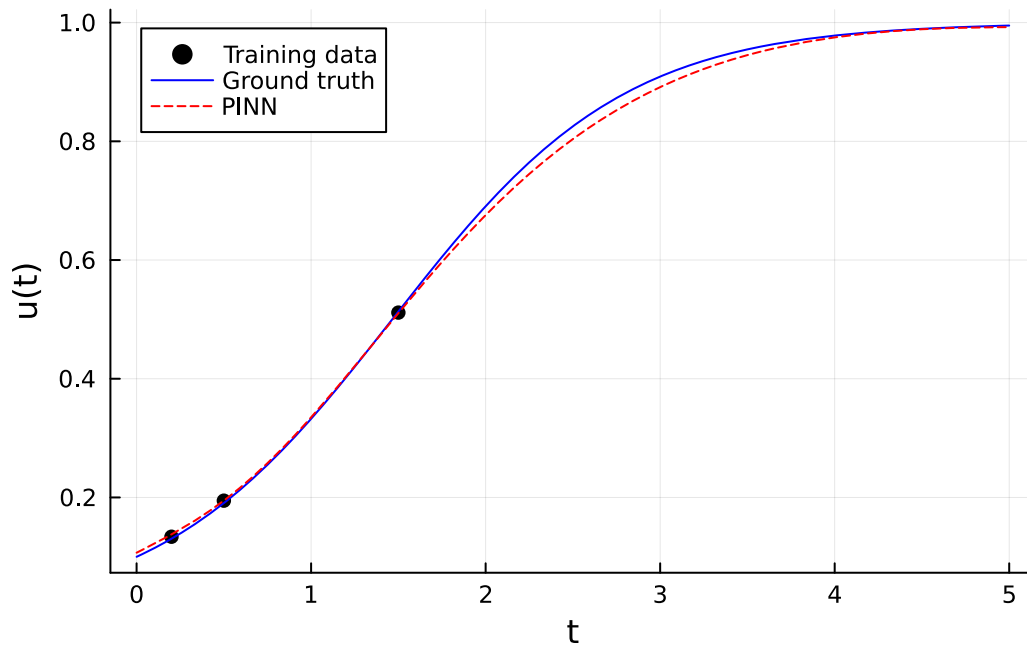


Figure 3.9: Comparison of PINN predicted population growth against training data and ground truth

First we define the independent variables, dependent variables and parameters used in the model:

```
using NeuralPDE, Lux, OptimizationOptimJL
import ModelingToolkit: Interval

@parameters t, r      # independent variable t and parameter r
@variables u(..)      # dependent variable u(t)
```

We then define the differential operator $\frac{d}{dt}$ and use it to define the logistic differential Equation 3.5:

```
Dt = Differential(t) # d/dt operator

# system of differential equations
eqs = [
    Dt(u(t)) ~ r * u(t) * (1 - u(t))
]
```

Next we define the independent variable domain and known boundary conditions:

```
# one domain per independent variable
domains = [
    t ∈ Interval(0.0, 5.0)
]

# boundary conditions based on measured data, one entry per measurement
bcs = map(data → u(data.t) ~ data.u, eachrow(u_measured))
```

We then define the Lux chain to use for the neural network:

```
# one chain per dependent variable
chains = [
    Chain(Dense(1 ⇒ 8, tanh), Dense(8 ⇒ 8, tanh), Dense(8 ⇒ 1))
]
```

Now we are ready to define the PINN and discretize it over a grid:

```
pinn = PhysicsInformedNN(chains, GridTraining(0.1); param_estim=true)
@named pde = PDESystem(eqs, bcs, domains, [t], [u(t)], [r]; defaults=Dict(r⇒1.0))
```

and finally solve it:

```
prob = discretize(pde, pinn)
sol = solve(prob, BFGS())
```

The `param_estim=true` option tells the PINN that we want to estimate parameters (in this case `[r]` with initial value `1.0`) as part of training.

Once we have the solution `sol`, we can use the trained neural network weights `sol.u.depvar.u` to estimate `u` at any time `t` (either a scalar or a row vector) using `pinn.phi[1](t, sol.u.depvar.u)`. The `pinn.phi[1]` is the neural network for the first dependent variable in the PINN. Let's plot it to check the solution in Figure 3.10.



Which optimizer to use?

`Adam()` is one of the more popular optimizers for deep learning today. In the above example, however, we used `BFGS()`. Had we used `Adam(0.01)` with `maxiters=5000`, it would have worked as well too – but `BFGS()` converges more quickly for this problem. How do we decide which optimizer is best suited to a problem? Sometimes we just have to try

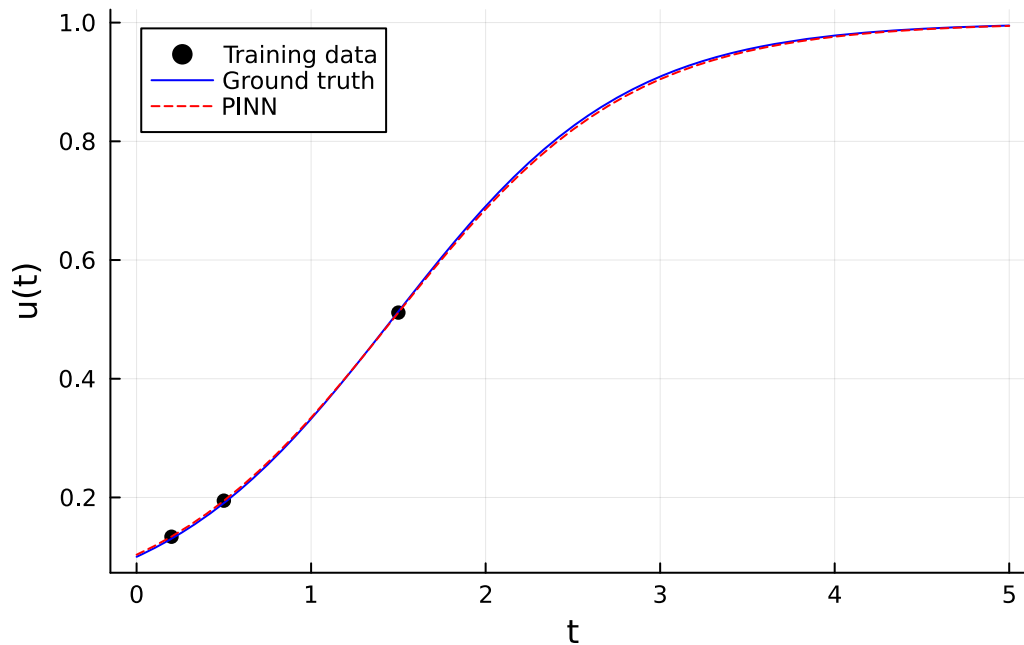


Figure 3.10: Comparison of PINN predicted population growth against training data and ground truth using `NeuralPDE.jl`

different ones to see what works, but having an understanding of potential issues is helpful in making educated guesses as to what might work.

`Adam()` is based on gradient descent, and progresses in the direction of negative gradient with a step size that is dependent on the learning rate. This works well when all parameters are scaled appropriately, but has convergence problems if they are scaled poorly (Holl, Koltun, and Thuerey 2022). It is therefore a common practice to scale inputs and outputs in neural networks to be roughly span ± 1 , and in some deep networks, normalization layers are introduced to ensure that the intermediate computations are also appropriately scaled. It is a good practice in physics-based learning problems to also scale inputs and outputs to aid with convergence; however, intermediate normalization may not always be consistent with physics, and therefore impossible to do. `BFGS()` is a quasi-Newton method that not only uses gradient, but also the Hessian to determine direction and step size, and often gets around the problem of poor scaling. However, it requires the Hessian of the loss function to exist and be computable efficiently. Other scale invariant optimizers have been used in some problems with good success (Holl, Koltun, and Thuerey 2022).

While `Adam()` usually converges rapidly in the initial phases of training, its convergence in the neighborhood of an optimum can be slow. `BFGS()` works well close to an optimum,

but often not as well further away, especially if the loss surface is not smooth. In some problems, it pays to start with `Adam()`, and after a fixed number of iterations, switch to using `BFGS()` to finish the optimization. This may look something like this:

```
sol = solve(prob, Adam(0.01); maxiters=100)
prob = remake(prob; u0=sol.u) # use Adam solution to initialize BFGS
sol = solve(prob, BFGS())
```

Tip

The [NeuralPDE.jl documentation](#) is well worth exploring, as it gives examples of ODE and PDE problems from various domains where neural differential equations or their variants may be applied.

3.5.3 Example: Netball-playing robot

Now that we know how to use PINN to combine partial knowledge of physics with observed data, let us go back to the netball-playing robot problem and see whether we can apply what we learned to it.

In Section 3.4, we assumed that the ball obeys Newton's equations of motion with unknown forces acting on it, and used a neural network to learn the acceleration due to the forces from the data. In many cases, we perhaps know a bit more physics – for example, we may know that gravity and drag forces act on the ball, but we may not know the drag coefficient, cross-sectional area of the ball, density of air, gravitational acceleration, and whether the netball court has some slope. Perhaps we can model these parameters as unknowns to be estimated from data, just like we did with the Malthusian parameter r in the last section?

Let g_x and g_y be the unknown acceleration due to gravity in x and y directions respectively. g_x may be non-zero if the netball court has some slope. Let $K = -\frac{1}{2}C_d\rho A$ be a lumped constant related to the drag force on the ball. The Equation 3.1 can be rewritten as:

$$\begin{aligned}\ddot{x} &= K\nu\dot{x} + g_x, \\ \ddot{y} &= K\nu\dot{y} + g_y,\end{aligned}\tag{3.7}$$

where $\nu = \sqrt{\dot{x}^2 + \dot{y}^2}$.

We can model this system of differentiation equations using `NeuralPDE.jl`:

```
using NeuralPDE
import ModelingToolkit: Interval
```

```

@parameters t, K, gx, gy
@variables x(..), y(..)

Dt = Differential(t)          # d/dt operator
Dtt = Differential(t)^2      # d^2/dt^2 operator

eqs = [
    Dtt(x(t)) ~ K * sqrt(Dt(x(t))^2 + Dt(y(t))^2) * Dt(x(t)) + gx,
    Dtt(y(t)) ~ K * sqrt(Dt(x(t))^2 + Dt(y(t))^2) * Dt(y(t)) + gy
]

domains = [t ∈ Interval(0.0, 1.5)]

```

The boundary conditions include the initial velocity $[\dot{x}(0), \dot{y}(0)]$ and noisy measurements (data from Section 3.2) of one throw of the ball:

```

bcs = vcat(
    [x(m.t) ~ m.x for m ∈ eachrow(data)],
    [y(m.t) ~ m.y for m ∈ eachrow(data)],
    Dt(x(0.0)) ~ data.v[1] * cosd(data.θ[1]),
    Dt(y(0.0)) ~ data.v[1] * sind(data.θ[1])
)

```

Next we define two neural networks, one for approximating $x(t)$ and the other for approximating $y(t)$:

```

using Lux

chains = [
    # neural network to approximate x(t)
    Chain(Dense(1 ⇒ 16, tanh), Dense(16 ⇒ 16, tanh), Dense(16 ⇒ 1)),
    # neural network to approximate y(t)
    Chain(Dense(1 ⇒ 16, tanh), Dense(16 ⇒ 16, tanh), Dense(16 ⇒ 1))
]

```

Finally, we put together the PINN and solve it:

```

using OptimizationOptimJL

```

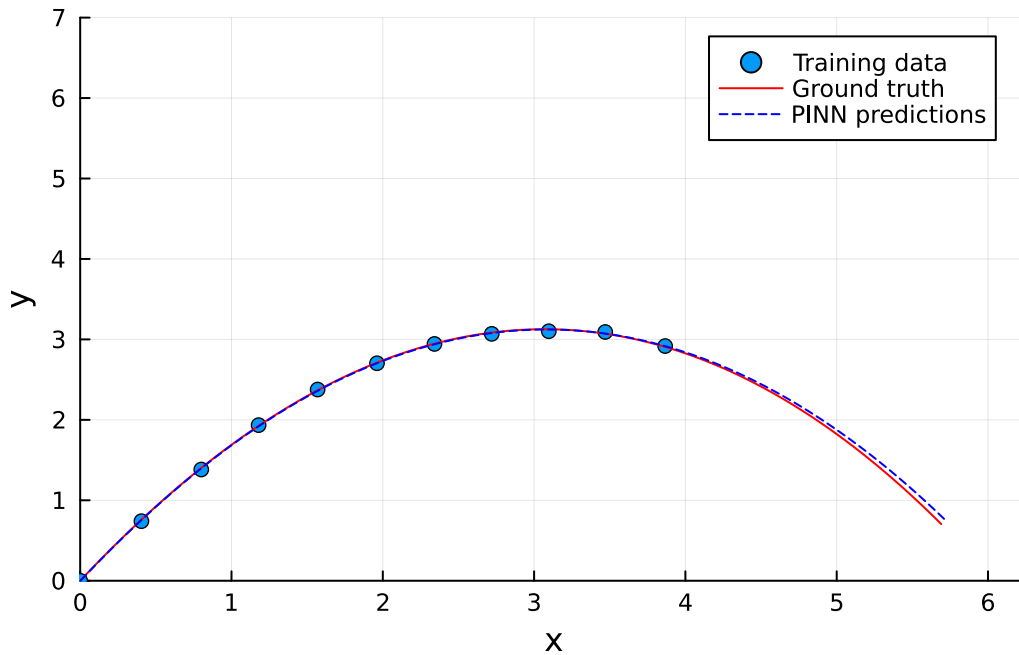


Figure 3.11: Comparison of PINN predicted trajectory against ground truth

```

pinn = PhysicsInformedNN(chains, GridTraining(0.01); param_estim=true)
@named pde = PDESystem(eqs, bcs, domains, [t], [x(t), y(t)], [K, gx, gy];
  defaults=Dict{K⇒0.0, gx⇒0.0, gy⇒-9.8})
prob = discretize(pde, pinn)
sol = solve(prob, BFGS(); maxiters=250)

```

The result is shown in Figure 3.11.

Now that we know K , g_x and g_y from the training, we can use it to predict trajectories for other initial velocities too:

```

eqs2 = let (K, gx, gy) = sol.u.p
  [ Dt(x(t)) ~ K * sqrt(Dt(x(t))^2 + Dt(y(t))^2) * Dt(x(t)) + gx,
    Dt(y(t)) ~ K * sqrt(Dt(x(t))^2 + Dt(y(t))^2) * Dt(y(t)) + gy ]
end

bcs2 = [
  x(0.0) ~ 0.0,
  y(0.0) ~ 0.0,

```

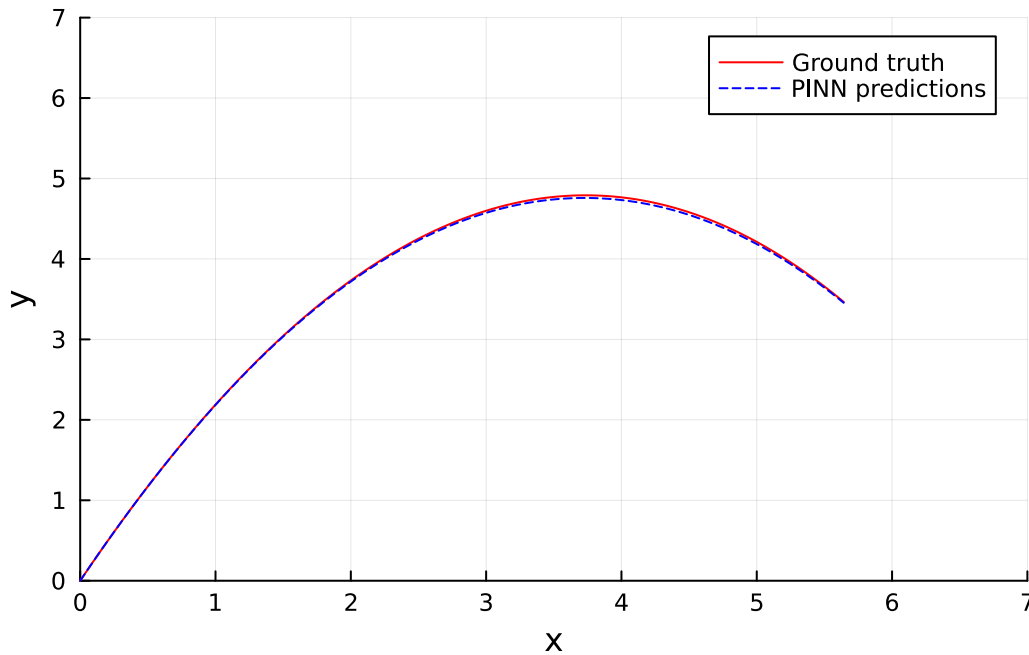



Figure 3.12: Comparison of PINN predicted trajectory against ground truth for a different initial velocity

```

Dt(x(0.0)) ~ 4.0,
Dt(y(0.0)) ~ 10.0
]

pinn2 = PhysicsInformedNN(chains, GridTraining(0.01))
@named pde2 = PDESystem(eqs2, bcs2, domains, [t], [x(t), y(t)])
prob2 = discretize(pde2, pinn2)
sol2 = solve(prob2, BFGS(); maxiters=250)

```

The results are shown in Figure 3.12.

We see that the trained PINN is able to predict the ball trajectory accurately extrapolating well beyond the training data.

3.6 Summary

Domain knowledge (generically called *physics*) in many problems is written in the language of differential equations. We studied the differential equation solver API in Julia to learn how to

solve such equations if the physics is fully known.

When physics is unavailable or partially known, but measurements are available, one may resort to machine learning methods to learn from data. We saw that traditional machine learning methods require substantial data to train from, and fail to extrapolate beyond available data. On the other hand, techniques such as *neural differential equations* and *physics-informed neural networks* are able to combine partial knowledge of physics with neural networks to learn from small datasets and extrapolate well beyond the available data.

Neural differential equations impose physics-based constraints structurally by modeling unknown derivative terms in the governing differential equations using neural networks. The governing differential equations can be learned from data. Physics-informed neural networks, on the other hand, use physics in the loss function. They require the differential equations to be known (perhaps with some unknown parameters) and model the solution to the differential equation as a neural network. Both methods are powerful ways to combine domain knowledge and data-driven methods to yield powerful techniques with wide applications across domains.

4 Probabilistic Programming

The real world is filled with uncertainty. Even if we know the relationship between quantities with certainty, any useful inference using the relationship involves some inputs from the real world in terms of measurements. Uncertainty creeps in into computation in the form of measurement noise. Rather than consider the effect of noise on a computation as an afterthought, it is sometimes useful to embrace uncertainty and weave it into the computation from the onset. Instead of input and output quantities of a computation taking definite values, they can be described by probability distributions. In this chapter, we learn how to work with uncertain quantities in both forward and inverse problems.

Roadmap

- We start this chapter by exploring a few ways of **describing uncertainty**.
- These descriptions lead to algorithms that enable us to **compute with uncertainty**. We explore some Julia packages that implement these algorithms.
- **Bayesian inference** is one of the most powerful tools that enables us to draw conclusions based on uncertain information and beliefs. We briefly review the mathematics of it.
- To help us describe complicated problems for automated Bayesian inference, we learn a **probabilistic programming language** called Turing.jl.
- Bayesian inference problems are often solved using **Markov Chain Monte Carlo** methods that are based on sophisticated sampling algorithms. We study some simple sampling algorithms to get an understanding of how they work, and then learn to apply more sophisticated sampling algorithms to our probabilistic programs.
- Monte Carlo methods are powerful but computationally intensive. Sometimes, we can obtain analytical approximations to the posterior through a process called **variational inference**. In the final section, we briefly study how variational inference works, and how we can use it to solve Bayesian inference problems.

4.1 Describing Uncertainty

We think of an uncertain quantity not to have definite value but instead to be described by a probability distribution (henceforth just called a *distribution*). While we often think of a distribution concretely in terms of a function that yields the probability (or probability density)

of an event, it is only one of many ways to represent that distribution. More abstractly, a distribution is anything that may be used to answer questions about the value the quantity of interest might take. Some examples of the questions one may ask a distribution:

1. What is the probability that the quantity has a certain value?
2. What is the probability that the quantity has a value within a given interval?
3. What is the expected value of the quantity or some function of the quantity?
4. What is the range of values the quantity might take on with high probability?
5. What are some example values (random variates) that the quantity is likely to take on?

Probability mass or density function representation of a distribution

While a probability mass function (pmf), probability density function (pdf) or a cumulative density function (cdf) are commonly used to help answer questions 1–5, these are not the only concrete representations of a distribution.

Characteristic function representation of a distribution

For example, knowledge of the characteristic function of a distribution enables one to answer these questions.

The *characteristic function* (cf) of a random variable X is closely related to the Fourier transform of the pdf of X , if the pdf exists. Specifically cf:

$$\phi_X(t) = \mathbb{E}[e^{itX}],$$

where $i = \sqrt{-1}$ and $\mathbb{E}[\cdot]$ is the expectation operator. The cf of a random variable exists even in cases where the pdf does not, and completely describes its distribution. For example, we can compute the n^{th} moment of the random variable X directly from the cf:

$$\mathbb{E}[X^n] = i^{-n} \left[\frac{d^n}{dt^n} \phi_X(t) \right]_{t=0}.$$

...

Having a procedure to generate an infinite number of random variates from a distribution is also sufficient to answer all the above questions.

Sampling procedure representation of a distribution

! Important

The knowledge of a procedure to generate an infinite number of random variates from a distribution is equivalent to knowing the distribution.

If we have a sampling procedure for a distribution of a random variable X , we can generate a set $\{X_1, X_2, \dots, X_N\}$ of a random variates from that distribution. We can answer questions about the distribution from this set as $N \rightarrow \infty$. For example:

1. The probability $\mathbb{P}[X = x] = \frac{1}{N} \sum_{i=1}^N \delta(X_i - x)$, where $\delta(\cdot)$ is the Kronecker delta function.
2. The probability $\mathbb{P}[x_{\min} \leq X \leq x_{\max}] = \frac{1}{N} \sum_{i=1}^N \mathbf{1}_{[x_{\min}, x_{\max}]}(X_i)$, where $\mathbf{1}_{\Phi}(\cdot)$ is an indicator function on set Φ .
3. The probability $\mathbb{P}[f(X) = y] = \frac{1}{N} \sum_{i=1}^N \delta(f(X_i) - y)$, for any function $f(\cdot)$.

For finite N , the answers are approximate. However, we can get answers to any desired level of accuracy by controlling N .

Methods that use sampling procedures to generate random variates to compute samples from a distribution of interest are known as *Monte Carlo* methods. ::

We will use different representations of distributions in solving different problems.

4.2 Computing with Uncertainty

Consider quantity $x \sim \mathcal{N}(\mu_x, \sigma_x)$, where ‘ $\sim$ ’ is read as ‘is distributed as’ and $\mathcal{N}(\mu_x, \sigma_x)$ represents a Normal (Gaussian) distribution with mean μ_x and standard deviation σ_x . We are interested in the distribution of the output w of some function $f(\cdot)$ applied to x :

$$w = f(x).$$

If $f(\cdot)$ is affine,

$$w \sim \mathcal{N}\left(f(\mu_x), \left|\frac{df}{dx}\right|_{\mu_x} \sigma_x\right).$$

If $f(\cdot)$ is not affine, the distribution of w is not guaranteed to be Gaussian. However, for any quantity x (not necessarily Gaussian distributed) with mean μ_x and standard deviation σ_x (we write it as $\mu_x \pm \sigma_x$), if σ_x is small and $f(\cdot)$ is well-behaved (Taylor series exists), we can approximate the mean μ_w and standard deviation σ_w using a linearization of $f(\cdot)$ around μ_x :

$$\begin{aligned} \mu_w &= f(\mu_x), \\ \sigma_w &\approx \left|\frac{df}{dx}\right|_{\mu_x} \sigma_x. \end{aligned}$$

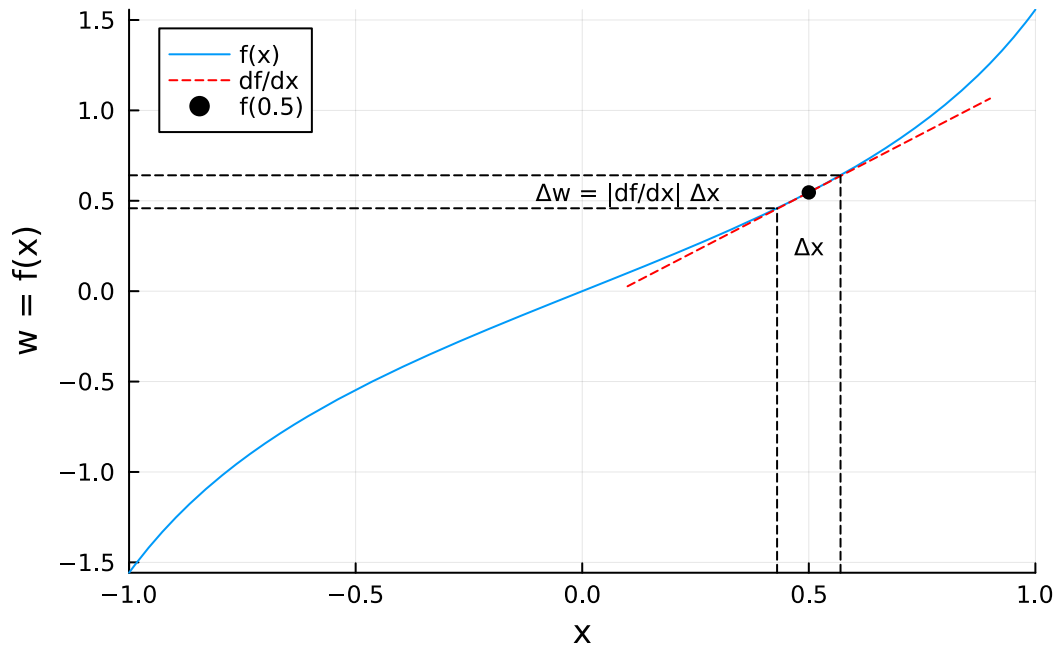


Figure 4.1: A small uncertainty Δx maps to uncertainty $\Delta w = |df/dx| \Delta x$ when passed through a function $w = f(x)$.

Figure 4.1 illustrates the linearization and mapping of uncertainty from x to $w = f(x)$ graphically.

Measurements.jl

The `Measurements.jl` package enables computation with uncertainty in Julia. We can define an uncertain quantity:

```
using Measurements
x = 1.0 ± 0.1
```

and compute with it:

```
sin(x)
```

```
0.841 ± 0.054
```

We verify that the answer is as per the mathematics described above:

```
import ForwardDiff: derivative
(sin(1.0), derivative(sin, 1.0) * 0.1)
```

```
(0.8414709848078965, 0.05403023058681398)
```

We can use `Measurements.jl` to work with much more complicated functions:

```
function f(x, y)
    s = 0.0
    for i ∈ 1:3
        s += x^i + y^(-i)
    end
    s
end

f(2.0 ± 0.01, 3.2 ± 0.02)
```

```
14.44 ± 0.17
```

For more details, see [package documentation](#).

Consider another quantity $y = \mu_y \pm \sigma_y$. We may be interested in knowing the distribution of the output w of some well-behaved non-linear function $f(\cdot, \cdot)$ applied to x and y :

$$w = f(x, y).$$

Again, generally, we cannot calculate the distribution analytically, but can approximate the mean μ_w and standard deviation σ_w (Giordano 2016):

$$\begin{aligned} \mu_w &= f(\mu_x, \mu_y), \\ \sigma_w^2 &\approx \left(\frac{\partial f}{\partial x} \sigma_x \right)^2 + \left(\frac{\partial f}{\partial y} \sigma_y \right)^2 + 2 \frac{\partial f}{\partial x} \frac{\partial f}{\partial y} \sigma_{xy}, \end{aligned} \quad (4.1)$$

where the partial derivatives are evaluated at (μ_x, μ_y) , and σ_{xy} is the covariance between x and y . If x and y are independent, this simplifies to:

$$\sigma_w^2 \approx \left(\frac{\partial f}{\partial x} \sigma_x \right)^2 + \left(\frac{\partial f}{\partial y} \sigma_y \right)^2.$$

Covariances are important

An incorrect independence assumption can lead to wrong answers. Consider a simple example:

$$f(x, y) = x - y$$

With $x = 1.0 \pm 0.1$, we wish to find the distribution of $w = f(x, x)$. Since $w = x - x = 0$ for all values of x , clearly there is no uncertainty in w . The correct answer is $w = 0.0 \pm 0.0$. This is consistent with Equation 4.1:

$$\begin{aligned}\sigma_w^2 &\approx \left(\frac{\partial f}{\partial x}\sigma_x\right)^2 + \left(-\frac{\partial f}{\partial x}\sigma_x\right)^2 - 2\frac{\partial f}{\partial x}\frac{\partial f}{\partial x}\sigma_{xx} \\ &= 2\left(\frac{\partial f}{\partial x}\sigma_x\right)^2 - 2\left(\frac{\partial f}{\partial x}\right)^2\sigma_x^2 \\ &= 0.\end{aligned}$$

However, if we made an independence assumption ($\sigma_{xx} = 0$), we would get an incorrect answer:

$$\begin{aligned}\sigma_w^2 &\approx \left(\frac{\partial f}{\partial x}\sigma_x\right)^2 + \left(-\frac{\partial f}{\partial x}\sigma_x\right)^2 \\ &= 2\left(\frac{\partial f}{\partial x}\sigma_x\right)^2 \\ &= 2 \times 0.1^2 = 0.02,\end{aligned}$$

i.e., $\sigma_w = 0.1414$.

The `Measurements.jl` package handles covariances correctly:

```
f(x, y) = x - y
x = 1.0 ± 0.1
y = 1.0 ± 0.1
@show f(x, x);      # x and x are not independent
@show f(x, y);      # x and y are independent
```

```
f(x, x) = 0.0 ± 0.0
```

```
f(x, y) = 0.0 ± 0.14
```

Even for more difficult functions:

```
f(x, y) = sin(x)/cos(x) - tan(y)
x = 1.0 ± 0.1
y = 1.0 ± 0.1
@show f(x, x);      # x and x are not independent
@show f(x, y);      # x and y are independent
```



```
f(x, x) = -2.220446049250313e-16 ± 0.0
f(x, y) = -2.2e-16 ± 0.48
```

The analytical procedure described in Equation 4.1 enables us to compute the mean and approximate the standard deviation of the output of a computation, given the mean and standard deviation for all inputs to the computation. In the special case that the input distribution is Gaussian and the computation is affine, the output distribution is Gaussian and fully defined by the mean and standard deviation. However, generally, the mean and standard deviation do not fully define the output distribution.

Caution

It is common to approximate the output distribution by a Gaussian for mathematical tractability, but sometimes this is not a good approximation. For example, if we apply $w = x^2$ to $x = 0.0 \pm 0.1$, the above procedure will yield $w = 0.0 \pm 0.0$. Clearly this is incorrect, as a small deviation from zero for x would lead to a positive value for w , but the procedure led us to believe that w would be exactly zero.

We next look at a Monte Carlo approach to estimating the distribution of the output of a non-linear computation with non-Gaussian inputs. Recall that a distribution can be approximated by a set of samples from the distribution. If we propagate each of the sample through the computation of interest, the resulting set of outputs faithfully represents the distribution of the output of that computation.

MonteCarloMeasurements.jl

The `MonteCarloMeasurements.jl` package implements a Julia data type that represents a distribution using a set of samples from that distribution. Computations using the distribution generate distributions that represent the output of the computation. Let's create a variable `x` with a uniform distribution between 1.0 and 2.0:

```
using MonteCarloMeasurements, Distributions
```

```
x = Particles(5000, Uniform(1.0, 2.0))
```

and compute using it:

```
w = x^2 + 2x - 3
```

```
2.33333 ± 1.45 Particles{Float64, 5000}
```

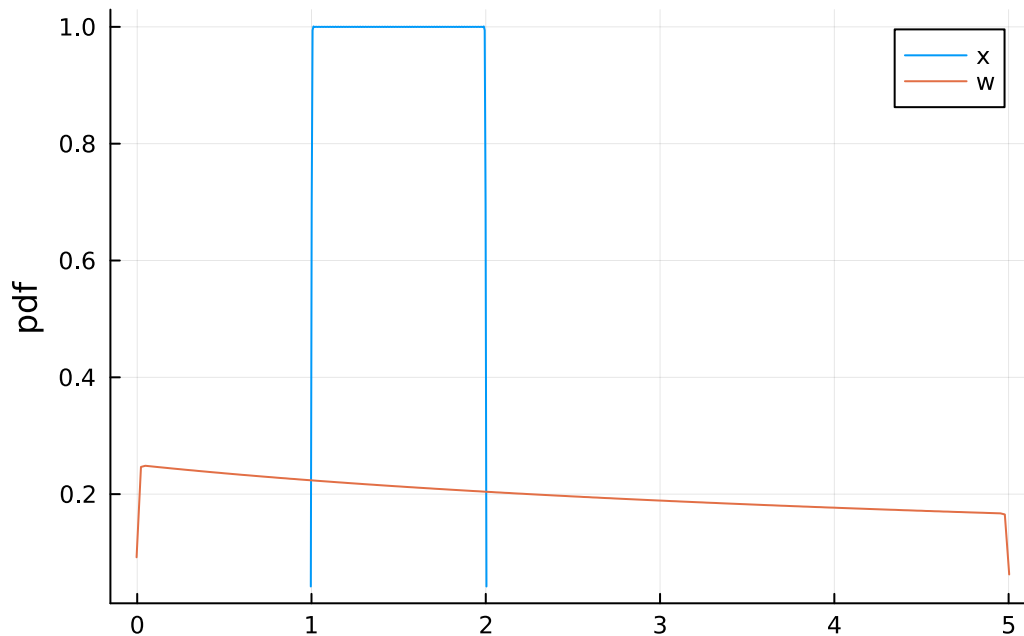


Figure 4.2: Probability density function estimate for $w = x^2 + 2x - 3$ with $x \sim \mathcal{U}(1, 2)$ using a Monte Carlo method with 5000 particles.

To see that x represents the entire distribution, we can plot an estimate of probability density of x and w :

```
using StatsPlots

density(x; label="x", bandwidth=0.001, ylabel="pdf")
density!(w; label="w", bandwidth=0.001)
```

The output is shown in Figure 4.2.

For more details, see [package documentation](#).

We can check that the example of $w = x^2$ with $x = 0.0 \pm 0.1$ is correctly handled:

```
x = Particles(5000, Normal(0.0, 0.1))
density(x^2; xlabel="w", ylabel="pdf", bandwidth=0.0001, legend=false)
```

As seen in Figure 4.3, the probability density indeed shows that w cannot be negative, and has a finite probability of being a small positive number.

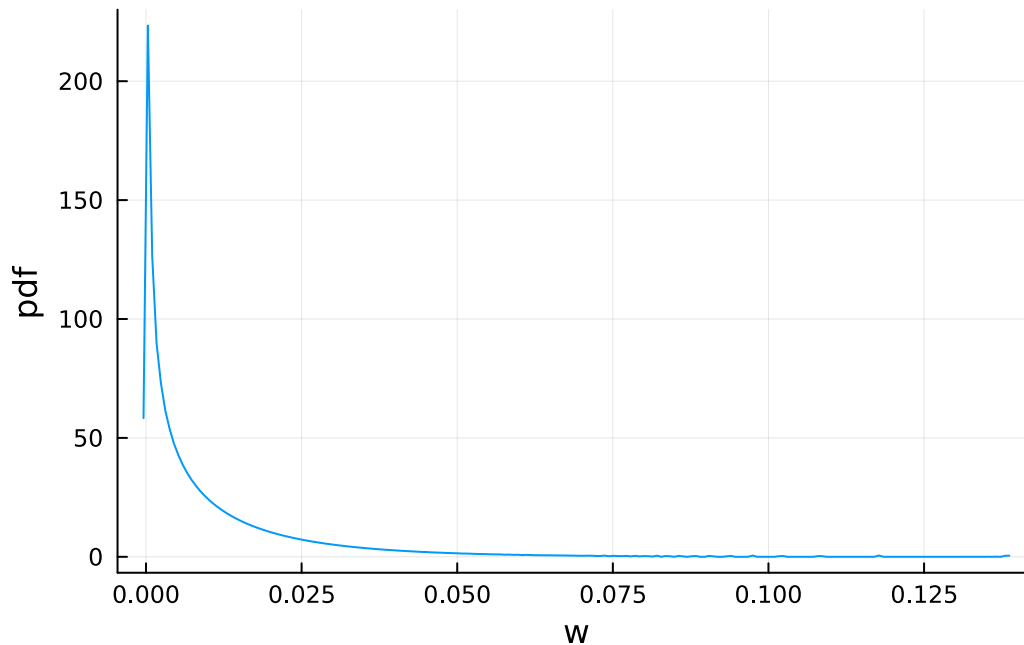


Figure 4.3: Probability density function estimate for $w = x^2$ with $x \sim \mathcal{N}(0, 0.1)$ using a Monte Carlo method with 5000 particles.

🔥 Caution

The $\pm$ operator is defined by both `Measurements.jl` and `MonteCarloMeasurements.jl`. If you load both in a REPL, you'll get a warning about it and only the first package loaded will successfully define the operator. To avoid confusion, we use $\pm$ from `Measurements.jl` in these notes, and use the `Particles()` constructor explicitly for `MonteCarloMeasurements.jl`.

Example: Netball-playing robot

Recall from Section 1.3 that the position of the ball could be computed as a function of time t using a simple ideal forward model Equation 1.2:

```
model1_xy(v, θ, t; x0=0.0, y0=0.0, g=-9.81) = [
    x0 + v * t * cosd(θ),
    y0 + v * t * sind(θ) + 0.5 * g * t^2
]
```

A more complete model (including drag, etc) was defined in Section 3.2:

```
using DifferentialEquations

function dudt(u, p, t; gx=-0.02, gy=-9.8, Cd=0.5, ρ=1.2, A=π*0.11^2)
    x, y, ẋ, ẏ = u
    v = sqrt(ẋ^2 + ẏ^2)
    D = 0.5 * Cd * ρ * A * v^2
    [ẋ, ẏ, -D * ẋ/v + gx, -D * ẏ/v + gy]
end

function model2_xy(v, θ, t; x0=0.0, y0=0.0)
    prob = ODEProblem(dudt, [x0, y0, v * cosd(θ), v * sind(θ)], (0.0, t))
    sol = solve(prob, Tsit5())
    sol(t)[1:2]
end
```

We can compute the position of the ball one second after the throw using each of the model and an uncertainty in launch velocity and angle:

```
using Measurements

@show model1_xy(10.0 ± 0.1, 45.0 ± 0.5, 1.0);
@show model2_xy(10.0 ± 0.1, 45.0 ± 0.5, 1.0);
```

```
model1_xy(10.0 ± 0.1, 45.0 ± 0.5, 1.0) = Measurement{Float64}[7.071 ± 0.094, 2.166 ± 0.094]
```

```
model2_xy(10.0 ± 0.1, 45.0 ± 0.5, 1.0) =
```

```
Measurement{Float64}[6.746 ± 0.087, 1.99 ± 0.088]
```

We could also estimate the uncertainty using the Monte Carlo method:

```
using MonteCarloMeasurements, Distributions

v = Particles(1000, Normal(10.0, 0.1))
θ = Particles(1000, Normal(45.0, 0.5))
@show model1_xy(v, θ, 1.0);
@show model2_xy(v, θ, 1.0);
```

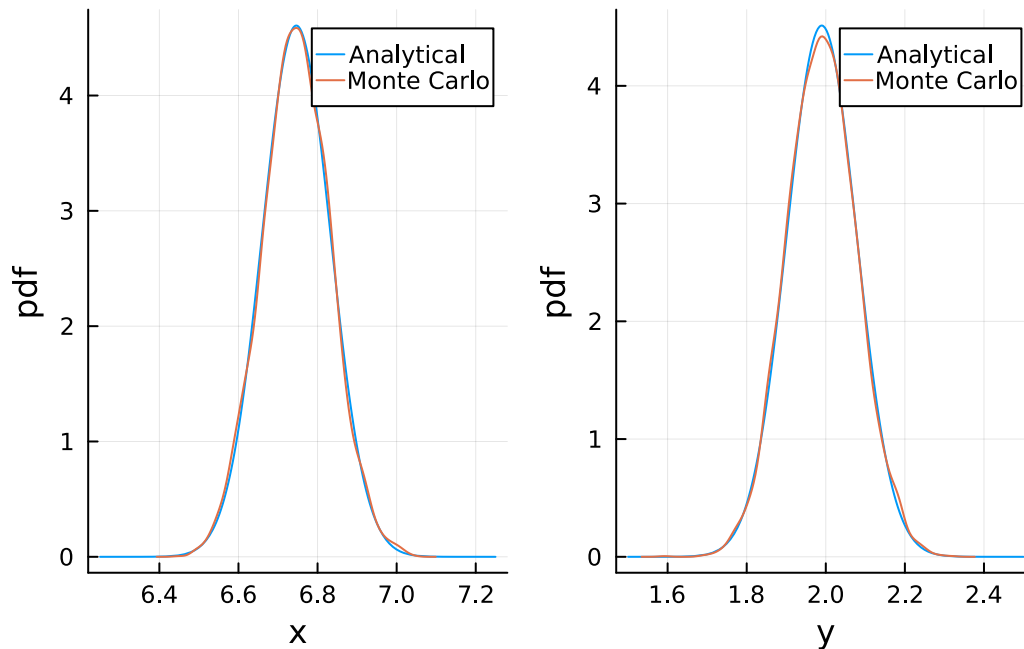


Figure 4.4: Probability density function estimate of the position of the ball after one second using the analytical and Monte Carlo methods.

```
model1_xy(v, θ, 1.0) = Particles{Float64, 1000}[7.07 ± 0.092, 2.17 ± 0.096]
model2_xy(v, θ, 1.0) =
Particles{Float64, 1000}[6.75 ± 0.085, 1.99 ± 0.09]
```

We can compare the pdf of the position based on the analytical method (with a Gaussian assumption) and the empirical pdf based on the Monte Carlo method in Figure 4.4. In this example, the linear approximation is good and the answers from both methods are very similar.

💡 Working with uncertainty – Summary

We studied two approaches of computing with uncertainty. The analytical approach uses linearization for mathematical tractability, and implicitly assumes Gaussian distributions. It is fast but approximate. It works well when the errors are small, noise is Gaussian, and the system being studied is well-behaved. The Monte Carlo approach is more general and works well in a wider variety of situations; however, it can be computationally intensive, depending on the number of particles required to adequately sample the uncertain quantities of interest.

4.3 Bayesian Inference

In the last section, we saw how to propagate uncertainty through computational models. While it is useful to predict uncertainty of output in a forward problem, uncertainty is of much more interest in inverse problems very often. In an inverse problem, we measure the output of a process with some uncertainty and wish to estimate some of the parameters (inputs) of the process. In our netball-playing robot example, we might measure (with noise) the position of the ball one second after a throw and wish to estimate the launch velocity and angle, or perhaps the gravitational acceleration and the drag coefficient. One way of solving such inverse problems is through *Bayesian inference*.

Recall Bayes' theorem:

$$\mathbb{P}(\Theta|\mathcal{D}) = \frac{\mathbb{P}(\mathcal{D}|\Theta)\mathbb{P}(\Theta)}{\mathbb{P}(\mathcal{D})}. \quad (4.2)$$

Here Θ represents the parameters to estimate and $\mathcal{D}$ represents the measured data. The term $\mathbb{P}(\Theta|\mathcal{D})$ is called the *posterior*, $\mathbb{P}(\mathcal{D}|\Theta)$ is the *likelihood*, $\mathbb{P}(\Theta)$ is the *prior*, and $\mathbb{P}(\mathcal{D})$ is the *marginal*. The prior contains our belief about the parameters before any measurements were made. The likelihood comes from domain knowledge – typically a forward model of what outcome to expect given parameters as input. The marginal is a scaling factor that ensures that the posterior is a valid probability distribution. It can theoretically be computed by integrating the model over all possible parameters:

$$\mathbb{P}(\mathcal{D}) = \int \mathbb{P}(\mathcal{D}|\Theta)\mathbb{P}(\Theta)d\Theta.$$

This is generally difficult to calculate analytically, and often intractable even numerically. Nevertheless, Bayes' rule gives us a recipe to compute the posterior $\mathbb{P}(\Theta|\mathcal{D})$, which contains our updated belief of what the parameter values are after making the measurement. If we wanted a single value for the parameters (rather than a probability distribution), we could extract a *point estimate* from the distribution. For example, we could pick the value of parameters that has the maximum posterior probability – such an estimate is known as *maximum a posteriori probability* (MAP).

To make Bayesian inference more concrete, let's look at a simple example next.

Example: Loaded coin toss

The loaded coin toss example is the “hello world!!!” of the Bayesian universe. Although simple, it has all the necessary features to understand how to apply Bayes' rule. More importantly, the mathematics is analytically tractable and hence illustrative.

Consider a loaded (biased) coin that has probability θ of yielding a heads and probability $(1 - \theta)$ of tails. We toss the coin N times and count the number of heads. We call the count n . We would like to estimate parameter θ .

In this simple example, we intuitively know a point estimate $\hat{\theta} = n/N$. We can check if the Bayesian inference leads to the same estimate. But the Bayesian inference will do more – it will also tell us how certain we are about the estimate by yielding a probability distribution rather than just a point estimate.

Recall that the outcome of a loaded coin toss is a Bernoulli random variable with success parameter θ (if we consider a heads outcomes as a success). If the coin is tossed N times, the number of heads n is a random variable which follows the binomial distribution:

$$\mathbb{P}(n|\theta) = \binom{N}{n} \theta^n (1 - \theta)^{N-n}.$$

With no prior information on how biased the coin is, we know that θ must be between 0 and 1 (since it is a probability). The natural prior to choose then is a uniform prior: $\theta \sim \mathcal{U}(0, 1)$. However, for reasons that will be clear shortly, we choose a beta distribution $\mathcal{B}(\alpha, \beta)$ as our prior: $\theta \sim \mathcal{B}(1, 1)$:

$$\mathbb{P}(\theta) = \frac{\theta^{\alpha-1} (1 - \theta)^{\beta-1}}{B(\alpha, \beta)},$$

where α and β are parameters of the distribution, and

$$B(\alpha, \beta) = \int_0^1 t^{\alpha-1} (1 - t)^{\beta-1} dt$$

is the well-known beta function that serves as a normalizing constant. The beta distribution $\mathcal{B}(1, 1)$ is identical to the uniform distribution $\mathcal{U}(0, 1)$, and so this swap does not change our prior belief about parameter θ .

Applying Bayes' theorem, we compute the posterior:

$$\begin{aligned} \mathbb{P}(\theta|n) &= \frac{\mathbb{P}(n|\theta)\mathbb{P}(\theta)}{\int_0^1 \mathbb{P}(n|\theta)\mathbb{P}(\theta)d\theta} \\ &= \frac{\binom{N}{n} \theta^n (1 - \theta)^{N-n} \frac{\theta^{\alpha-1} (1-\theta)^{\beta-1}}{B(\alpha, \beta)}}{\int_0^1 \binom{N}{n} \theta^n (1 - \theta)^{N-n} \frac{\theta^{\alpha-1} (1-\theta)^{\beta-1}}{B(\alpha, \beta)} d\theta} \\ &= \frac{\theta^{n+\alpha-1} (1 - \theta)^{N-n+\beta-1}}{\int_0^1 \theta^{n+\alpha-1} (1 - \theta)^{N-n+\beta-1} d\theta} \\ &= \frac{\theta^{n+\alpha-1} (1 - \theta)^{N-n+\beta-1}}{B(n + \alpha, N - n + \beta)}, \end{aligned} \tag{4.3}$$

which is a beta distribution $\mathcal{B}(n + \alpha, N - n + \beta)$. Since $\alpha = 1$ and $\beta = 1$ for the uniform prior, we have the posterior distribution:

$$\theta|n \sim \mathcal{B}(n + 1, N - n + 1).$$

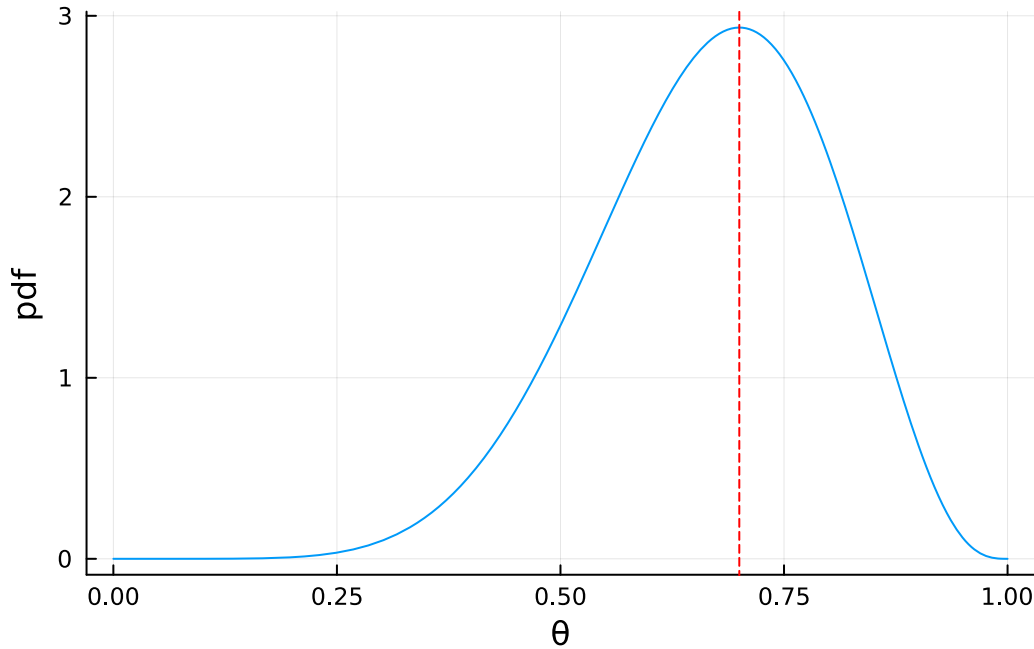


Figure 4.5: Posterior beta distribution $\mathcal{B}(8,4)$ for the coin toss example with $N = 10$ and $n = 7$. The dashed red line shows the mode of the distribution, which corresponds to the point estimate of 0.7 for parameter θ .

The mode (maximum probability value) of a beta distribution $\mathcal{B}(\alpha, \beta)$ is $(\alpha - 1)/(\alpha + \beta - 2)$. Since our posterior distribution is $\mathcal{B}(n + 1, N - n + 1)$, the MAP estimate:

$$\hat{\theta} = \frac{n + 1 - 1}{n + 1 + N - n + 1 - 2} = \frac{n}{N},$$

as we intuitively guessed. However, the posterior distribution contains much more information than just a point estimate n/N . For example, we can plot the probability distribution of $\theta|n$ as shown in Figure 4.5.

```
using Distributions, Plots
```

```
let posterior = Beta(8,4)
plot(x → pdf(posterior, x), 0, 1; xlabel="θ", ylabel="pdf", legend=false)
vline!([mode(posterior)]; linestyle=:dash, color=:red)
end
```


 Conjugate prior

A prior beta distribution combined with a binomial likelihood yielded a posterior beta distribution. If more measurements (coin tosses) become available later, the posterior beta distribution could be used as a prior in applying Bayes' theorem with exactly the same formulation. This was the reason we picked a beta distribution over a uniform distribution as a prior. Distributions that have the property that they remain unchanged (except for change in parameters) after a Bayesian update are known as *conjugate priors*. For a Bernoulli or binomial likelihood, a beta distribution is a conjugate prior. For a Gaussian likelihood, a Gaussian distribution is a conjugate prior. Many such likelihood–conjugate prior pairs are known.

In this simple example, we were able to analytically calculate the posterior distribution given the prior and likelihood. This is often not the case in real-world problems, as there are no closed form expressions for the posterior. One could resort to discretizing θ into a finite set and tracking the probability of each value using Bayes' theorem. For this example, that approach would work well too – but as the dimensionality as the problem increases, the parameter space grows rapidly and a discretized approach becomes computationally infeasible. The rest of this chapter explores other approaches to computing the posterior distribution efficiently.

4.4 Probabilistic Programming Languages

The mathematical description of probabilistic models becomes complicated as models grow in sophistication. *Probabilistic programming* is a programming paradigm in which such models are declared and used for inference automatically. This considerably eases the process of working with these models. Probabilistic programming languages (PPLs) are programming languages or DSLs specifically designed to aid in probabilistic programming. In this section, we take an in-depth look at `Turing.jl`, a PPL implemented as a DSL in Julia.

 PPLs

Many PPLs have been developed over the years. Some popular ones include:

- [STAN](#) – standalone language with bindings for Python, Julia, etc.
- [Turing.jl](#) – Julia package.
- [Gen](#) – Julia package.
- [PyMC](#) – Python package.
- [Pyro](#) – Python package.

The best way to learn `Turing.jl` is to look at a few examples.

Example: Loaded coin toss

Let's start with the simple example we studied in detail in Section 4.3, and see how to write it using a PPL:

```
using Turing

@model function cointoss(n, N)
     $\theta \sim \text{Beta}(1, 1)$       # prior on parameter  $\theta$ 
    n ~ Binomial(N,  $\theta$ )    # likelihood of observation n
end
```

Every Turing.jl model is declared using the macro `@model`. The name of our model is `cointoss` and it takes in data `(n, N)` where `n` is the number of heads and `N` is the total number of tosses. We then declare the prior distribution for parameter θ to be a `Beta(1, 1)` as we did in Section 4.3. We also declare that we expect the number of heads `n` to be binomial distributed with count `N` and success probability θ .

Once we have a model, we can run Bayesian inference on it:

```
chains = sample(cointoss(7, 10), NUTS(), 10000)
```

We invoked `cointoss` with `n=7` and `N=10`, and used a Monte Carlo sampling algorithm `NUTS()` to generate 10000 samples from the posterior distribution. We will explore various Monte Carlo sampling algorithms in the next section, but for now it is sufficient to know that NUTS is a state of the art Monte Carlo sampling algorithm that generates samples from the posterior distribution that we desire. Recall from Section 4.1 that having a procedure to generate any number of samples from a distribution is equivalent to knowing the distribution. The samples are stored in `chains` and can be used to ask questions about the parameter θ :

```
@show mean(chains[: $\theta$ ]);
@show mode(chains[: $\theta$ ]);
```

```
mean(chains[: $\theta$ ]) = 0.6683732143633375
```

```
mode(chains[: $\theta$ ]) = 0.6886168595634236
```

We can plot an estimate of the probability distribution of θ as shown in Figure 4.6.

```
histogram(chains[: $\theta$ ]; normalize=:pdf)
plot!(x → pdf(Beta(8,4), x), 0, 1;
       xlabel=" $\theta$ ", ylabel="pdf", legend=false, color=:red)
```

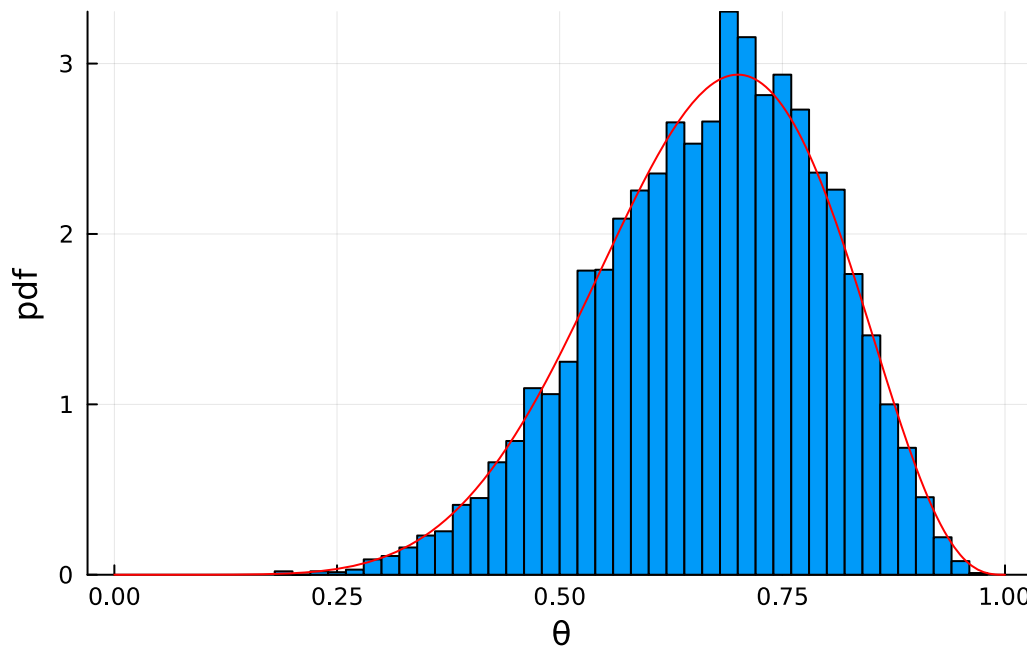


Figure 4.6: Estimate of probability distribution of θ for the coin toss example (with data modeled as a Binomial distribution) shown as a histogram. The analytic solution from Section 4.3 (shown in red) matches the estimate well.

In the above example, our data consisted of counts (n , N). If, instead, our data could be the individual coin toss outcomes:

```
tosses = [true, false, false, true, true, false, true, true, true, true]
```

where a `true` indicates a head and `false` a tail. The model could be written in terms of the Bernoulli outcomes:

```
@model function cointoss(data)
  theta ~ Beta(1, 1)          # prior on parameter theta
  for i in eachindex(data)
    data[i] ~ Bernoulli(theta) # likelihood of each toss outcome
  end
end

chains = sample(cointoss(tosses), NUTS(), 10000)
```

As before, we can get point estimates of θ :

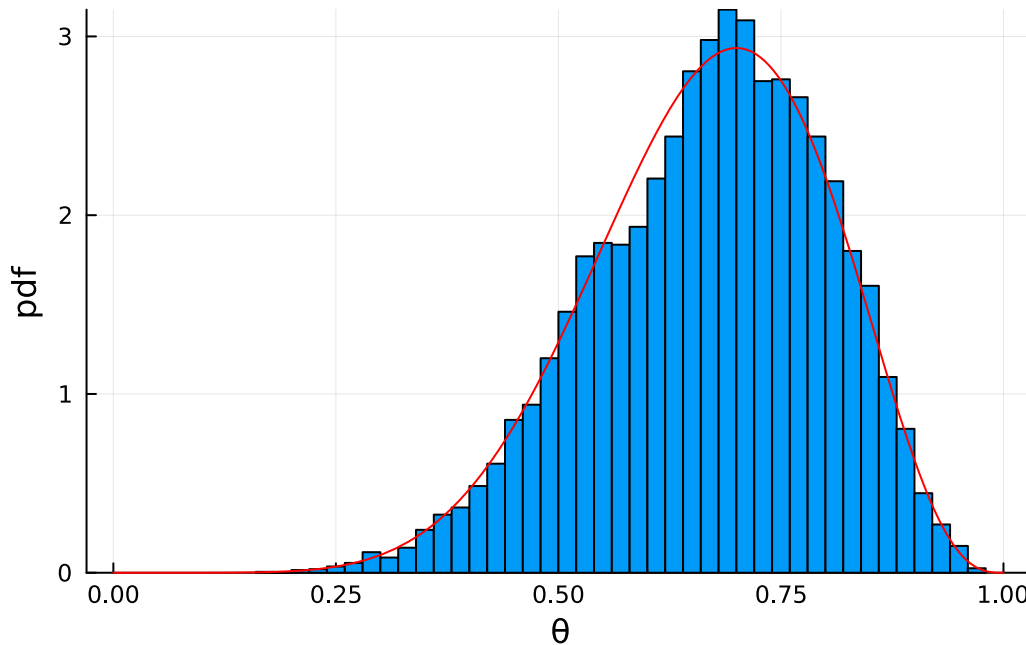


Figure 4.7: Estimate of probability distribution of θ for the coin toss example (with data modeled with Bernoulli distributions) shown as a histogram. The analytic solution from Section 4.3 (shown in red) matches the estimate well.

```
@show mean(chains[:θ]);
@show mode(chains[:θ]);
```

```
mean(chains[:θ]) = 0.6697799190066657
```

```
mode(chains[:θ]) = 0.6660780487661241
```

or plot the whole distribution (Figure 4.7).

Example: Netball-playing robot

Now that we know how to solve simple problems with `Turing.jl`, let's see if we can solve a problem involving our Netball-playing robot from Section 3.2. Recall the noisy data collected by the robot by throwing the ball once:

i	t	v	θ	x	y
1	0.0	8.94	63.4	0.0	0.0

2	0.1	8.94	63.4	0.404731	0.740347
3	0.2	8.94	63.4	0.800349	1.38249
4	0.3	8.94	63.4	1.17928	1.93416
5	0.4	8.94	63.4	1.56923	2.37687
6	0.5	8.94	63.4	1.9616	2.70341
7	0.6	8.94	63.4	2.34156	2.94354
8	0.7	8.94	63.4	2.72002	3.06893
9	0.8	8.94	63.4	3.09716	3.10138
10	0.9	8.94	63.4	3.47015	3.09218
11	1.0	8.94	63.4	3.86619	2.91732

We want to use this data to find the gravitational accelerations (g_x, g_y) and drag parameter K that govern the trajectory of the ball, as we did in Section 3.5.3. We reproduce Equation 3.7 here for easy reference:

$$\begin{aligned}\ddot{x} &= K\nu\dot{x} + g_x, \\ \ddot{y} &= K\nu\dot{y} + g_y,\end{aligned}$$

where $\nu = \sqrt{\dot{x}^2 + \dot{y}^2}$.

We characterize the sensors on the robot and find that the position measurement noise is ± 0.01 m and the initial velocity and launch angle noise is ± 0.01 m/s and $\pm 0.05^\circ$ respectively. We know that the acceleration g_y due to gravity must be -9.805 ± 0.040 m/s² (since the robot operates anywhere on Earth). We know that the g_x must be small (since the Netball court will not be constructed on a significant slope), say 0 ± 0.5 m/s², and that the drag parameter $-0.05 < K < 0$, based on experience with netballs.

We are now ready to write our model and sample from the posterior:

```
using DifferentialEquations

function dudt(u, p, t)
    x, y,  $\dot{x}$ ,  $\dot{y}$  = u
    gx, gy, K = p
     $\nu$  = sqrt( $\dot{x}^2 + \dot{y}^2$ )
    [ $\dot{x}$ ,  $\dot{y}$ ,  $K * \nu * \dot{x} + gx$ ,  $K * \nu * \dot{y} + gy$ ]
end

# initial state and parameters here are ignored and overridden later
prob = ODEProblem(dudt, zeros(4), (0.0, 1.0), zeros(3))

@model function netball(data, prob)
    gx ~ Normal(0.0, 0.5)
```

```

gy ~ Normal(-9.805, 0.040)
K ~ Uniform(-0.05, 0.0)
v ~ Normal(data.v[1], 0.01)
θ ~ Normal(data.θ[1], 0.05)
vx = v * cosd(θ)
vy = v * sind(θ)
sol = solve(prob, Tsit5()); u0=[0.0, 0.0, vx, vy], p=[gx, gy, K], saveat=data.t)
for i ∈ 1:length(data.x)
    data.x[i] ~ Normal(sol.u[i][1], 0.01)
    data.y[i] ~ Normal(sol.u[i][2], 0.01)
end
end

chains = sample(netball(data, prob), NUTS(), MCMCSerial(), 3000, 3;
↳ progress=false)

```

We ran the sampling serially 3 times to check that the results are consistent. This is considered good practice to ensure that the Monte Carlo procedure is working correctly. The ground truth values that were used to generate the data were $g_x = -0.02$, $g_y = -9.8$ and $K = 0.011$. We plot the posterior distributions for various parameters in Figure 4.8 and compare them against the ground truth.

Sample parameters from the posterior can be generated from the chains easily:

```

# 10 samples of (gx, gy, K, v, θ) without replacement
samples = Array(sample(chains, 10; replace=false))

```

```

10×5 Matrix{Float64}:
 0.0548047  -9.779   -0.0122263   8.93741   63.511
-0.0053433  -9.7435  -0.0129047   8.9373    63.3641
-0.00514497 -9.76899 -0.0127504   8.94439   63.4446
-0.144463   -9.87729 -0.00838813  8.94268   63.309
-0.0377445  -9.83091 -0.009506    8.93285   63.4318
-0.0267053  -9.80363 -0.0111473   8.94573   63.3381
-0.0491071  -9.81458 -0.0103366   8.94384   63.3864
 0.0155128  -9.7783  -0.0115875   8.92722   63.451
 0.0135167  -9.77513 -0.012282    8.93426   63.4179
-0.0450231  -9.8319  -0.0101924   8.94329   63.3653

```

We plot 100 sample trajectories using the posterior distribution in Figure 4.9. We see that the posterior distributions of the parameters are able to accurately predict the trajectory of

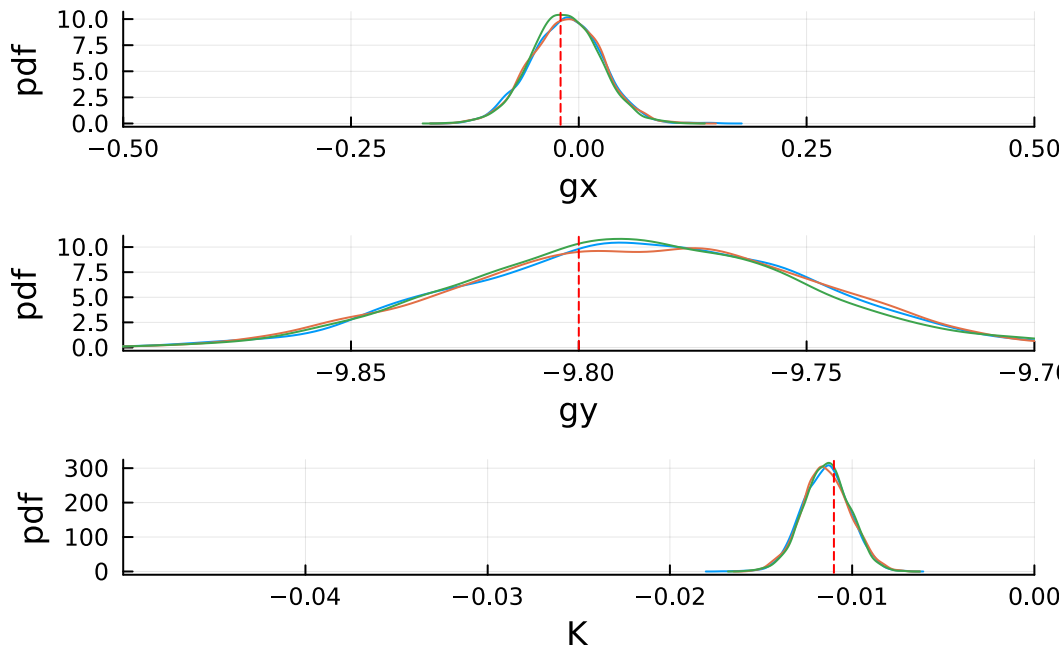


Figure 4.8: Posterior distributions for parameters estimated for the Netball-playing robot problem. The dashed red lines represent the ground truth.

the ball. This check is a good practice to ensure that the sample trajectories generated are consistent with the measured data.

💡 Tip

The examples here only scratch the surface of what is possible with PPLs. See [Turing Tutorials](#) for many examples implementing Gaussian mixture models, hidden Markov models, Bayesian regression, Bayesian neural networks, Bayesian time series analysis, etc. with `Turing.jl`.

4.5 Markov Chain Monte Carlo

In the previous section, we used NUTS as a Monte Carlo sampling algorithm to generate any number of samples from the posterior distribution. NUTS is one of many from a family of sampling algorithms known as *Markov Chain Monte Carlo* (MCMC). In this section, we seek to understand the basics of how and why these algorithms work. We will not delve into the mathematical details of the algorithms, but instead focus on the intuition behind them and how to use them reliably in practice.

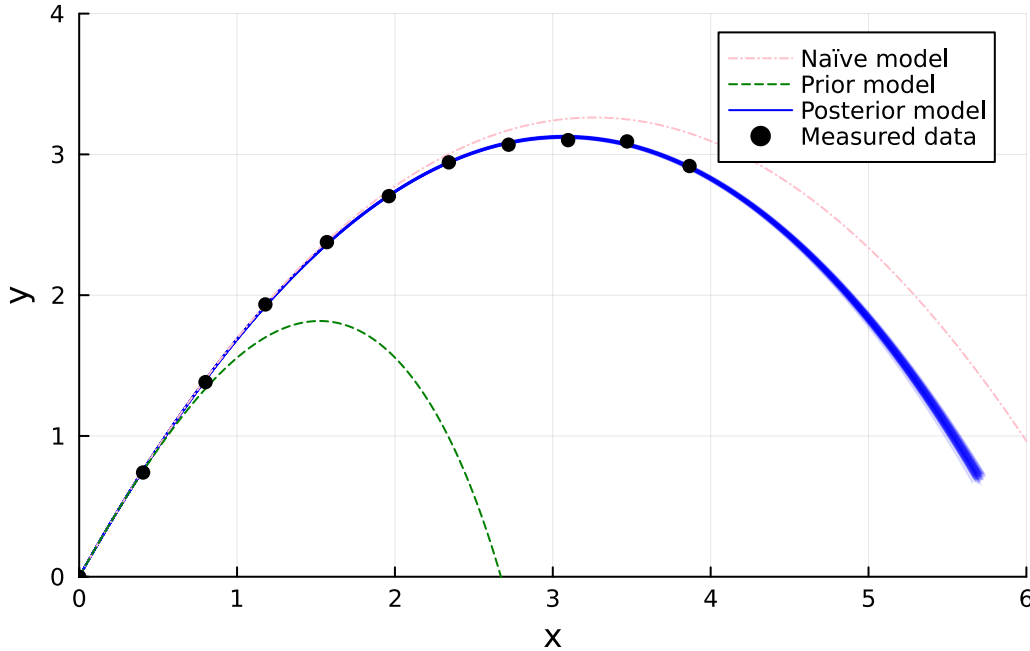


Figure 4.9: Sample trajectories with the posterior model for the Netball-playing robot problem. The trajectories with prior expected values for all parameters, and for a naïve model with no drag and nominal gravity are shown for comparison. The measured data used for the Bayesian inference is also shown.

Recall from Section 4.3 that computation of the posterior is often difficult because calculating the marginal is intractable. From Section 4.1, we know that having a sampling procedure that yields as many samples as we desire from a distribution is equivalent to knowing the distribution perfectly. Would it not be nice if we had a procedure to generate samples from the posterior without ever having to compute the marginal? That is exactly what MCMC methods do!

Most MCMC sampling algorithms consist of the following steps:

1. Pick some initial parameters Θ from the space of possible parameters.
2. Given Θ , generate a proposal for some new parameters Θ' .
3. Compute the ratio of posterior probabilities $\alpha = \mathbb{P}(\Theta'|\mathcal{D})/\mathbb{P}(\Theta|\mathcal{D})$.
4. Generate a uniform random number $u \sim \mathcal{U}(0, 1)$.
5. If $u \leq \alpha$, accept the proposal, i.e., $\Theta \leftarrow \Theta'$, otherwise do not change Θ .
6. Go to step 2.

After an initial warm-up number of steps of running this algorithm, the samples Θ generated are consistent with the posterior probability distribution $\mathbb{P}(\Theta|\mathcal{D})$. Various MCMC algorithms

essentially differ in step 2 (and sometimes step 3), where different procedures are used to generate proposal Θ' from Θ .

The computation of α in step 3 seems to require the posterior distribution $\mathbb{P}(\Theta|\mathcal{D})$. However, note that:

$$\begin{aligned}\alpha &= \frac{\mathbb{P}(\Theta'|\mathcal{D})}{\mathbb{P}(\Theta|\mathcal{D})} \\ &= \frac{\mathbb{P}(\mathcal{D}|\Theta')\mathbb{P}(\Theta')}{\mathbb{P}(\mathcal{D})} \times \frac{\mathbb{P}(\mathcal{D})}{\mathbb{P}(\mathcal{D}|\Theta)\mathbb{P}(\Theta)} \\ &= \frac{\mathbb{P}(\mathcal{D}|\Theta')\mathbb{P}(\Theta')}{\mathbb{P}(\mathcal{D}|\Theta)\mathbb{P}(\Theta)}.\end{aligned}$$

The marginal $\mathbb{P}(\mathcal{D})$ cancels out and hence to compute α , we do not have to compute the marginal! As long as the likelihood and prior can be computed for the current Θ and the proposed Θ' , the ratio α can be computed easily.

The algorithms are also memory efficient since we do not have to keep a history of previously generated samples for Θ . The next sample only depends on the current sample. This is the *Markov* property that gives this family of algorithms its name.

4.5.1 Metropolis algorithm

The Metropolis algorithm (Metropolis et al. 1953) is the simplest of the family of MCMC algorithms, but also one that converges slowly. It is rarely used in practice, with Metropolis-Hastings, Hamiltonian Monte Carlo or NUTS used for faster convergence. However, it is instructive to study it in some detail.

The Metropolis algorithm follows the steps for a general MCMC algorithm above. In step 2, a proposal for some new parameters Θ' is generated by sampling a proposal distribution $J(\Theta'|\Theta)$. The proposal distribution is required to be symmetric, i.e., $J(\Theta'|\Theta) = J(\Theta|\Theta')$. The proposal distribution favors proposed parameters Θ' that are similar to the current parameters Θ in some sense. For example, if the parameter $\Theta \in \mathbb{Z}$, then $J(\Theta'|\Theta)$ might be $\Theta \pm 1$ with equal probability. If instead parameter $\Theta \in \mathbb{R}$, one could pick $J(\Theta'|\Theta) \sim \mathcal{N}(\Theta, \lambda)$ for some hyperparameter λ .

Convergence

The choice of proposal distribution and hyperparameters affects convergence: If the proposal distribution favors too small a change, the samples generated are highly correlated and a larger number of samples are required to adequately represent the posterior distribution. If the proposal distribution favors too large a change, more proposals are likely to be rejected and hence sampling of the posterior distribution becomes inefficient.

Example: Loaded coin toss

Let's revisit the loaded coin toss example from Section 4.3 and implement a solution using the Metropolis algorithm.

We write a function `metropolis_cointoss(n, N,  $\lambda$ , nsamples)` to generate `nsamples` random samples from the posterior distribution of our estimate of the probability θ of heads, given we observed `n` heads when we tossed the coin `N` times. The hyperparameter λ controls the step size of the Metropolis algorithm.

```
using Distributions

function metropolis_cointoss(n, N,  $\lambda$ , nsamples)
    # function to compute  $f(\theta) = P(n|\theta)P(\theta)$ 
     $f(\theta) = 0 \leq \theta \leq 1 ? \text{pdf}(\text{Binomial}(N, \theta), n) : 0.0$ 
    # Metropolis algorithm
     $\theta = \text{rand}()$  # 1. Choose initial  $\theta$  from  $U(0,1)$ 
    samples = zeros(nsamples)
    for i ∈ 1:nsamples
         $\theta' = \theta + \lambda * \text{randn}()$  # 2. Draw proposal  $\theta'$  from  $N(\theta, \lambda)$ 
         $\alpha = f(\theta') / f(\theta)$  # 3. Compute ratio  $\alpha$ 
         $u = \text{rand}()$  # 4. Generate  $u$  randomly from  $U(0,1)$ 
         $u \leq \alpha \ \&\& \ (\theta = \theta')$  # 5. Accept  $\theta'$  if  $u \leq \alpha$ 
        samples[i] =  $\theta$ 
    end
    samples
end
```

The inner function $f(\theta)$ computes $P(n|\theta)P(\theta)$ (numerator of Equation 4.3). Since $P(\theta) \sim \mathcal{U}(0,1)$, it simply returns the binomial probability $P(n|\theta)$ if $0 \leq \theta \leq 1$ and zero otherwise.

We run the sampling algorithm with $\lambda=0.1$ and generate 10000 samples. We ignore the first 1000 samples to allow for warm-up, and plot a histogram of the samples in Figure 4.10. We see that the samples match the theoretical distribution well.

We repeat the experiment the same with $\lambda=0.001$ in Figure 4.11 and $\lambda=100$ in Figure 4.12. We see that a poor choice of step size λ leads to poor convergence. This highlights the importance of a good proposal distribution and hyperparameters for good convergence.

4.5.2 Metropolis-Hastings algorithm

The Metropolis-Hastings (MH) algorithm is a slight variant on the Metropolis algorithm where the proposal distribution $J(\Theta'|\Theta)$ need not be symmetric. This requires an additional term

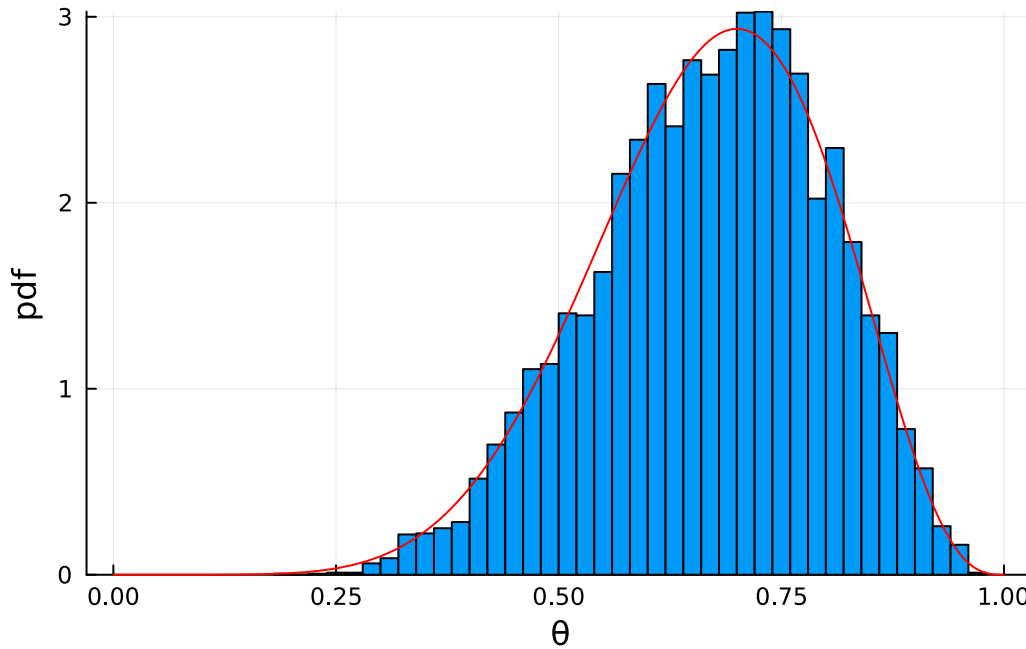


Figure 4.10: A histogram of the samples generated by the Metropolis algorithm for the loaded coin toss example with `metropolis_cointoss(7, 10, 0.1, 10000)`. The theoretical pdf is shown in red for comparison.

in the computation of α in step 3:

$$\alpha = \frac{\mathbb{P}(\mathcal{D}|\Theta')\mathbb{P}(\Theta')}{\mathbb{P}(\mathcal{D}|\Theta)\mathbb{P}(\Theta)} \times \frac{J(\Theta'|\Theta)}{J(\Theta|\Theta')}.$$

The Metropolis algorithm can be slow, especially if your initial starting point is poorly chosen. Asymmetric proposal distributions can speed up convergence.

For more information on the Metropolis-Hastings algorithm, see (Hastings 1970).

4.5.3 Hamiltonian Monte Carlo algorithm

Hamiltonian Monte Carlo (HMC) (Duane et al. 1987) modifies the step 2 in Metropolis-Hastings to generate proposals based on Hamiltonian dynamics. This involves simulating physics (solving Hamilton's differential equations) in parameter space to propose new parameters that are less correlated as compared to proposals generated by Gaussian random walks that are typically used in Metropolis-Hastings. This greatly improves convergence.

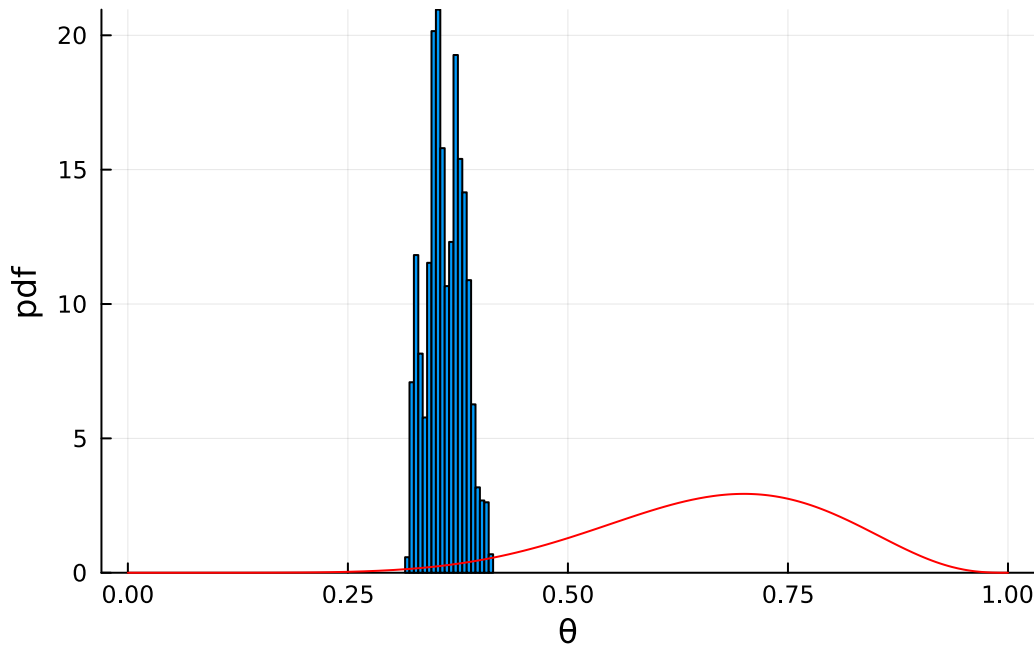


Figure 4.11: A histogram of the samples generated by the Metropolis algorithm for the loaded coin toss example with `metropolis_cointoss(7, 10, 0.001, 10000)`. The theoretical pdf is shown in red for comparison.

🔥 Limitations

- Since HMC is based on solving differential equations, it requires that the parameter space be continuous. HMC cannot be used for Bayesian inference with discrete parameters.
- Solving the differential equations also requires the posterior pdf to be differentiable. `Turing.jl` uses AD to automatically take gradients, and therefore requires the posterior pdf implementation to be compatible with Julia's AD packages. The backend package to be used for AD can be selected using `Turing.setadbackend()`. The default backend is `:forwarddiff`, but you can change it to `:zygote`, `:reversediff` or `:tracker`, if desired.

The exact details of HMC are beyond the scope of this course, but I highly recommend reading (Betancourt 2018) if you want to understand it more. For our purpose here, it is useful to visualize trajectories taken by the Hamiltonian physics in 2D space to understand the effect of hyperparameters. Some trajectories are shown in Figure 4.13. If the parameters are correctly chosen, proposed parameters sample the parameter space well as seen in Figure 4.13a and Figure 4.13d. Incorrectly choosing parameters (especially a parameter called *leapfrog steps*)

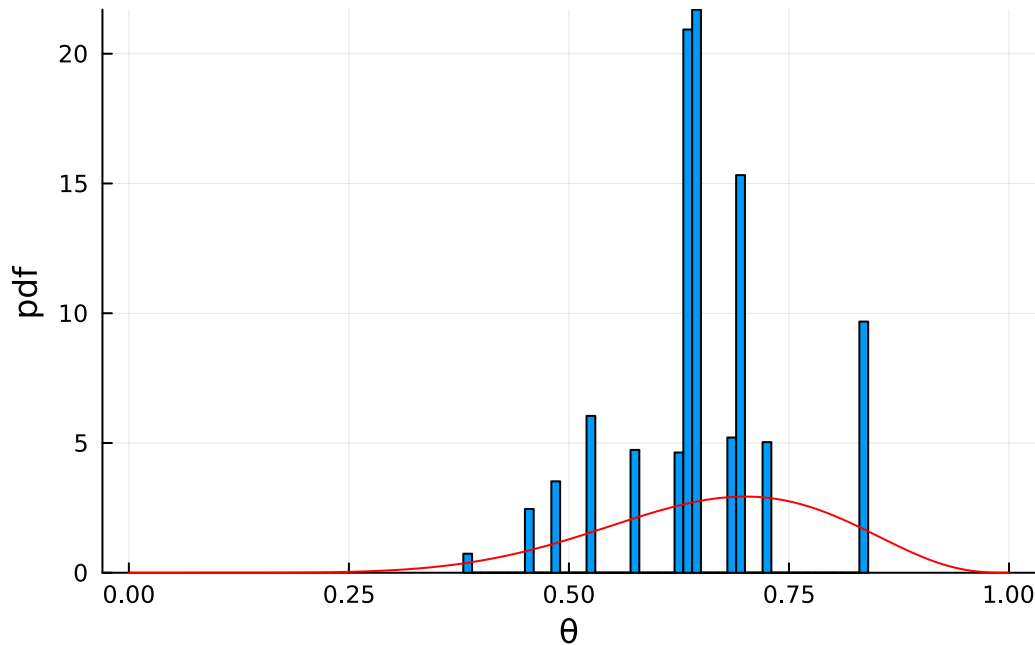


Figure 4.12: A histogram of the samples generated by the Metropolis algorithm for the loaded coin toss example with `metropolis_cointoss(7, 10, 100, 10000)`. The theoretical pdf is shown in red for comparison.

can cause the trajectory to U-turn onto itself as shown in Figure 4.13c – this leads to correlated parameter proposals that adversely affects convergence.

💡 Tip

Visit [MCMC online demo](#) for an excellent tool that animates various MCMC algorithms (including HMC) to help you understand the effect of various hyperparameters on proposals made by the algorithm.

4.5.4 NUTS algorithm

As we noted in the previous section, the HMC trajectories can U-turn on themselves if the parameters of HMC are not correctly set. The No U-Turn Sampler (NUTS) extends HMC by automatically tuning the leapfrog steps parameter to avoid the problem. The NUTS algorithm is considered state of art today, and requires very little tuning.

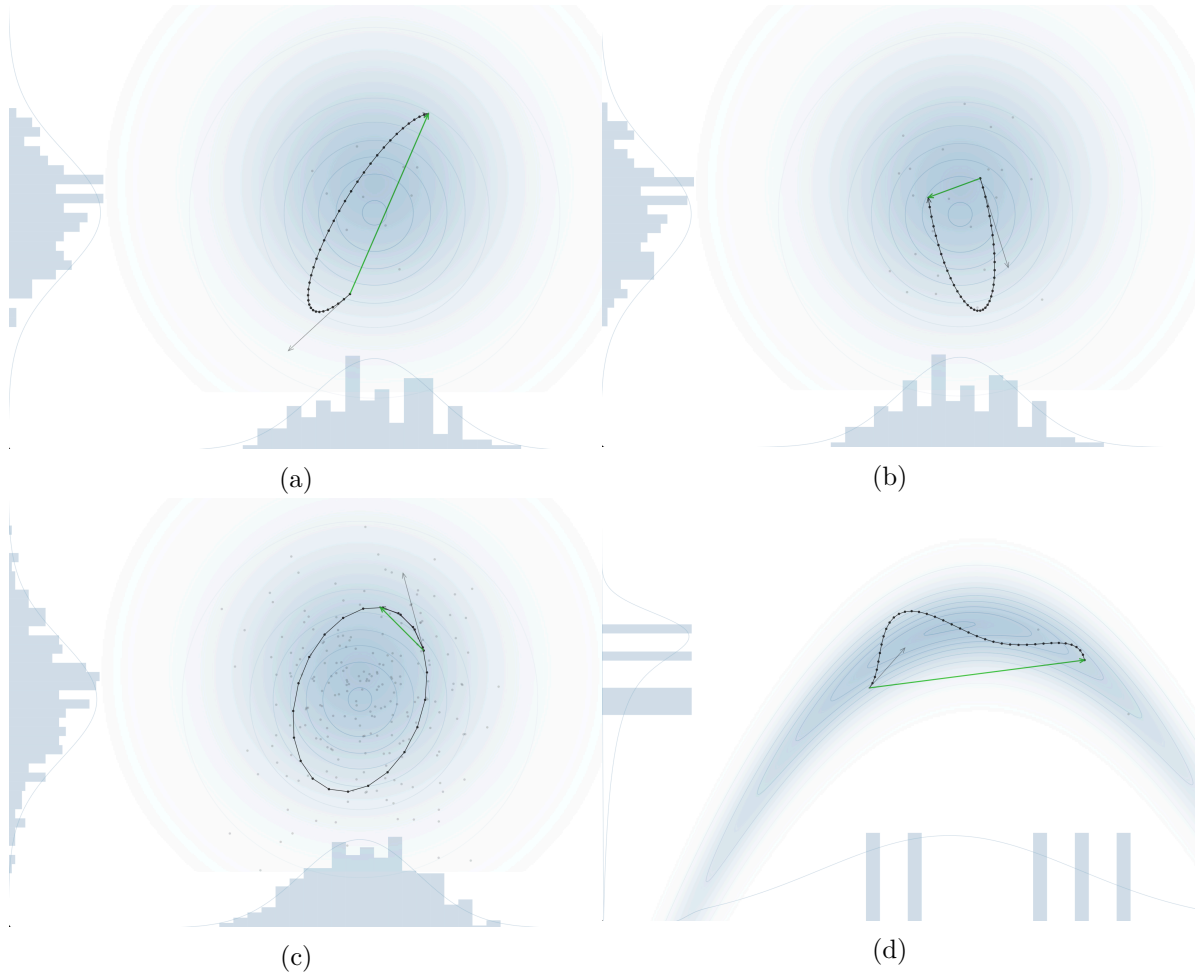


Figure 4.13: Sample trajectories on different 2D parameter spaces using HMC. The grey arrow shows the direction of motion from the initial point Θ in the parameter space. The black dotted line shows the trajectory, and the orange arrow shows the proposed change $\Theta \rightarrow \Theta'$.

 Limitations

NUTS is a variant of HMC and has the same limitations as HMC – it requires the parameter space to be continuous and the posterior pdf to be differentiable.

For more information on NUTS, see (Hoffman, Gelman, et al. 2014).

4.5.5 Diagnostics

MCMC sampling algorithms are very powerful tools to solve Bayesian inference problems. However, if improperly tuned, they can give incorrect answers. It is therefore important to check that the samples they produce are reasonable. We outline a few standard good practices that help to check the samples.

When we sample the posterior using a MCMC algorithm, `Turing.jl` displays many parameter statistics. For example, for the loaded coin toss problem in Section 4.4, we declared a probabilistic model:

```
@model function cointoss(n, N)
    θ ~ Beta(1, 1)      # prior on parameter θ
    n ~ Binomial(N, θ)  # likelihood of observation n
end
```

Let's sample this model using the HMC sampler with a step size of 0.1 and 10 leapfrog steps. We request the sampling to be run 3 times, each with 10000 samples. The `MCMCSerial()` argument asks the 3 sampling tasks to be run in serial on the same CPU; you can use `MCMCThreads()` to run each on a different CPU threads for speed.

```
chains = sample(cointoss(7, 10), HMC(0.1, 10), MCMCSerial(), 10000, 3;
    ↪ progress=false)
```

Chains MCMC chain (10000×11×3 Array{Float64, 3}):

```
Iterations          = 1:1:10000
Number of chains    = 3
Samples per chain   = 10000
Wall duration       = 1.31 seconds
Compute duration    = 1.09 seconds
parameters          = θ
internals            = lp, n_steps, is_accept, acceptance_rate, log_density,
hamiltonian_energy, hamiltonian_energy_error, numerical_error, step_size, nom_step_size
```

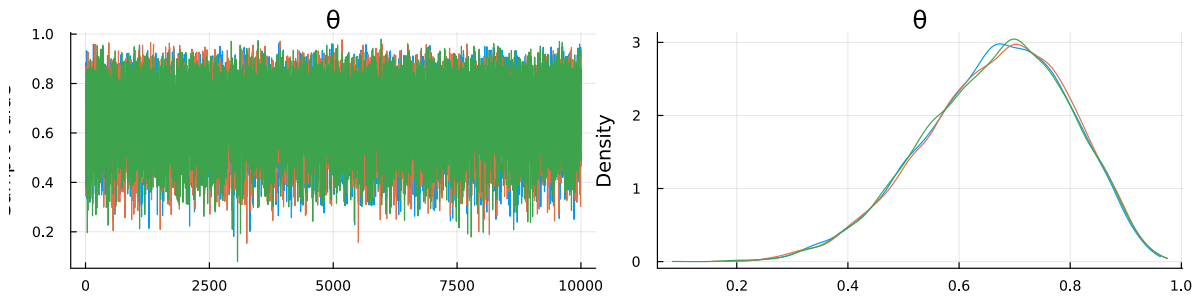


Figure 4.14: Chain diagnostic plot for a properly sampled posterior.

Summary Statistics

parameters	mean	std	mcse	ess_bulk	ess_tail	rhat ...
Symbol	Float64	Float64	Float64	Float64	Float64	Float64 ...
θ	0.6672	0.1305	0.0008	28974.0595	25233.3903	1.0002 ...

1 column omitted

Quantiles

parameters	2.5%	25.0%	50.0%	75.0%	97.5%
Symbol	Float64	Float64	Float64	Float64	Float64
θ	0.3916	0.5803	0.6772	0.7632	0.8901

The output shows summary statistics for each of the parameters estimated. For this example, we have θ as the only parameter. It's mean value is within the 0–1 range that we would expect it to be, and so reasonable. A non-zero std value suggests that the distribution is not degenerate, which is good. The `ess_bulk` statistic measures the effective number of samples. We asked for $3 \times 10000 = 30000$ samples. A `ess_bulk` value close to this number says that the samples are not very correlated and therefore sample the parameter space well. The `rhat` value close to 1.0 is an indication that the 3 chains have converged and have similar distributions. We can further plot the chains (using `plot(chains)`) to visually inspect them as shown in Figure 4.14. We observe that all 3 chains have similar density estimates and visually look similar.

Contrast this against a poorly tuned HMC sampling run with a step size of 0.01 and 2 leapfrog steps:

```
chains = sample(cointoss(7, 10), HMC(0.01, 2), MCMCSerial(), 10000, 3;
  ↪ progress=false)
```

Chains MCMC chain (10000×11×3 Array{Float64, 3}):

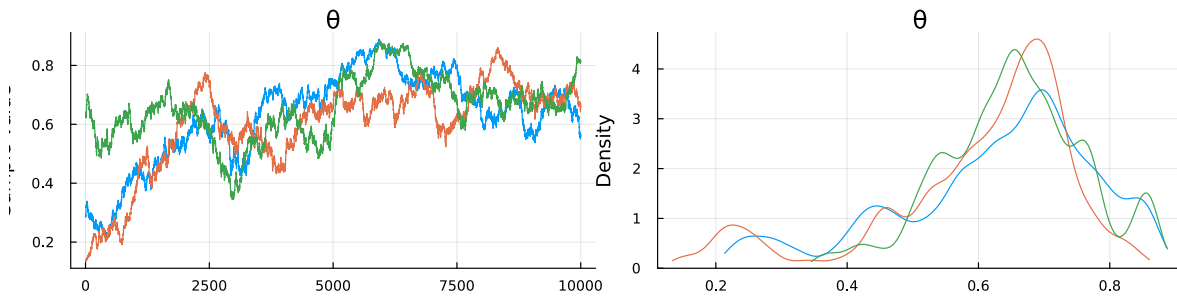


Figure 4.15: Chain diagnostic plot for a poorly sampled posterior (step size too small).

```

Iterations      = 1:1:10000
Number of chains = 3
Samples per chain = 10000
Wall duration    = 0.58 seconds
Compute duration = 0.5 seconds
parameters       =  $\theta$ 
internals        = lp, n_steps, is_accept, acceptance_rate, log_density,
hamiltonian_energy, hamiltonian_energy_error, numerical_error, step_size, nom_step_size

```

Summary Statistics

parameters	mean	std	mcse	ess_bulk	ess_tail	rhat	e ...
Symbol	Float64	Float64	Float64	Float64	Float64	Float64	...
θ	0.6269	0.1412	0.0175	65.8527	70.9266	1.4398	...

1 column omitted

Quantiles

parameters	2.5%	25.0%	50.0%	75.0%	97.5%
Symbol	Float64	Float64	Float64	Float64	Float64
θ	0.2412	0.5607	0.6534	0.7154	0.8528

We see that the `ess_bulk` statistic is very small – this suggests highly correlated samples in the chain. The `rhat` statistic is not close to 1.0, and this suggests that the 3 chains have not converged to the same distribution. Plotting the chains (Figure 4.15) gives further insights. We see that the density estimates are poor and inconsistent across the chains. We also see that the samples are correlated and inconsistent across the chains.

We try another HMC sampling run with a much larger step size of 2.0 and 10 leapfrog steps:

```
chains = sample(cointoss(7, 10), HMC(2.0, 10), MCMCSerial(), 10000, 3;
  ↪ progress=false)
```

Chains MCMC chain (10000×11×3 Array{Float64, 3}):

```
Iterations          = 1:1:10000
Number of chains    = 3
Samples per chain    = 10000
Wall duration       = 0.77 seconds
Compute duration    = 0.69 seconds
parameters          = θ
internals            = lp, n_steps, is_accept, acceptance_rate, log_density,
hamiltonian_energy, hamiltonian_energy_error, numerical_error, step_size, nom_step_size
```

Summary Statistics

parameters	mean	std	mcse	ess_bulk	ess_tail	rhat	e ...
Symbol	Float64	Float64	Float64	Float64	Float64	Float64	...
θ	0.7051	0.1180	0.0124	87.4813	89.9831	1.2126	...

1 column omitted

Quantiles

parameters	2.5%	25.0%	50.0%	75.0%	97.5%
Symbol	Float64	Float64	Float64	Float64	Float64
θ	0.3795	0.6440	0.7298	0.7910	0.8734

Again, we see that the `ess_bulk` statistic is very small. Plotting the chains (Figure 4.16) gives further insights. We see that the density estimates are poor and inconsistent across the chains, and that samples mostly remain constant and the distribution is inconsistent across the chains.

Such diagnostics must always be checked when we use MCMC algorithms to sample from the posterior. In addition, domain knowledge of the problem you're solving can help with sanity checks on answers. For example, in Figure 4.9, we used samples from the chain to generate trajectories to see that they were consistent with the training data. This was a form of a *posterior predictive check* where we simulate samples from the posterior distribution and compare them against observed data. Posterior predictive checking is a standard practice in statistical inference.

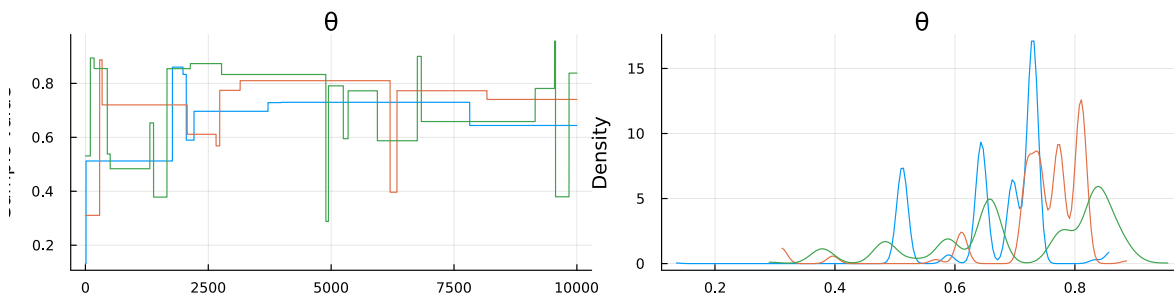


Figure 4.16: Chain diagnostic plot for a poorly sampled posterior (step size too large).



Takeaway

Always check the results of sampling from the posterior before believing them. Badly written models or wrongly tuned samplers can silently give incorrect results.

4.6 Variational Inference

In Section 4.5, we studied MCMC techniques to obtain samples from the posterior distribution of a Bayesian inference problem. In this section, we explore the possibility of finding analytical approximations to the posterior distribution instead.

The posterior distribution $\mathbb{P}(\Theta|\mathcal{D})$ is generally difficult to obtain because the marginal computation is often intractable. In variational inference, we wish to approximate the posterior distribution with a surrogate distribution with pdf $Q^*(\Theta)$ without having to evaluate the marginal. To find a good approximation, we minimize the Kullback–Leibler (KL) divergence between the surrogate and posterior distributions:

$$Q^*(\Theta) = \arg \min_Q \text{KL}(Q(\Theta) \parallel \mathbb{P}(\Theta|\mathcal{D})). \quad (4.4)$$

The KL divergence is a distance measure between two distributions; it is non-negative with a zero distance signifying an exact match. The minimization is ideally over all possible distributions, but in practice over some family of distributions. The KL divergence is the expected

value of the log-ratio of pdfs, where the expectation is taken over the surrogate distribution:

$$\begin{aligned}
 \mathbb{KL}(Q(\Theta) \parallel \mathbb{P}(\Theta|\mathcal{D})) &= \mathbb{E} \left[\log \frac{Q(\Theta)}{\mathbb{P}(\Theta|\mathcal{D})} \right] \\
 &= \mathbb{E} \left[\log \frac{Q(\Theta)\mathbb{P}(\mathcal{D})}{\mathbb{P}(\mathcal{D}|\Theta)\mathbb{P}(\Theta)} \right] \\
 &= -\mathbb{E} \left[\log \frac{\mathbb{P}(\mathcal{D}|\Theta)\mathbb{P}(\Theta)}{Q(\Theta)} \right] + \mathbb{E} [\mathbb{P}(\log \mathcal{D})] \\
 &= -\mathcal{L}(Q) + \log \mathbb{P}(\mathcal{D}),
 \end{aligned}$$

where we define:

$$\mathcal{L}(Q) = \mathbb{E} \left[\log \frac{\mathbb{P}(\mathcal{D}|\Theta)\mathbb{P}(\Theta)}{Q(\Theta)} \right]$$

is called the evidence lower bound (ELBO).

i Why is it called ELBO?

The term $\log \mathbb{P}(\mathcal{D})$ is called the evidence. Since the KL distance must be non-negative, $\log \mathbb{P}(\mathcal{D}) \geq \mathcal{L}(Q)$. In other words, $\mathcal{L}(Q)$ is a lower bound on the evidence, and hence the term *evidence lower bound* or ELBO.

The optimization in Equation 4.4 can be rewritten in terms of the ELBO:

$$\begin{aligned}
 Q^*(\Theta) &= \arg \min_Q -\mathcal{L}(Q) + \log \mathbb{P}(\mathcal{D}) \\
 &= \arg \max_Q \mathcal{L}(Q) - \log \mathbb{P}(\mathcal{D}) \\
 &= \arg \max_Q \mathcal{L}(Q),
 \end{aligned} \tag{4.5}$$

where we drop the term $\log \mathbb{P}(\mathcal{D})$ in the last step since the term does not depend on Q and hence plays no role in determining the optimal Q .

As seen above, we are interested in finding a distribution Q that maximizes the ELBO. This problem is not easy to solve if Q is any distribution, but if we choose a family of distributions parametrized by Φ , i.e., $Q(\Theta) \equiv Q(\Theta; \Phi)$, then the ELBO $\mathcal{L}(Q) \equiv \mathcal{L}(\Phi)$. Equation 4.5 then becomes:

$$\Phi^* = \arg \max_{\Phi} \mathcal{L}(\Phi),$$

and $Q^*(\Theta) \equiv Q(\Theta; \Phi^*)$. This problem can be solved using gradient descent, provided the family of distributions $Q(\Theta; \Phi)$ is differentiable with respect to Φ .

i Automatic Differentiation Variational Inference (ADVI)

Turing.jl provides an implementation of automatic differentiation based variational inference (Kucukelbir et al. 2016). It uses a transformed mean-field Gaussian (multivariate Gaussian distribution with independent components) as the family of surrogate distributions for variational inference. The parameters Φ are simply all the means and variances for all components of the distribution. To use ADVI, we define a probabilistic model as we learnt in Section 4.4, and call `vi()` on the model with `ADVI()` as the algorithm:

```
posterior = vi(model, ADVI())
```

The return value `posterior` is a multivariate distribution that can be used just like any other [multivariate distribution](#) in the Julia ecosystem.

Example: Loaded coin toss

Let's again look at the simple coin toss example we studied in Section 4.3 and Chapter 4 to see how to solve it using variational inference with Turing.jl and ADVI:

```
using Turing

@model function cointoss(n, N)
    θ ~ Beta(1, 1)      # prior on parameter θ
    n ~ Binomial(N, θ)  # likelihood of observation n
end

posterior = vi(cointoss(7, 10), ADVI())
```

Now that we have a `posterior` distribution, we can plot the pdf and compare it with the analytical solution in Figure 4.17. We see that the ADVI pdf approximates the analytical solution well, but not perfectly.

Example: Netball-playing robot

Finally, we solve the Netball-playing robot problem from Section 4.4 using ADVI:

```
using DifferentialEquations

function dudt(u, p, t)
```

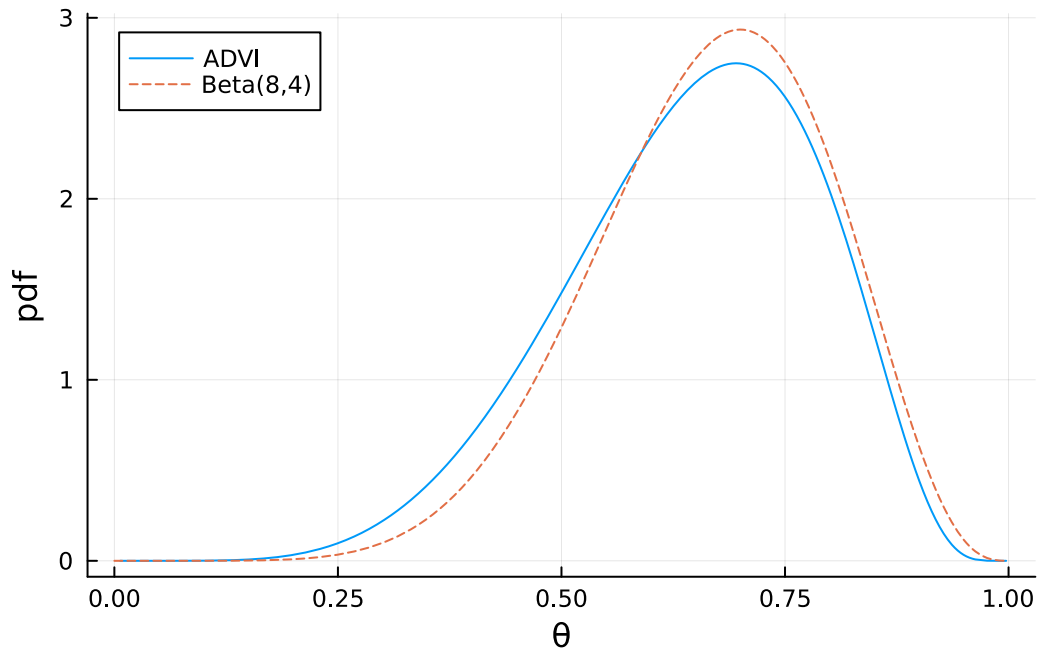


Figure 4.17: Posterior distribution pdf from the loaded coin toss example computed using ADVI. The $\text{Beta}(8,4)$ pdf is the analytical solution and is shown here for comparison.

```

x, y,  $\dot{x}$ ,  $\dot{y}$  = u
gx, gy, K = p
v = sqrt( $\dot{x}^2 + \dot{y}^2$ )
[ $\dot{x}$ ,  $\dot{y}$ , K * v *  $\dot{x}$  + gx, K * v *  $\dot{y}$  + gy]
end

# initial state and parameters here are ignored and overridden later
prob = ODEProblem(dudt, zeros(4), (0.0, 1.0), zeros(3))

@model function netball(data, prob)
  gx ~ Normal(0.0, 0.5)
  gy ~ Normal(-9.805, 0.040)
  K ~ Uniform(-0.05, 0.0)
  v ~ Normal(data.v[1], 0.01)
   $\theta$  ~ Normal(data. $\theta$ [1], 0.05)
  vx = v * cosd( $\theta$ )
  vy = v * sind( $\theta$ )

```

```

sol = solve(prob, Tsit5()); u0=[0.0, 0.0, vx, vy], p=[gx, gy, K], saveat=data.t)
for i ∈ 1:length(data.x)
    data.x[i] ~ Normal(sol.u[i][1], 0.01)
    data.y[i] ~ Normal(sol.u[i][2], 0.01)
end
end

# we use ADVI(1,10000) to increase the number of iterations for convergence
posterior = vi(netball(data, prob), ADVI(1,10000))

```

Sample parameters from the posterior can be generated from the chains:

```

# 10 samples of (gx, gy, K, v, θ)
rand(posterior, 10)'

```

```

10×5 adjoint(::Matrix{Float64}) with eltype Float64:
-0.00890098  -9.77648  -0.0110504  8.9379  63.4009
-0.0211553   -9.8066   -0.0115379  8.94025  63.4731
-0.0101865   -9.78412   -0.0112533  8.94304  63.3658
-0.0183907   -9.76969   -0.0118235  8.93671  63.4787
 0.0049666   -9.79258   -0.011364   8.94217  63.358
-0.0310766   -9.79787   -0.0110708  8.94266  63.434
 0.000411725 -9.78454   -0.0110291  8.93657  63.4039
-0.0198476   -9.76448   -0.011544   8.94481  63.4102
-0.00398879  -9.77658   -0.0115005  8.9441  63.4063
-0.0192849   -9.78779   -0.0118971  8.94633  63.3922

```

We plot 100 sample trajectories using the posterior distribution in Figure 4.18. We see that the posterior distributions of the parameters are able to accurately predict the trajectory of the ball. This check is a good practice to ensure that the sample trajectories generated are consistent with the measured data.

4.7 Summary

In this chapter, we learned how to work with quantities whose values are not perfectly known. When the inputs of a computation (e.g., neural networks or differential equation solvers) have uncertainty, we expect the outputs to have uncertainty too. Propagating uncertainty through computations requires gradients, which can be obtained through AD.

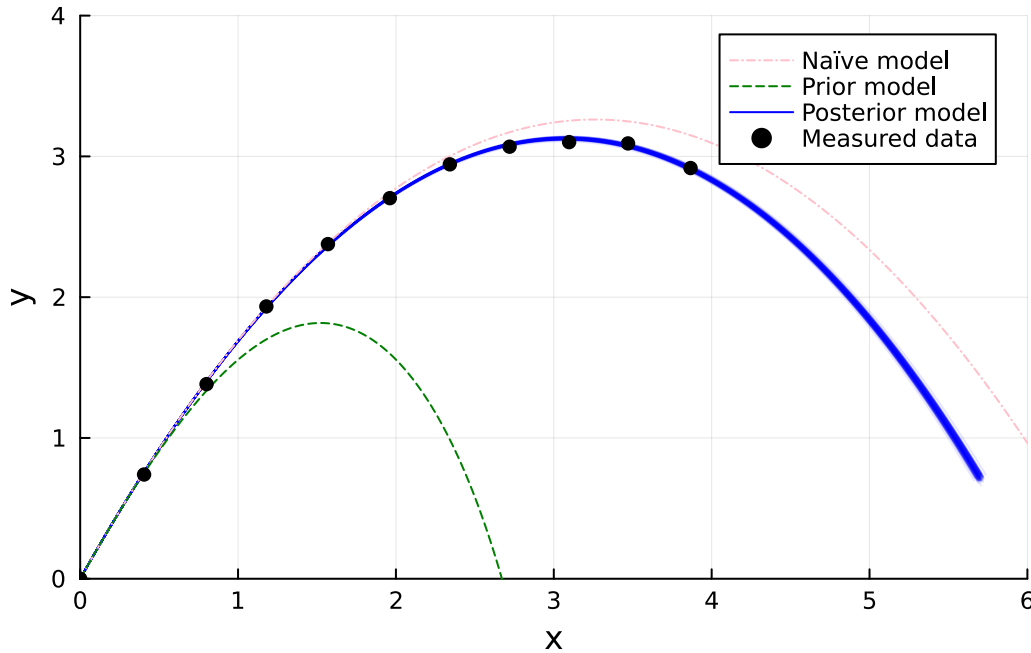


Figure 4.18: Sample trajectories with the posterior model for the Netball-playing robot problem using variational inference. The trajectories with prior expected values for all parameters, and for a naïve model with no drag and nominal gravity are shown for comparison. The measured data used for the Bayesian inference is also shown.

Forward propagation of uncertainty through a computation allows us to understand how the output of a process behaves. However, often we are interested in inverse problems where the output is observed with some noise and the inputs (or parameters) have to be estimated. Such problems are theoretically formulated as Bayesian inference problems, and in some cases can be solved analytically. However, in the vast majority of cases, Bayesian inference requires numerical methods. To aid with application of numerical methods, PPLs have been developed to describe Bayesian inference problems effectively.

We learned that probability distributions can be described in many ways – as a pdf or cdf, or as a sampling procedure that generates random variates from that distribution. The latter representation forms the basis of Monte Carlo algorithms, and is a very powerful way to deal with uncertainty. For example, MCMC methods enable us to perform Bayesian inference in a computationally efficient way, and form the bedrock of modern statistical machine learning.

Another approach to solving Bayesian inference problems is to approximate the posterior distribution using parametrized surrogates. This approach leads to variational inference methods. We learned how to use AD-based variational inference to solve Bayesian inference problems.

5 Parting Words

While the key ideas behind automatic differentiation, non-linear optimization, Bayesian inference and Monte Carlo methods are not new, they have been traditionally reserved for niche scientific problems and not been widely applied in main-stream software. The past decade has seen rapid development in machine learning. A lot of that development heavily leverages optimization theory, automatic differentiation and Bayesian inference. Even more recently, there is a strong move to combine domain knowledge with machine learning to reduce the need for training data and computational power. This requires combining traditional numerical software with differentiable and probabilistic methods. Computer languages and software packages are rapidly evolving to support these new trends. In this course, we touched upon a variety of tools and techniques to support differentiable and probabilistic programming and saw numerous examples of how to use them.

My aim was to provide you with a basic understanding of differentiable and probabilistic programming. The focus was on learning to use available tools effectively, and being able to solve real-world problems using them. The theory and tools in this field are evolving rapidly, and we will no doubt see advances in years to come. Hopefully, this course has given you the fundamentals required to appreciate and understand the theoretical advances, and the confidence to learn new tools and techniques as they become available.

I hope you enjoyed the learning journey and are excited to apply what you learned in your field!

References

- Arrowsmith, S. J., D. T. Trugman, J. MacCarthy, K. J. Bergen, D. Lumley, and M. B. Magnani. 2022. “Big Data Seismology.” *Reviews of Geophysics* 60 (2): e2021RG000769. <https://doi.org/https://doi.org/10.1029/2021RG000769>.
- Baker, Nathan, Frank Alexander, Timo Bremer, Aric Hagberg, Yannis Kevrekidis, Habib Najm, Manish Parashar, et al. 2019. “Workshop Report on Basic Research Needs for Scientific Machine Learning: Core Technologies for Artificial Intelligence,” February. <https://doi.org/10.2172/1478744>.
- Bergen, Karianne J, and Gregory C Beroza. 2018. “Detecting earthquakes over a seismic network using single-station similarity measures.” *Geophysical Journal International* 213 (3): 1984–98. <https://doi.org/10.1093/gji/ggy100>.
- Betancourt, Michael. 2018. “A Conceptual Introduction to Hamiltonian Monte Carlo.” <https://arxiv.org/abs/1701.02434>.
- Chen, Ricky T. Q., Yulia Rubanova, Jesse Bettencourt, and David Duvenaud. 2019. “Neural Ordinary Differential Equations.” <https://arxiv.org/abs/1806.07366>.
- Duane, Simon, A. D. Kennedy, Brian J. Pendleton, and Duncan Roweth. 1987. “Hybrid Monte Carlo.” *Physics Letters B* 195 (2): 216–22. [https://doi.org/https://doi.org/10.1016/0370-2693\(87\)91197-X](https://doi.org/https://doi.org/10.1016/0370-2693(87)91197-X).
- Giordano, Mosè. 2016. “Uncertainty Propagation with Functionally Correlated Quantities.” <https://arxiv.org/abs/1610.08716>.
- Hastings, W. K. 1970. “Monte Carlo Sampling Methods Using Markov Chains and Their Applications.” *Biometrika* 57 (1): 97–109. <http://www.jstor.org/stable/2334940>.
- Hoffman, Matthew D, Andrew Gelman, et al. 2014. “The No-u-Turn Sampler: Adaptively Setting Path Lengths in Hamiltonian Monte Carlo.” *J. Mach. Learn. Res.* 15 (1): 1593–623.
- Holl, Philipp, Vladlen Koltun, and Nils Thuerey. 2022. “Scale-Invariant Learning by Physics Inversion.” <https://arxiv.org/abs/2109.15048>.
- Karniadakis, George Em, Ioannis G. Kevrekidis, Lu Lu, Paris Perdikaris, Sifan Wang, and Liu Yang. 2021. “Physics-Informed Machine Learning.” *Nature Reviews Physics* 3 (6): 422–40. <https://doi.org/10.1038/s42254-021-00314-5>.
- Karpatne, Anuj, Gowtham Atluri, James H. Faghmous, Michael Steinbach, Arindam Banerjee, Auroop Ganguly, Shashi Shekhar, Nagiza Samatova, and Vipin Kumar. 2017. “Theory-Guided Data Science: A New Paradigm for Scientific Discovery from Data.” *IEEE Transactions on Knowledge and Data Engineering* 29 (10): 2318–31. <https://doi.org/10.1109/TKDE.2017.2720168>.
- Karpatne, Anuj, Imme Ebert-Uphoff, Sai Ravela, Hassan Ali Babaie, and Vipin Kumar. 2019.

- “Machine Learning for the Geosciences: Challenges and Opportunities.” *IEEE Transactions on Knowledge and Data Engineering* 31 (8): 1544–54. <https://doi.org/10.1109/TKDE.2018.2861006>.
- Kucukelbir, Alp, Dustin Tran, Rajesh Ranganath, Andrew Gelman, and David M. Blei. 2016. “Automatic Differentiation Variational Inference.” <https://arxiv.org/abs/1603.00788>.
- Liu, Lu, Jelmer M Wolterink, Christoph Brune, and Raymond NJ Veldhuis. 2021. “Anatomy-Aided Deep Learning for Medical Image Segmentation: A Review.” *Physics in Medicine & Biology* 66 (11): 11TR01.
- Metropolis, Nicholas, Arianna W. Rosenbluth, Marshall N. Rosenbluth, Augusta H. Teller, and Edward Teller. 1953. “Equation of State Calculations by Fast Computing Machines.” *The Journal of Chemical Physics* 21 (6): 1087–92. <https://doi.org/10.1063/1.1699114>.
- Rackauckas, Christopher, Yingbo Ma, Julius Martensen, Collin Warner, Kirill Zubov, Rohit Supekar, Dominic Skinner, and Ali Ramadhan. 2020. “Universal Differential Equations for Scientific Machine Learning.” *arXiv Preprint arXiv:2001.04385*.
- Rackauckas, Christopher, Yingbo Ma, Julius Martensen, Collin Warner, Kirill Zubov, Rohit Supekar, Dominic Skinner, Ali Ramadhan, and Alan Edelman. 2021. “Universal Differential Equations for Scientific Machine Learning.” <https://arxiv.org/abs/2001.04385>.
- Raissi, M., P. Perdikaris, and G. E. Karniadakis. 2019. “Physics-Informed Neural Networks: A Deep Learning Framework for Solving Forward and Inverse Problems Involving Non-linear Partial Differential Equations.” *Journal of Computational Physics* 378: 686–707. <https://doi.org/https://doi.org/10.1016/j.jcp.2018.10.045>.
- Sauer, Timothy. 2011. *Numerical Analysis*. 2nd ed. Pearson.
- Swann, WH. 1969. “A Survey of Non-Linear Optimization Techniques.” *FEBS Letters* 2 (S1): S39–55.
- Vamathevan, Jessica, Dominic Clark, Paul Czodrowski, Ian Dunham, Edgardo Ferran, George Lee, Bin Li, et al. 2019. “Applications of Machine Learning in Drug Discovery and Development.” *Nature Reviews Drug Discovery* 18 (6): 463–77. <https://doi.org/10.1038/s41573-019-0024-5>.
- Wang, Sifan, Shyam Sankaran, Hanwen Wang, and Paris Perdikaris. 2023. “An Expert’s Guide to Training Physics-Informed Neural Networks.” <https://arxiv.org/abs/2308.08468>.
- Willard, Jared, Xiaowei Jia, Shaoming Xu, Michael Steinbach, and Vipin Kumar. 2022. “Integrating Scientific Knowledge with Machine Learning for Engineering and Environmental Systems.” *ACM Comput. Surv.* 55 (4). <https://doi.org/10.1145/3514228>.